Programming GPUs

A collage of several tutorials

Introduction to General Purpose GPU Computing

HPC Course @CINECA
9-11 June 2021

Sergio Orlandini
s.orlandini@cineca.it

Luca Ferraro
l.ferraro@cineca.it

GPGPU
Introduction

Alan Gray

EPCC

The University of Edinburgh

What is a GPU

- **G**raphics **P**rocessing **U**nit

  a device equipped with an highly parallel microprocessor (*thousands of cores*) and a private memory with very high bandwidth (about 900GB/s)

- born in '90 as a response to the growing demand for high definition 3D rendering graphic applications (gaming, animations, etc)

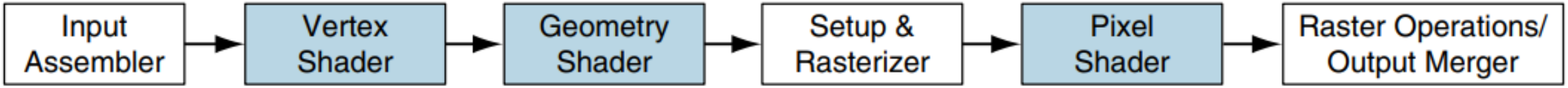
GPU are specialized for parallel intensive computation

GPUs are designed to render complex 3D scenes composed of ***milions of data*** points/vertex at high frame rates (60-120 FPS)

The rendering process requires a set of transformations based on linear algebra operations and (mostly local) filters

- the ***same set* of operations** are **applied *on each data point*** of the scene
- each operation is ***independent*** with respect to data
- all **operations are performed *in parallel*** using a **huge number of *threads*** which process all data independently

Graphics Logical Pipeline



- The 3D application sends the GPU a sequence of vertices grouped into geometric primitives—points, lines, triangles, and polygons.
- Vertex shader programs map the position of triangle vertices onto the screen, altering their position, color, or orientation.
- Geometry shader programs operate on geometric primitives (such as lines and triangles) defined by multiple vertices, changing them or generating additional primitives.
- Pixel fragment shaders each “shade” one pixel.
- The GPU hardware creates a new independent thread to execute a vertex, geometry, or pixel shader program for every vertex, every primitive, and every pixel fragment.

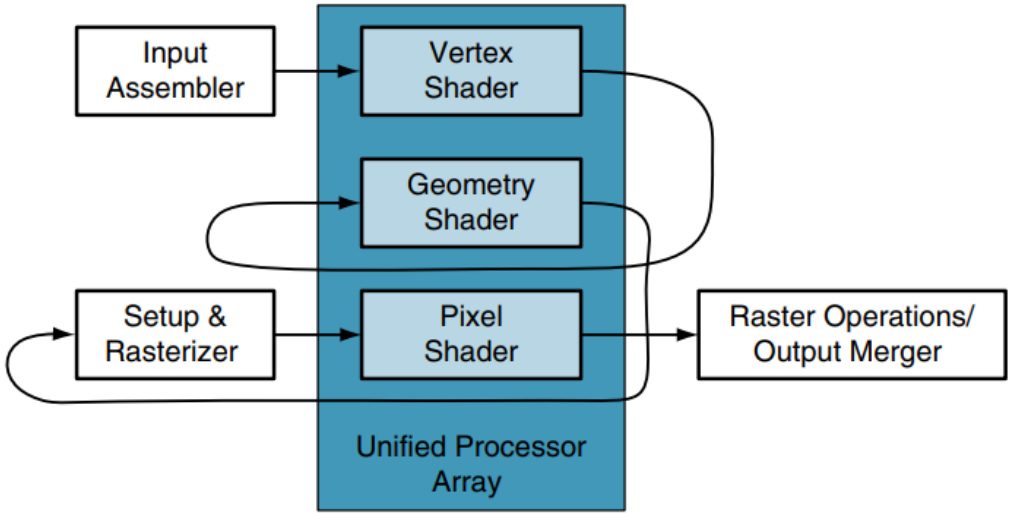
Unified GPU architectures

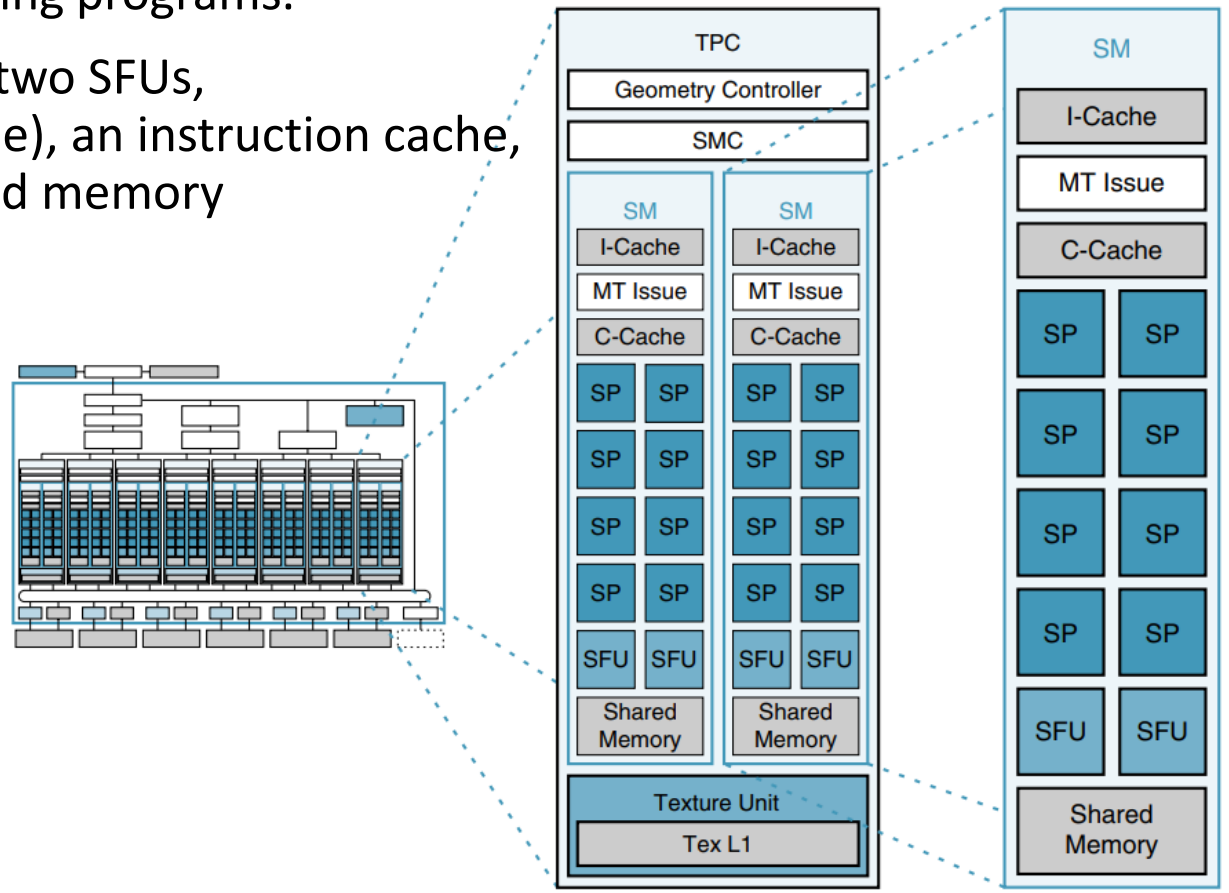
Unified GPU architectures are based on a parallel array of many programmable processors.

They unify vertex, geometry, and pixel shader processing and parallel computing on the same processors, unlike earlier GPUs which had separate processors dedicated to each processing type.

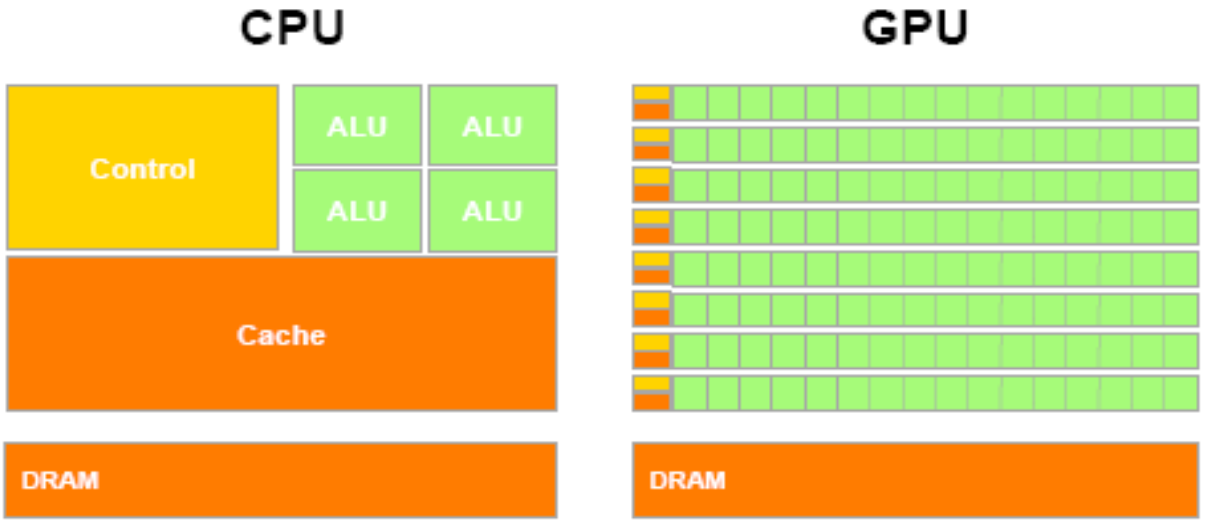
The Texture Processing Cluster is compoded by two Streaming Multiprocessors that execute vertex, geometry, and pixel-fragment shader programs and parallel computing programs.

Each SM consists of eight SP, i.e. thread processor cores, two SFUs,
a multithreaded instruction fetch and issue unit (MT issue), an instruction cache,
a read-only constant cache, and a 16KB read/write shared memory





GPU vs CPU: different philosophies

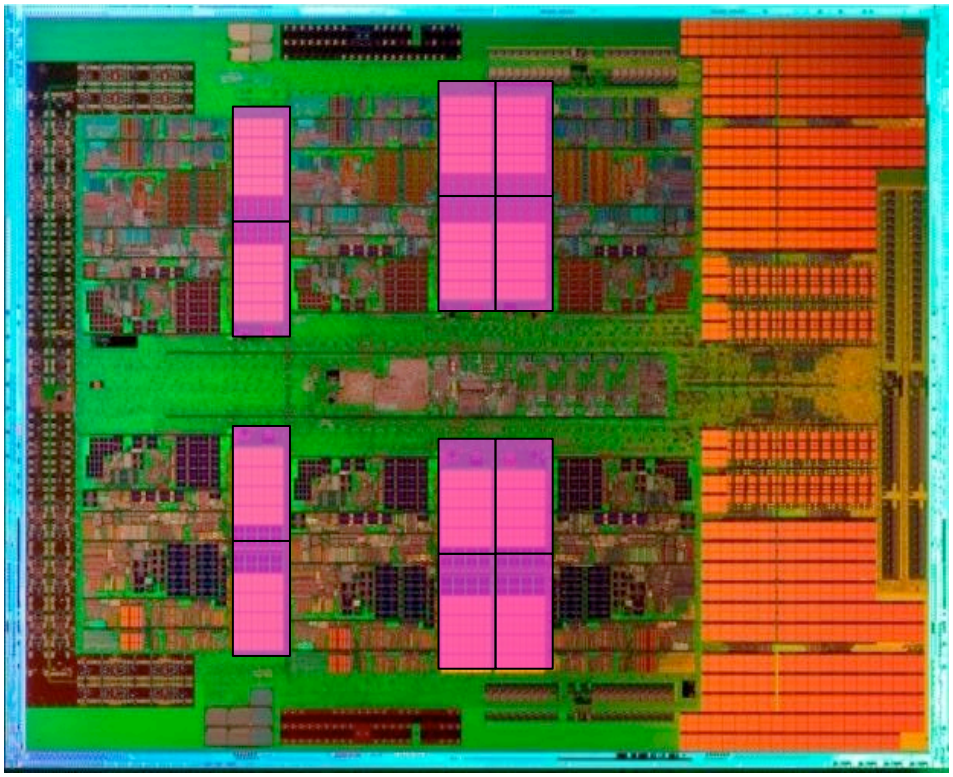


- Design of CPUs optimized for sequential code performance:
- multi-core
- sophisticated control logic unit
- large cache memories to reduce access latencies

Design of GPUs optimized for the execution of large number of threads dedicated to floating-points calculations:

- many-cores (several hundreds)
- minimized the control logic in order to manage leightweight threads and maximize execution throughput
- taking advantage of large number of threads to overcome long-latency memory accesses

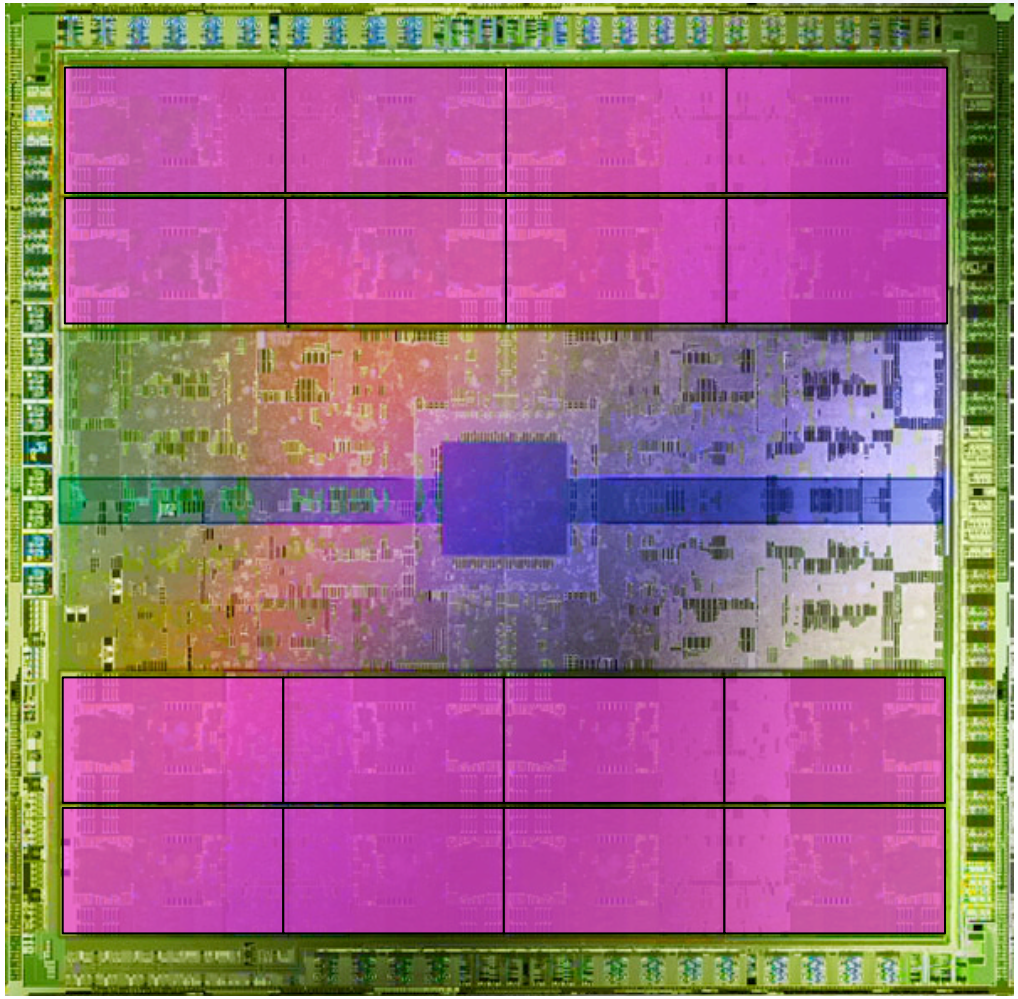
AMD 12-core CPU



= compute unit
(= core)

- Not much space on CPU is dedicated to compute

NVIDIA Fermi GPU



= compute unit
(= SM
= 32 CUDA cores)

- GPU dedicates much more space to compute
  - At expense of caches, controllers, sophistication etc

NVIDIA HPC GPU Solutions

Model	FP32 [TFlops]	cores	RAM [GB]	Bandwidth [GB/s]	Link
Kepler K40	4.3	2280	12 GDDR5	240	PCIe 3.0 (15.8 GB/s)
Pascal P100	10.6	3584	16 HBM2	720	PCIe 3.0 (15.8 GB/s)
Volta V100	15.7	5120	16/32 HBM2	900	PCIe 3.0 (15.8 GB/s)
Ampere A100	19.5	8192	40 HBM2	1500	PCIe 4.0 (31.6 GB/s)

- Tesla serie is the NVIDIA top gamma GPU solution for HPC
  - the GeForce series are for gaming
- HBM2: gen2 high-performance RAM interface for 3D-stacked DRAM with Error-correcting (ECC)

AMD HPC GPU Solutions

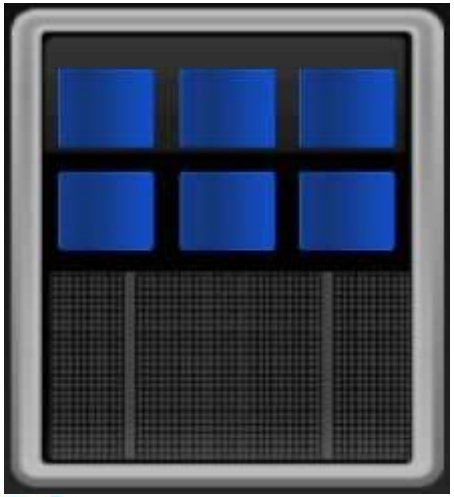
Model	FP32 [TFlops]	cores	RAM [GB]	Bandwidth [GB/s]	Link
Radeon MI8	8.2	4096	4 HBM	512	PCIe 3.0 (15.8 GB/s)
Radeon MI25	12.3	4096	16 HBM2	484	PCIe 3.0 (15.8 GB/s)
Radeon MI50	13.4	3840	16 HBM2	1024	PCIe 3.0 (15.8 GB/s)
Radeon MI100	32.1	7680	32 HBM2	1200	PCIe 4.0 (31.6 GB/s)

- VEGA Processor is the AMD top gamma GPU solution for HPC
  - the Radeon RX VEGA/500/400 series are for gaming
- HBM2: gen2 high-performance RAM interface for 3D-stacked DRAM with Error-correcting (ECC)
- Infinity Fabric™ Links per GPU deliver up to 200 GB/s of peer-to-peer bandwidth
- very high TDP factor sustainable on HPC server blades

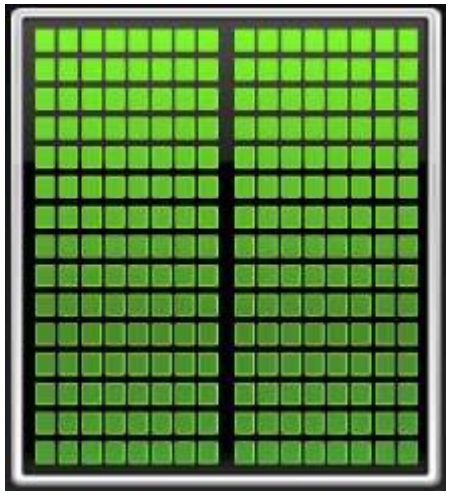
GPGPU Programming Model

- General Purpose GPU Programming relates to use GPU computational power to solve problems other than graphics
- CPU and GPU are **separate devices** with **separate memory** space addresses
- GPU is seen as an auxilirary coprocessor equiped with thousands of cores and a high bandwidth memory
- They should work togheter for best benefit and performances

CPU



GPU

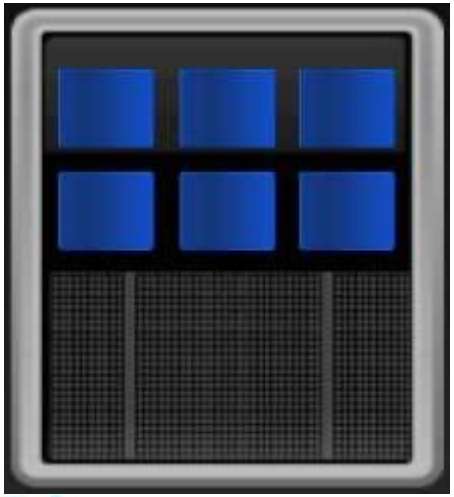


GPGPU Programming Model

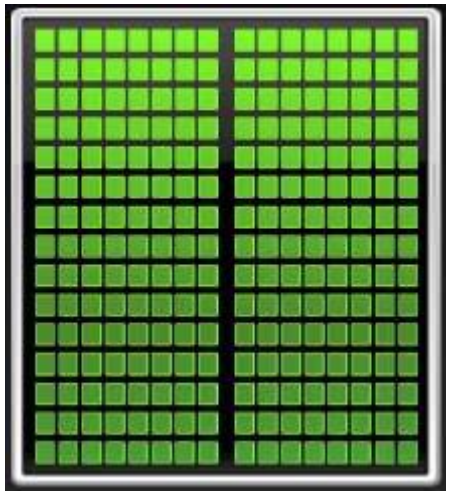
- Optimized for low-latency accesses to caches data sets
- Control logic for out-of-order and speculative execution
- Best for serial or event driven tasks

- Optimized for data-parallel, throughput computation
- Architecture tolerant of memory latency
- Best for data-parallel tasks

CPU

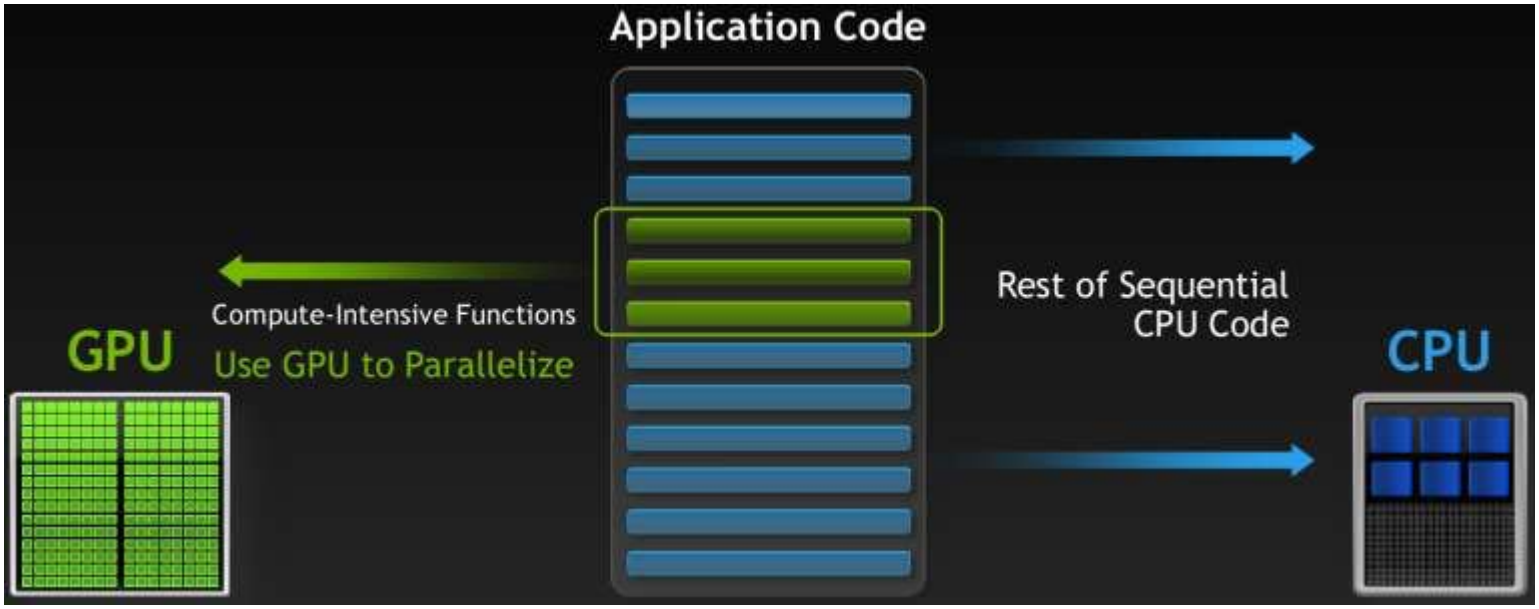


GPU



GPGPU Programming Model

- **serial parts** of a program, or those with low level of parallelism, keep running **on the CPU** (host)
- computational-intensive **data-parallel** regions are executed **on the GPU** (device)
- required data is moved on GPU memory and back to HOST memory

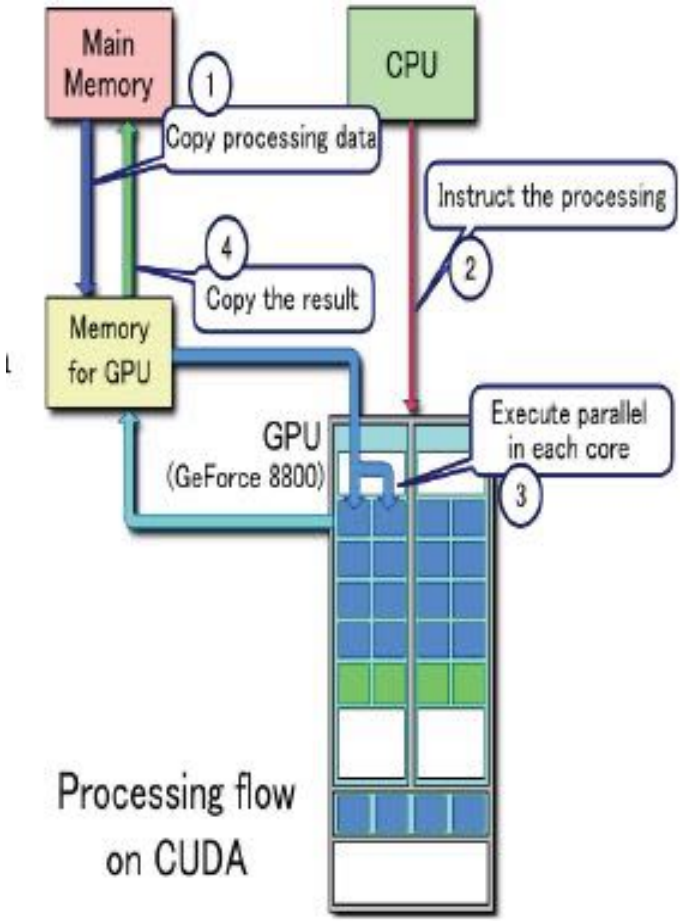


There cannot be a GPU without a CPU

GPUs are designed as numeric computing engines, therefore they will not perform well on other tasks.

Applications should use both CPUs and GPUs, where the latter is exploited as a coprocessor in order to speed up numerically intensive sections of the code by a massive fine grained parallelism.

CUDA programming model introduced by NVIDIA in 2007, is designed to support joint CPU/GPU execution of an application.



DIY GPU Workstation

Do It Yourself

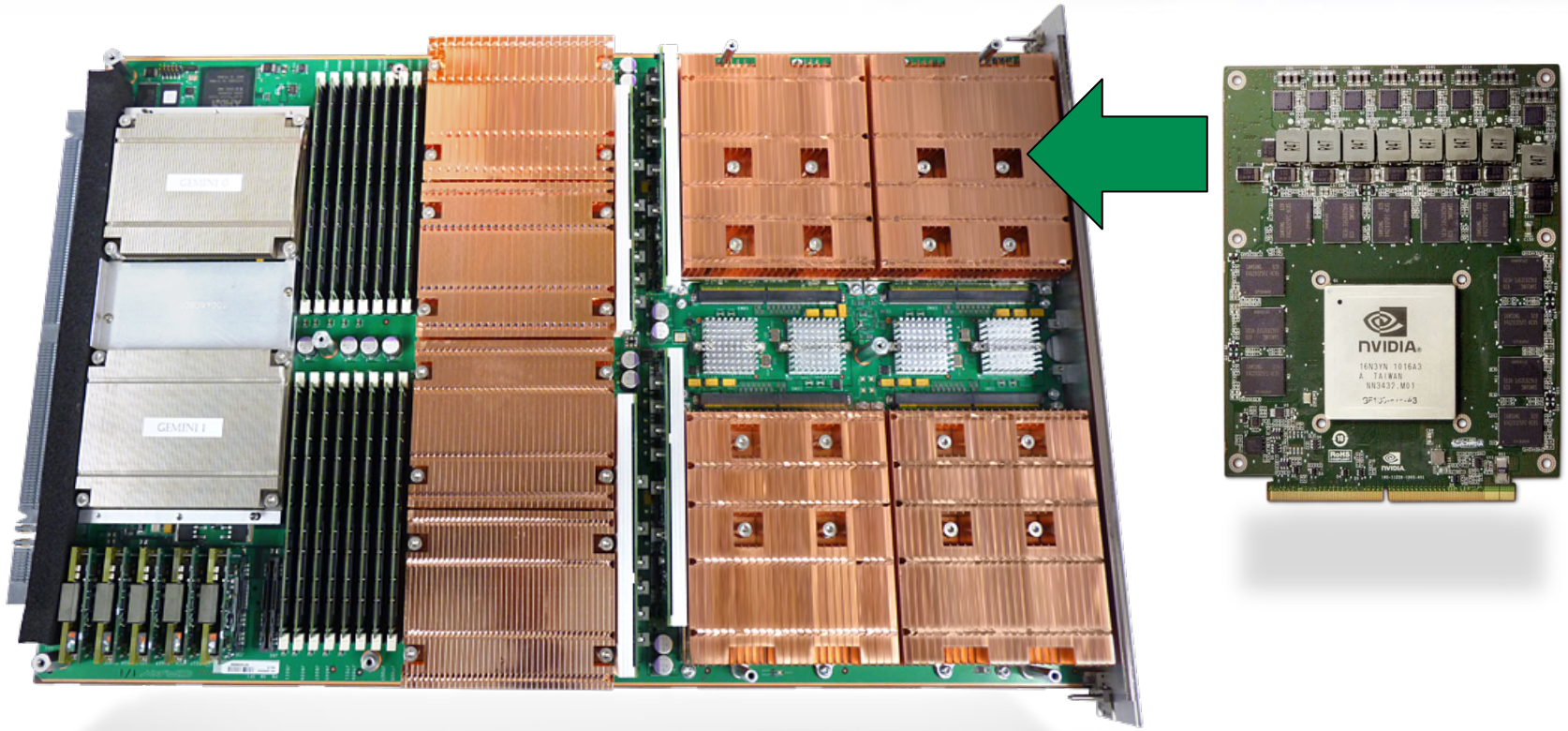
- Just need to slot GPU card into PCI-e
- Need to make sure there is enough space and power in workstation

GPU Servers

- Several vendors offer GPU Servers
- Example Configuration:
  - 4 GPUs plus 2 (multi-core) CPUs

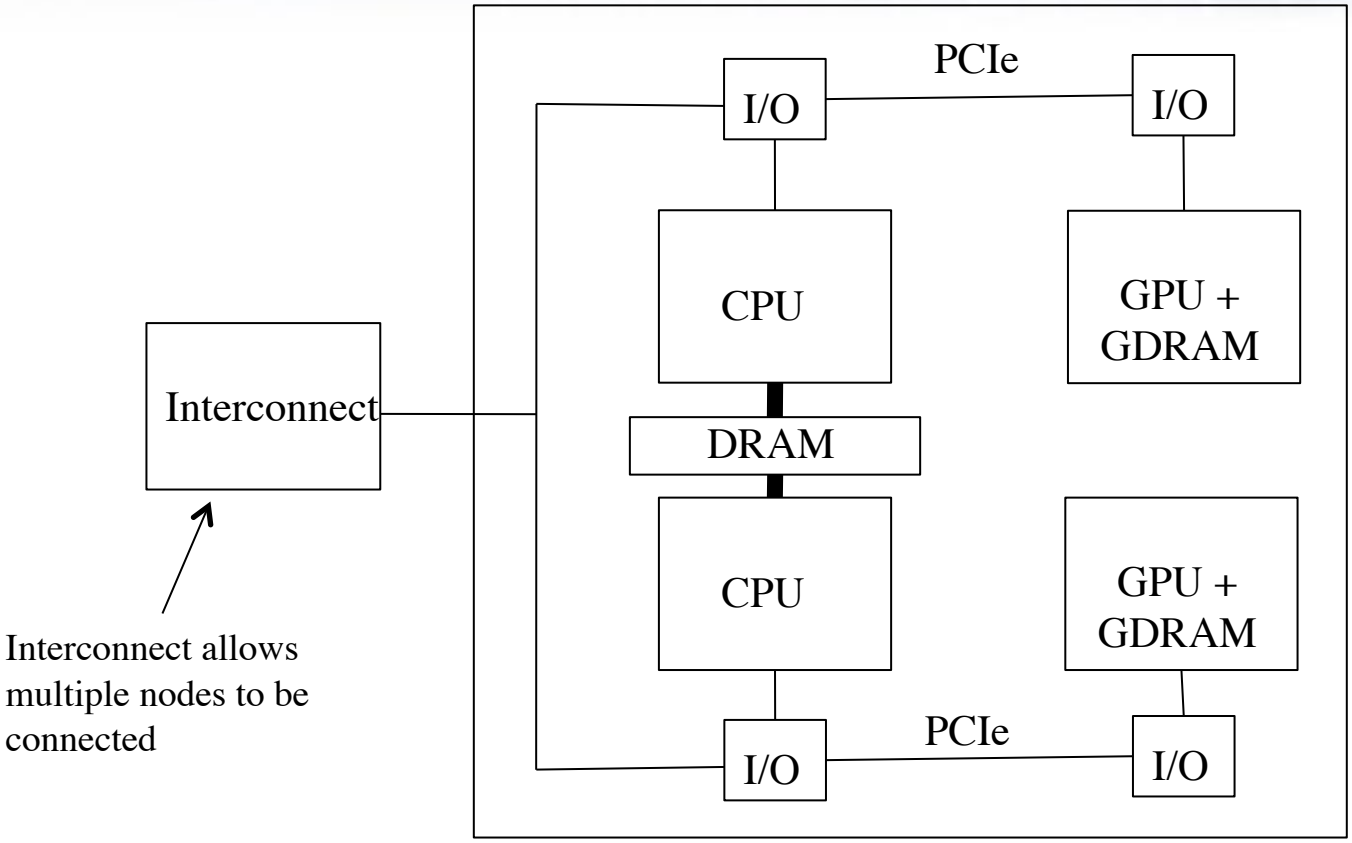
- Multiple servers can be connected via interconnect

Cray XK6 Compute Blade



- Compute Blade: 4 Compute Nodes
  - 4 CPUs (middle) + 4 GPUs (right)
  - + 2 interconnect chips (left) (2 compute nodes share a single interconnect chip)

Scaling to larger systems



- Can have multiple CPUs and GPUs within each "workstation" or "shared memory node"
  - E.g. 2 CPUs +2 GPUs (above)
  - CPUs share memory, but GPUs do not

TOP 500
The List.

November 2021

A total of 151 systems on the list are using accelerator/co-processor technology, up from 147 six months ago. 84 of these use NVIDIA Volta chips, 43 use NVIDIA Ampere, and 8 systems with NVIDIA Pascal.

		Count	System Share (%)	Rmax (TFlops)	Rpeak (TFlops)	Cores
1	NVIDIA Tesla V100	68	13.6	226,796	443,631	4,688,680
2	NVIDIA A100	18	3.6	237,246	344,051	2,260,896
3	NVIDIA Tesla V100 SXM2	11	2.2	90,370	180,163	2,031,440
4	NVIDIA A100 80GB	9	1.8	121,225	160,208	1,026,944
5	NVIDIA A100 40GB	8	1.6	52,766	84,264	555,588
6	NVIDIA A100 SXM4 40 GB	8	1.6	115,838	160,747	1,229,272
7	NVIDIA Tesla P100	7	1.4	46,445	68,784	944,960
8	NVIDIA Volta GV100	4	0.8	269,439	362,565	4,408,096
9	NVIDIA Tesla K40	3	0.6	8,824	14,612	201,328
10	Matrix-2000	1	0.2	61,445	100,679	4,981,760

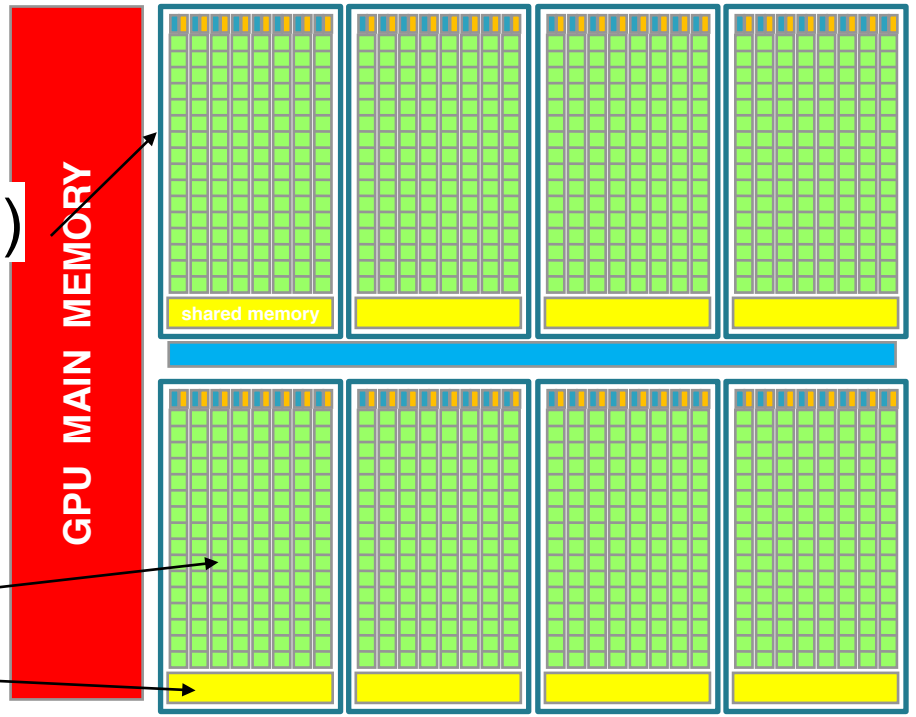
...

GPU Architecture Scheme

A tipical GPU architecture consists of

- Main Global Memory
  - medium size (8-16 GB)
  - very hgh bandwidth (250-800 GB/s)
- Streaming Multiprocessors (SM)
  - grouping independent cores and control units
- each SM unit has

  Streaming Processors, i.e.
  - many ALU cores (> 100 cores)
  - lots of registers (32K-64K)
  - instruction scheduler dispatchers
  - a shared memory with very fast access to data

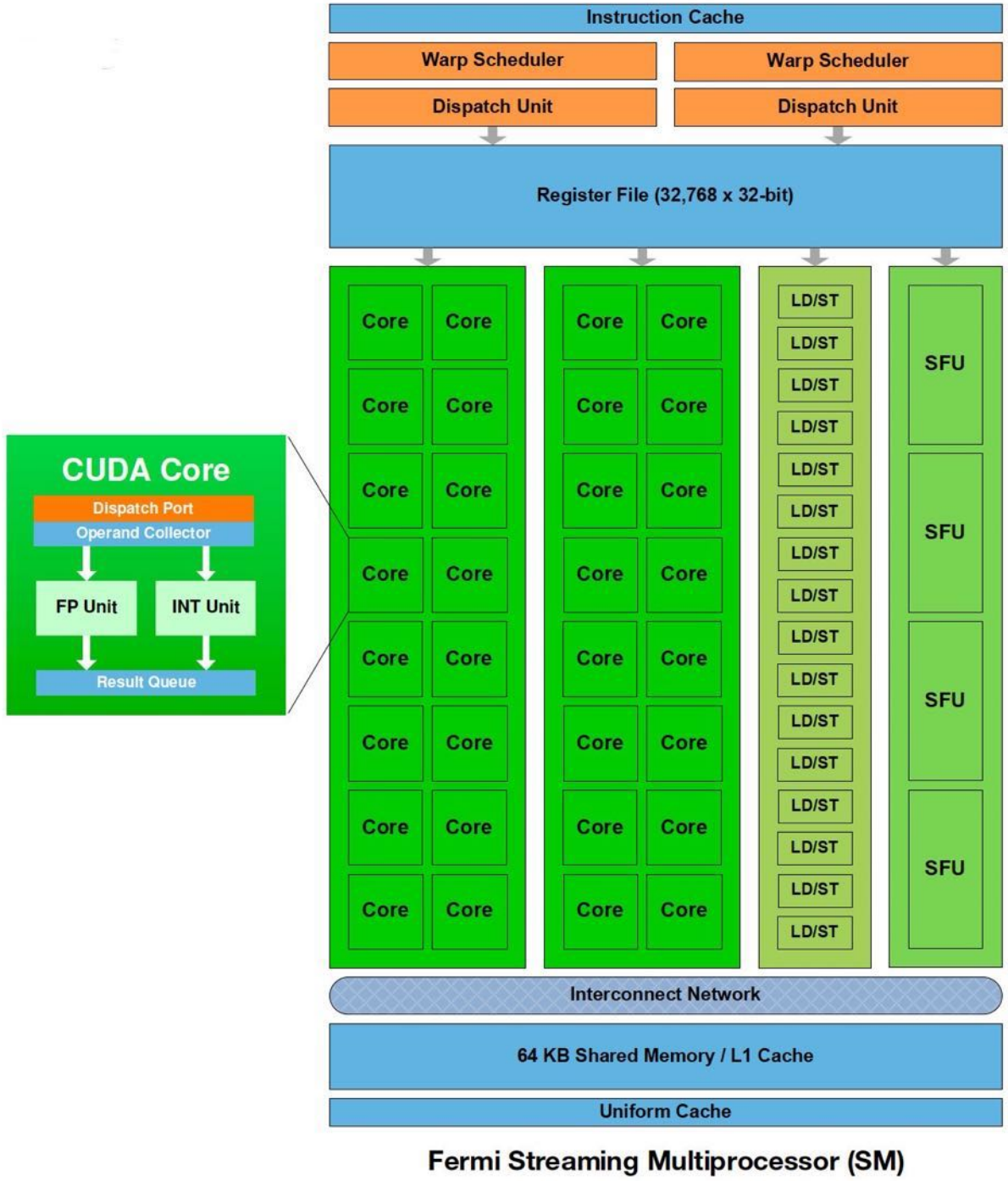


GPU Functional Unit Types

- FP32: performs 32-bit floating point add, multiply, multiply/add, and similar instructions.
- INT32: performs 32-bit add, multiply, multiply-add, and maybe some logical operations.
- FP64: pxecutes 64-bit FP operations
- Special Functional Unit (SFU): performs reciprocal (1x) and transcendental instructions such as sine, cosine, and reciprocal square root.
- Load/Store (LS): performs loads and stores from shared, constant, local, and global memory address spaces.
- Tensor Core: specialized units to compute A*B+C matrix product.

Nvidia SM

- Less scheduling units than cores
- Threads are scheduled in groups of 32, called a *warp*
- Threads within a warp always execute the same instruction in lock-step (on different data elements)
- Configurable L1 Cache/ Shared Memory



NVIDIA Volta V100 Architecture (2017)

- A full GV100 GPU unit contains 6 Compute Graphic Clusters (CGC) with 14 SM each, total 84 SMs
- 5376 FP32 cores
- 5376 INT32 cores
- 6MB L2 cache
- High Bandwidth Memory
  - 16 GB HBM2 SDRAM
  - 732 GB/s bandwidth
- NVLink tecnology
  - 300GB/s bandwidth to host data transfers
  - 12X respect PCIe Gen3 16x

https://developer.nvidia.com/blog/inside-volta

Peak Performance:
15,7 FP32 TFlops
Max Power Consumption: 300W

Streaming Multiprocessor of nVIDIA Volta (2017)

- SM composed of 4 independent blocks
- each block sports:
  - 1 warps x 2 dispatchers
  - 16FP32 + 16INT32 ALU units
  - separate FP32 and INT32 cores, allowing simultaneous execution of FP32 and INT32 operations at full throughput
  - 8FP64 ALU units
  - 2 Tensor Core units (HW matmul)
  - 8 Load/Store units
  - 4 SFU units
  - 32768 32bits registers
- each block accesses:
  - 128KB for L1/shared memory
  - 4 texture units

NVIDIA Ampere A100 Architecture (2020)

- A full GA100 GPU unit contains 8 Compute Graphic Clusters (CGC) with 16 SM each, total 128 SMs
- 8192 FP32 cores
- 8192 INT32 cores
- **40MB L2** cache
- High Bandwidth Memory
  - **40GB HBM2**
  - 732 GB/s bandwidth
- NVLink tecnology
  - 600GB/s bandwidth to host data transfers
  - 24X respect PCIe Gen3 16x

developer.nvidia.com/blog/nvidia-ampere-architecture-in-depth

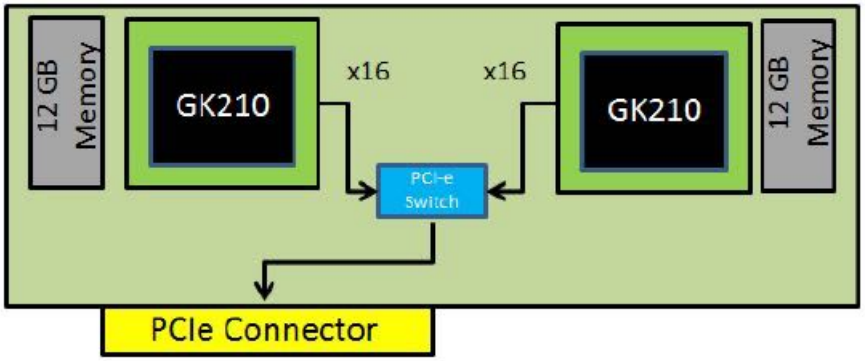
Peak Performance:
19,5 FP32 TFlops
Max Power Consumption: 400W

Streaming Multiprocessor of nVIDIA Ampere (2020)

- SM composed of 4 independent blocks
- each block sports:
  - 1 warps x 2 dispatchers
  - 16FP32 + 16INT32 ALU units
  - separate FP32 and INT32 cores, allowing simultaneous execution of FP32 and INT32 operations at full throughput
  - 8FP64 ALU units
  - 2 Tensor Core units (HW matmul)
  - 8 Load/Store units
  - 4 SFU units
  - 32768 32bits registers
- each block accesses:
  - 192KB for L1/shared memory
  - 4 texture units

GPU nVIDIA K80 (2013)

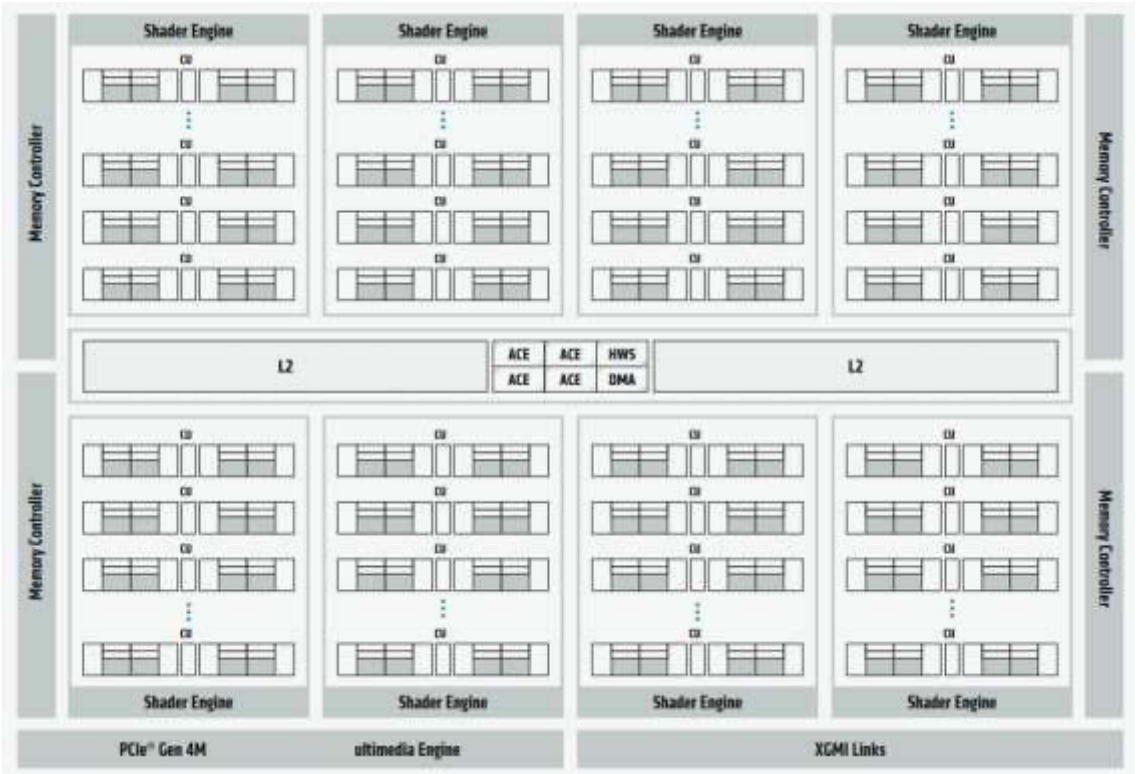
- Two GPUs (K40) per device
  - 12GB RAM per GPU
  - 480 GB/s memory bandwidth
  - 15 SM per GPU
  - 192 CUDA cores/SM
    - total of 2880 cuda cores
  - 500-800 MHz clock
  - 250W



AMD Radeon MI100 Architecture (2020)

- A full MI100 GPU unit contains a total of 120 Compute Unit (like SMs)
- 7680 FP32 cores
- 8MB L2 cache
- High Bandwidth Memory
  - **32GB HBM2**
  - 1200 GB/s bandwidth
- **AMD Infinity Fabric**
  500GB/s bandwidth to host data transfers
  - 24X respect PCIe Gen3 16x

www.amd.com MI100 microarchitecture

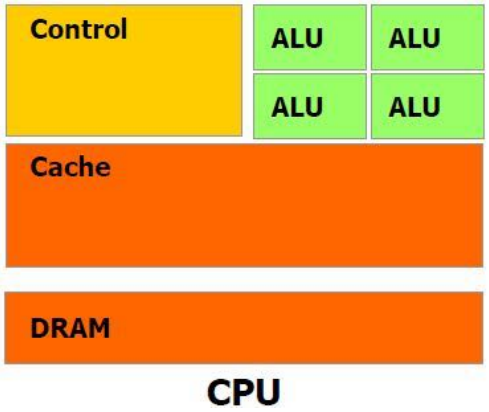


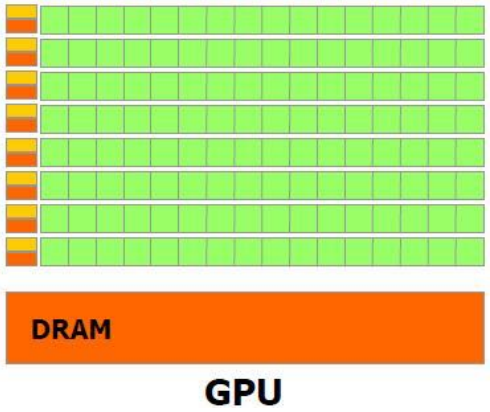
Peak Performance:
23.0 FP32 TFlops
Max Power Consumption: 400W

- To utilise a GPU, programs must
  - Contain parts targeted at host CPU (most lines of source code)
  - Contain parts targeted at GPU (key computational kernels)
  - Manage data transfers between distinct CPU and GPU memory spaces
  - Traditional language (e.g C/Fortran) does not provide these facilities
- To run on multiple GPUs in parallel
  - Normally use one host CPU core (thread) per GPU
  - Program manages communication between host CPUs in the same fashion as traditional parallel programs
    - e.g. MPI and/or OpenMP (latter shared memory node only)

Different worlds: host and device

	Host	Device
Threading resources	2 threads per core (SMT), 24/32 threads per node. The thread is the atomic execution unit.	e.g.: 1536 (thd x sm) * 14 (sm) = 21504. The Warp (32 thd) is the atomic execution unit.
Threads	«Heavy» entities, context switches and resources management.	Extremely lightweight, managed grouped into warps, fast context switch, no resources management (statically allocated once).
Memory	e.g.: 48 GB / 32 thd = 1.5 GB/thd, 300 cycles lat., 6.4 GB/s band (DDR3), 3 caching levels with lots of speculation logic.	e.g.: 6 GB / 21504 thd = 0.3 MB/thd, 600 cycles lat*, 144 GB/s band (GDDR5)*, fake caches. * coalesced





How do I program GPUs?

The situation is changing rapidly but possibilities include:

Declarative languages

- OpenMP
  - v 4.0+ allows offloading of tasks onto GPUs
- OpenAcc
  - High-level model, particularly suited for devices such as GPUs.

Languages

- CUDA
  - Extension to C developed by NVIDIA. With PGI compilers, FORTRAN extension also possible.
- OpenCL
  - General framework for writing programs across heterogenous devices. Often used for non-NVIDIA GPUs and FPGAs.

GPU Programming Languages

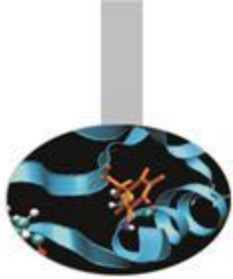
- **CUDA** (Compute Unified Device Architecture)
  - a set of extensions to higher level programming language to use GPU as a coprocessor for heavy parallel task
  - a developer toolkit to compile, debug, profile programs and run them easily in a heterogeneous systems
- **OpenCL** (Open Computing Language):
  - a standard open-source programming model developed by major brands of hardware manufacters (Apple, Intel, AMD/ATI, nVIDIA).
    - like CUDA, provides extentions to C/C++ and a developer toolkit
    - extensions for specific hardware (GPUs, FPGAs, MICs, etc)
    - it's very low level (verbose) programming

There are many other approaches and solutions such as SYCL (Khronos), HIP (AMD), OneAPI (Intel), DirectCompute (Microsoft), ... but current market is basically dominated by CUDA and some OpenCL

GPU Programming Languages

Numerical analytics	MATLAB Mathematica, LabVIEW
Fortran	CUDA Fortran
C	CUDA C
C++	CUDA C++
Python	PyCUDA, Copperhead, Numba
F#	Alea.cuBase

CUDA programming model



Compute **U**nified **D**evice **A**rchitecture:

- extends ANSI C language with minimal extensions
- provides application programming interface (API) to manage host and device components

CUDA program:

- Serial sections of the code are performed by CPU (host)
- The parallel ones (that exhibit rich amount of *data parallelism*) are performed by GPU (device) in the SIMD mode as CUDA kernels.
- host and device have separate memory spaces: programmers need to transfer data between CPU and GPU in a manner similar to “one-sided” message passing.

CUDA: Compute Unified Device Architecture

CUDA is a general purpose parallel computing platform and programming model that easy GPU programming, which provides:

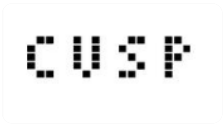
- a hierarchical multi-threaded programming paradigm that matches GPU hardware structure
- an extensions to higher level programming languages for C/C++ and Fortran to express thread parallelism within a familiar programming environment
- a new architecture instruction set called PTX (Parallel Thread eXecution) to match GPU tipical hardware
- a complete mature SDK: compiler (nvcc), debugger (cuda-gdb), profiler (nvvp), IDE (insight/eclipse/VS plugins)
- a set of GPU accelerated libraries for common scientific algorithms and requirements ...

CUDA GPU ready scientific libraries

- dense/sparse linear algebra single/multi-GPU: cuBLAS, nvBLAS, cuSparse
- dense/sparse direct solver and factorizations: cuSOLVER
- Fast Fourier Transform (and related): cuFFT
- Random number generator: cuRAND
- Common primitives for digital signal processing and imaging elaboration: NPP (nVIDIA Performance Primitives)
- Deep Learning libraries
- ... and many, many more

GPU Accelerated Libraries

Linear Algebra
FFT, BLAS,
SPARSE, Matrix



Numerical & Math
RAND, Statistics

Data Struct. & AI
Sort, Scan, Zero Sum

Visual Processing
Image & Video

NVIDIA
Video
Encode

CUDA - C


Applications
Libraries
Compiler Directives
Programming Languages
Easy to use
Most Performance
Easy to use
Portable code
Most Performance
Most Flexibility

GPGPU Programming Model

- A function which runs on a GPU is called "**kernel**"
  - when a kernel is launched on a GPU thousands of threads will execute its code
  - programmer chooses the number of threads to run
  - **each thread acts on a different data element independently**
  - the GPU parallelism is very close to the SPMD paradigm

```
void vecAddCPU (int N, const float *A,
                const float *B, float *C)
{
   for ( int i = 0; i < N; i++ )
      c[i] = a[i] + b[i];
}
...
// call vecAddCPU on N elements
vecAddCPU ( N, a, b, c );
```

```
void vecAddGPU (int N, const float *A,
                const float *B, float *C)
{
   int i = blockIdx.x*blockDim.x + threadIdx.x;
    if ( i < N) c[i] = a[i] + b[i];
}
...
// call vecAddGPU on N elements
vecAddGPU<<<1, N>>>( N, a, b, c );
```

Asynchronous execution

By default, GPU operations are **asynchronous**.

- When you call a function that uses the GPU, the operations are enqueued to the particular device, but not necessarily executed until later.
- This allows us to execute more computations in parallel, including operations on CPU or other GPUs.

Instead they are **synchronous** if

- The environment variable CUDA_LAUNCH_BLOCKING equals to 1.
- using a profiler(nvprof), without enabling concurrent kernel profiling
- memcpy that involve host memory which is not page-locked.

A first program

```
#include <stdio.h>

void CPUFunction() {
  printf("Hello world from the CPU.\n");
}

__global__ void GPUFunction() {
  printf("Hello world from the the GPU.\n");
}

int main() {
  CPUFunction();
  GPUFunction<<<1, 1>>>();
  cudaDeviceSynchronize();
}
```

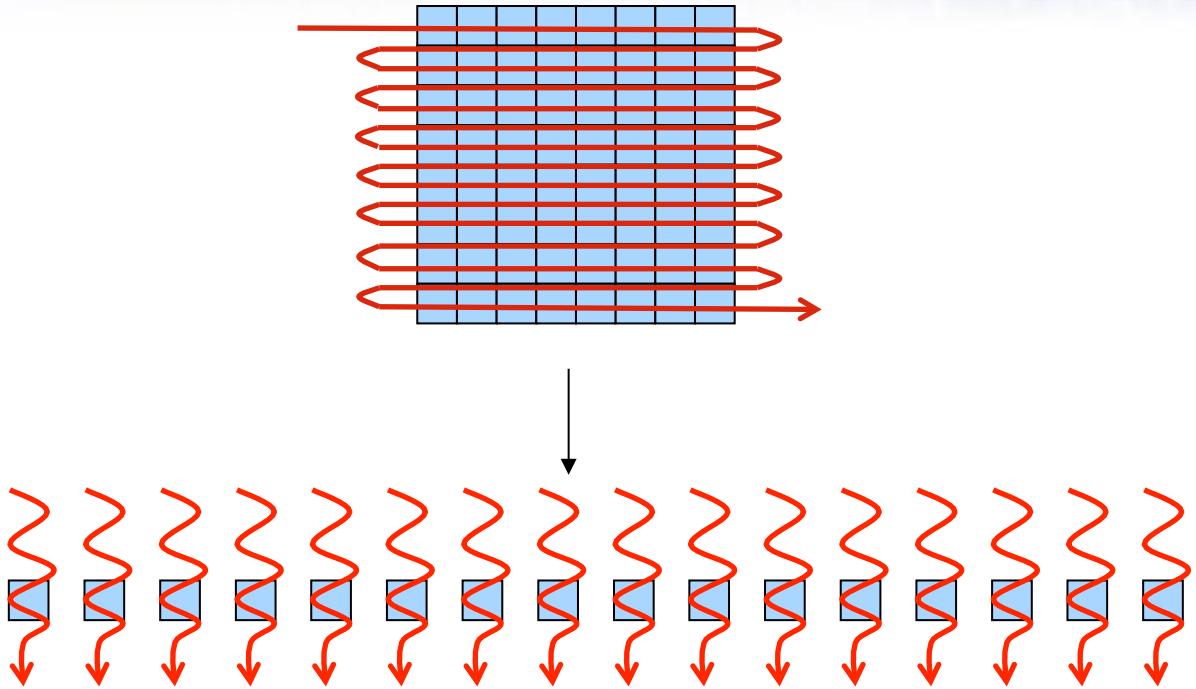
Try to remove one of the following at a time and see what happens

- __global__
- <<<1,1>>>
- cudaDeviceSynchronize();

COMPILE with

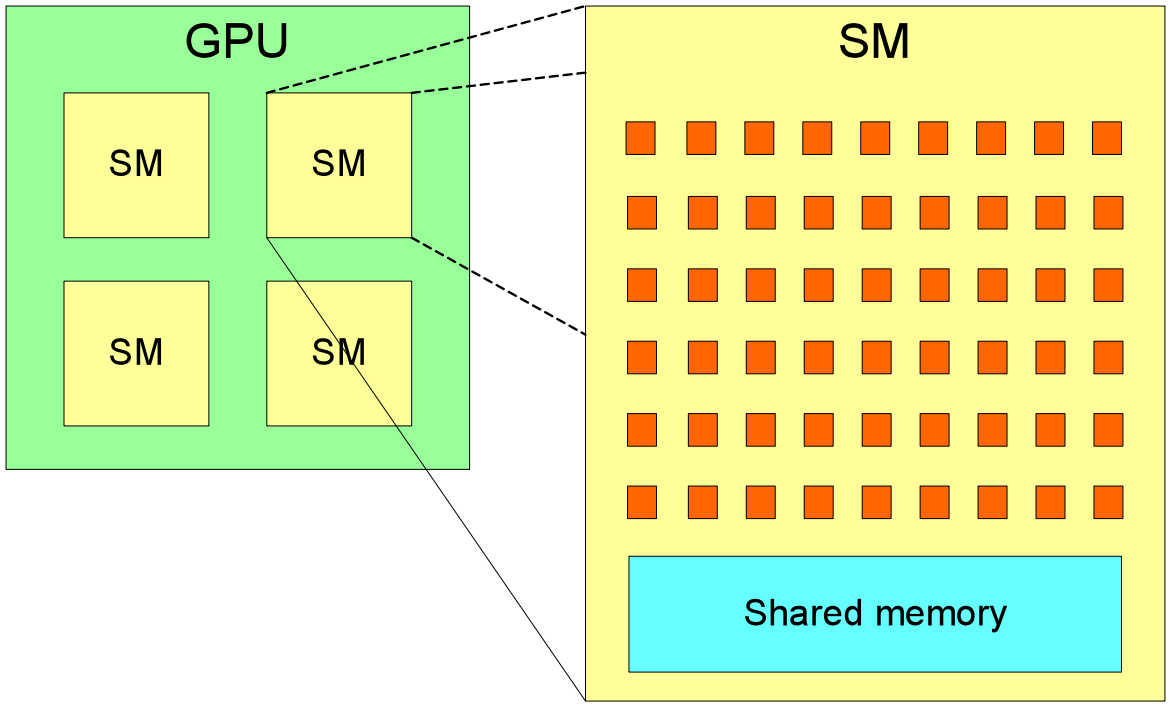
nvcc -o first first.cu -run

GPGPU: Stream Computing



- Data set decomposed into a *stream* of elements
- A single computational function (*kernel*) operates on each element
  - "thread" defined as execution of kernel on one data element
- Multiple cores can process multiple elements in parallel
  - i.e. many threads running in parallel
- Suitable for data-parallel problems

- NVIDIA GPUs have a 2-level hierarchy:
  - Multiple Stream Multiprocessors SMs
    - each with multiple cores



- In CUDA, this is abstracted as *Grid* of *Thread Blocks*
  - The multiple **blocks** in a grid map onto the multiple **SMs**
    - Each block in a grid contains multiple **threads**, mapping onto the **cores** in an SM
- We don't need to know the exact details of the hardware (number of SMs, cores per SM).
  - Instead, *oversubscribe*, and system will perform scheduling automatically
    - Use more blocks than SMs, and more threads than cores
  - Same code will be portable and efficient across different GPU versions.

CUDA Kernel Launch Parameters Syntax

Triple chevron launch syntax <<< >>> contains "**kernel launch parameters**"

vecAddGPU<<< **1, 1024** >>>(N, a, b, c)

1° parameter defines the number of **blocks** to use

2° parameter defines the number of **threads per block**

```
void vecAddGPU (int N, const float *A,
                const float *B, float *C)
{
   int i = blockIdx.x*blockDim.x + threadIdx.x;
    if ( i < N) c[i] = a[i] + b[i];
}
...
// call vecAddGPU on N elements
vecAddGPU<<<1, N>>>( N, a, b, c );
```

A second program

```
#include <stdio.h>

__global__ void GPUfunction()
{
  printf("This is running in parallel.\n");
}

int main()
{
 GPUfunction <<<5, 5>>>();
  cudaDeviceSynchronize();
}
```

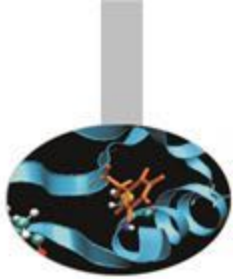
Try

- <<<1, 1>>
- <<<1,10>>>
- <<<10, 1>>
- <<<10, 10>>
- and, again, remove cudaDeviceSynchronize();

COMPILE with

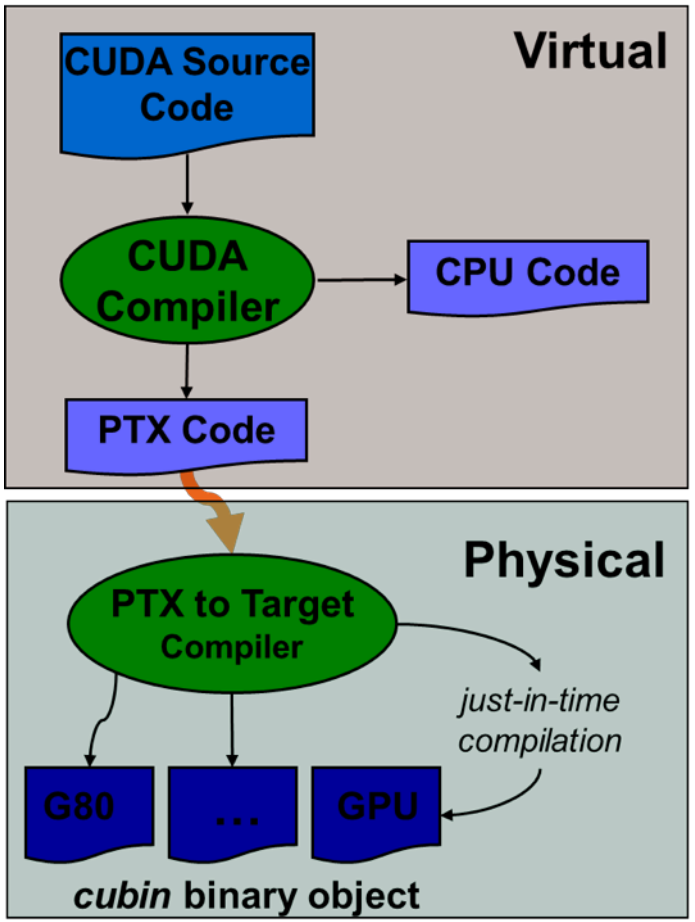
nvcc -o second second.cu -run

NVIDIA C compiler



nvcc front-end for compilation:

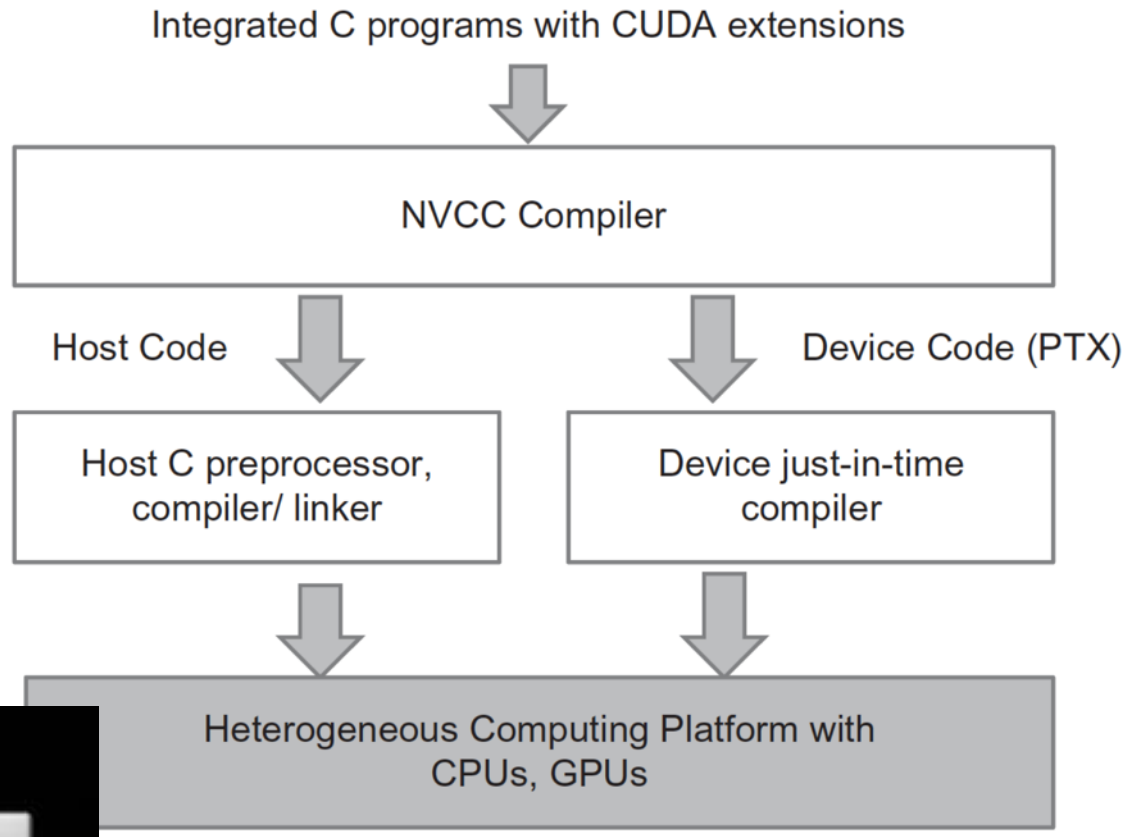
- separates GPU code from CPU code
- CPU code -> C/C++ compiler (Microsoft Visual C/C++, GCC, ecc.)
- GPU code is converted in an intermediate assembly language: PTX, then in binary form (the *cubin* object)
- link all executables

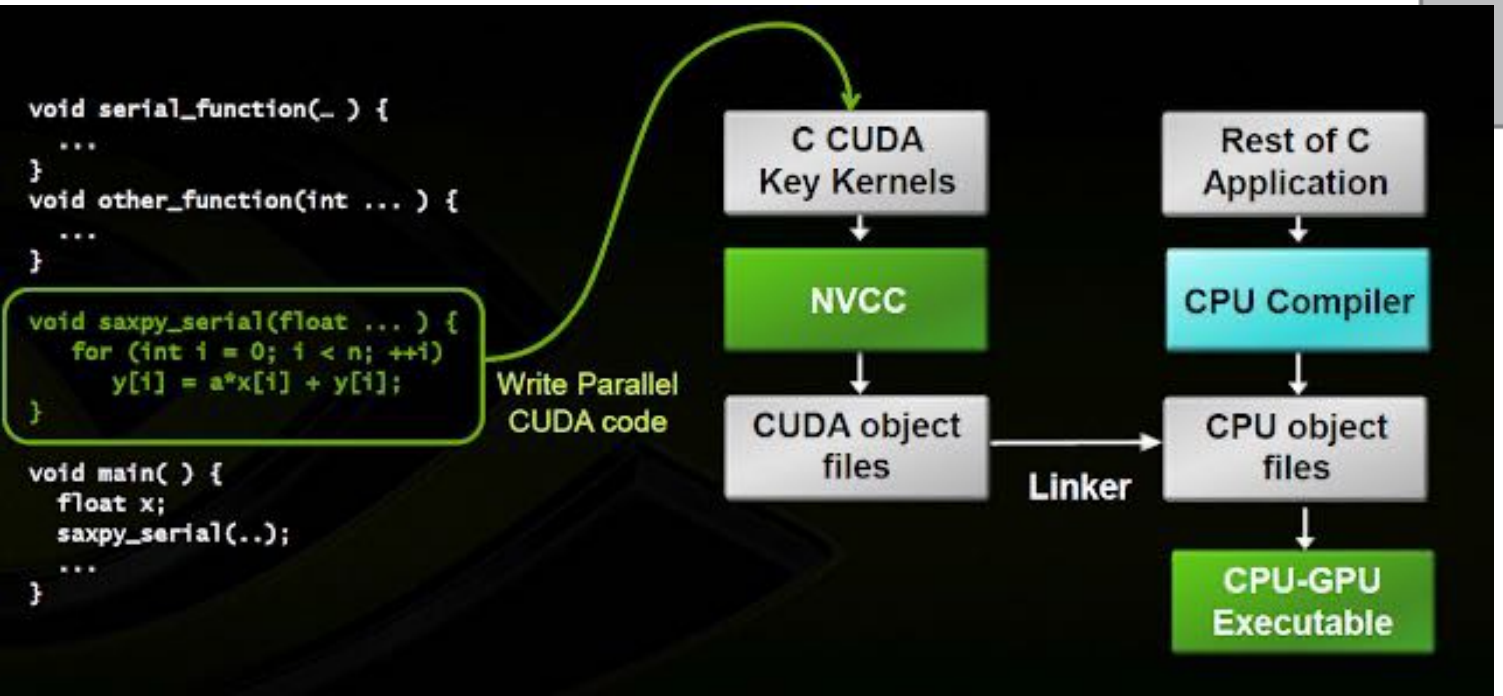


How to compile

```
nvcc myprog.cu –o myprog
```

Nvcc only parses .cu files for CUDA



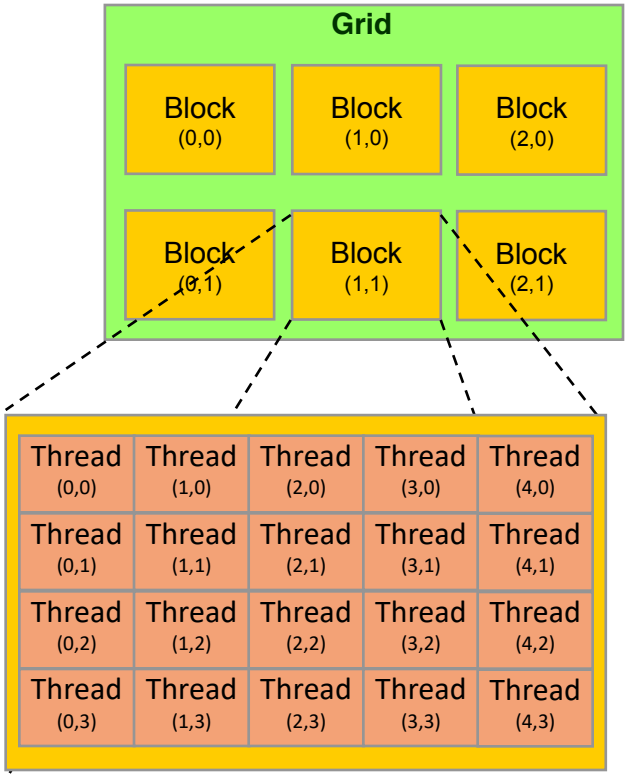


Device Code

- CUDA keyword `__global__` indicates a kernel function that:
  - Runs on the device.
  - Called from the host.
- CUDA keyword `__device__` indicates a device function that:
  - Runs on the device.
  - Called from a kernel function or another device function.
- Triple angle brackets `<<< >>>` indicate a call from host code to device code.
  - Kernel launch
- `nvcc` separates source code into two components:
  - Device functions are processed by NVIDIA compiler.
  - Host functions are processed by standard host compiler.
  - `$ nvcc hello.cu`

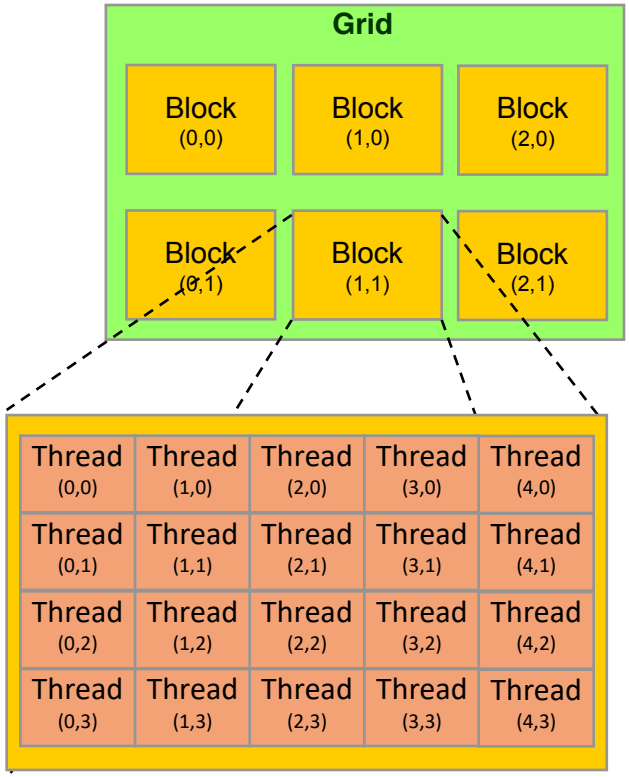
GPU Thread Hierarchy

- In order to compute N elements on the GPU in parallel, at least N concurrent threads must be created on the device
- GPU threads are grouped togheter in teams or blocks of threads
- Threads belonging to the same block or team can cooperate togheter exchanging data through a shared memory cache area
- each block of threads will be executed independently
- no assumption is made on the blocks execution order



GPU Thread Hierarchy

- threads are organized into blocks of threads
  - blocks can be 1D, 2D, 3D sized in threads
  - blocks can be organized into a 1D, 2D, 3D grid of blocks
- Blocks are organized in a grid of blocks
- each block or thread has a unique ID
  - use .x, .y, .z to access its components

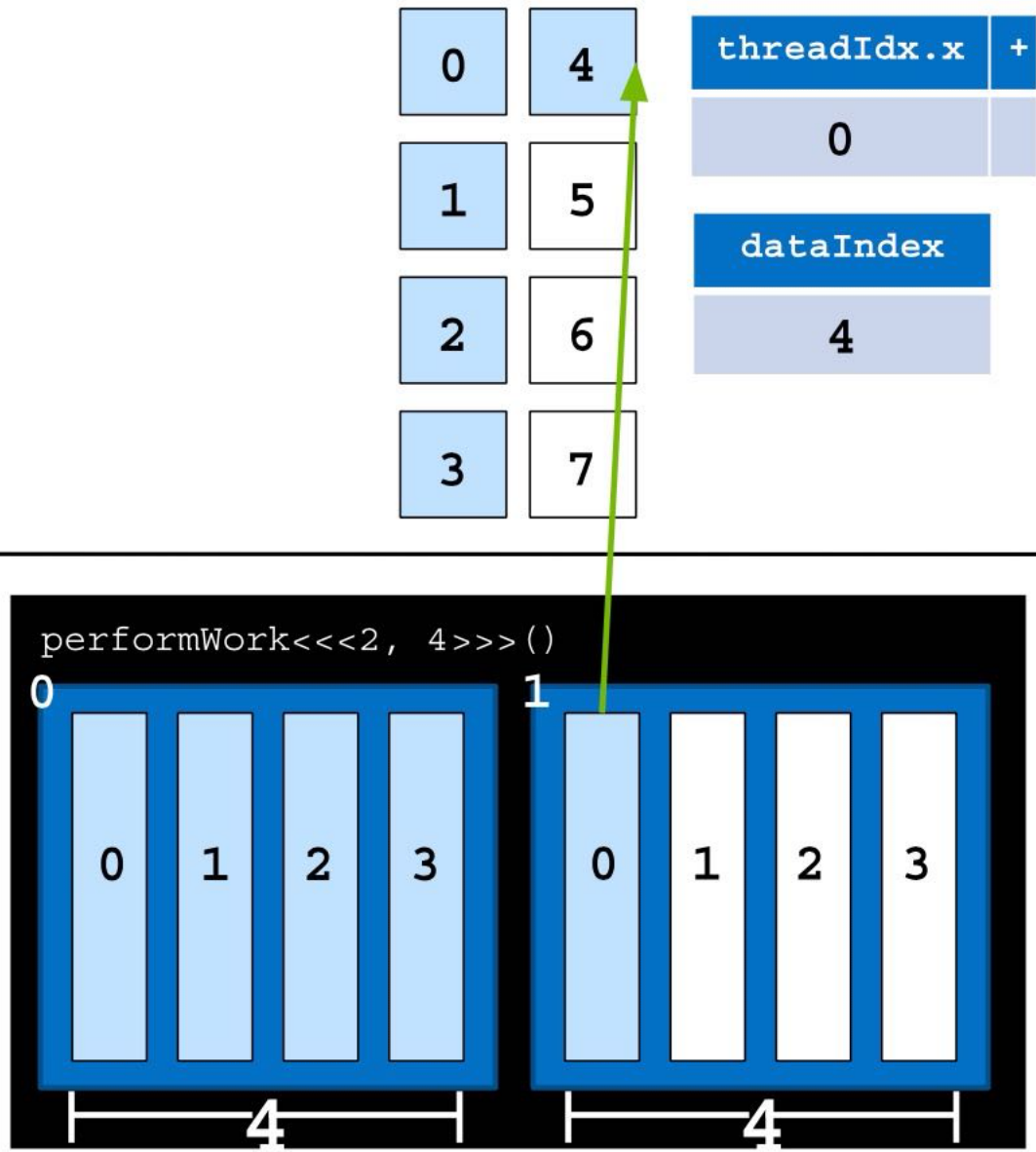


`threadIdx:`
thread coordinates inside the block

`blockIdx:`
block coordinates inside the grid

`blockDim:`
block dimensions in thread units

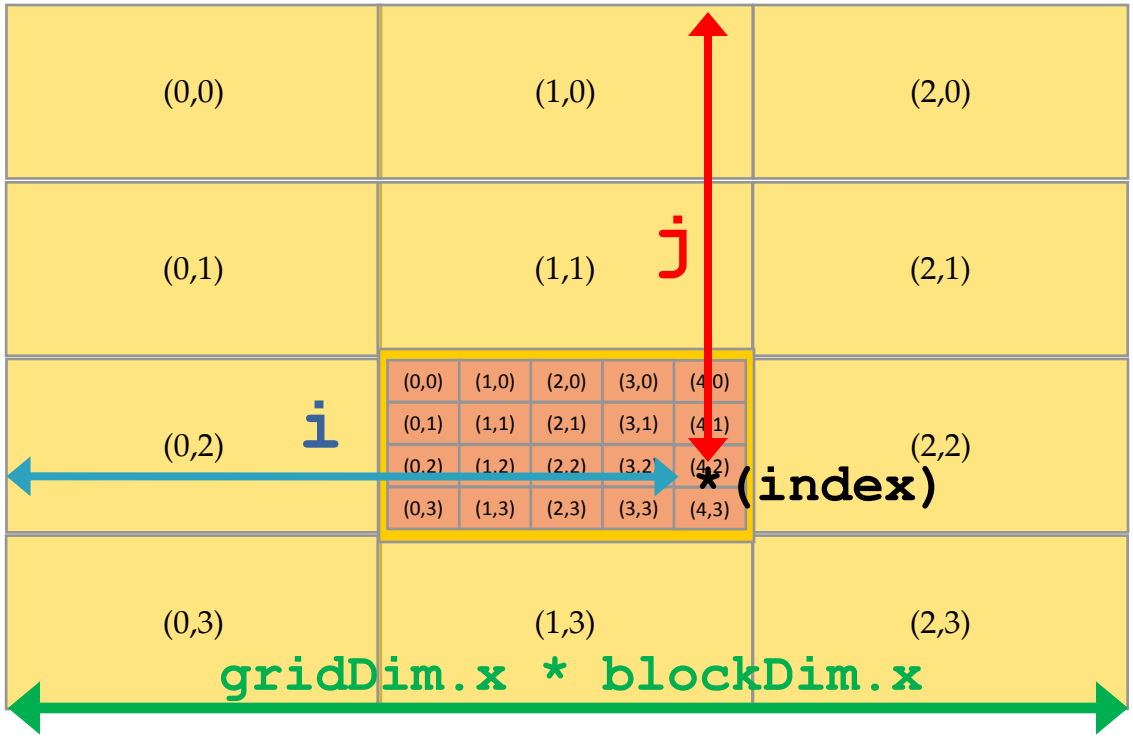
`gridDim:`
grid dimensions in block units



This idiomatic expression gives each thread a unique index within the entire grid.

```
int i = blockIdx.x * blockDim.x + threadIdx.x;
```

CUDA Thread Grid



`threadIdx:`
thread coordinates inside a block

`blockIdx:`
block coordinates inside the grid

`blockDim:`
block dimensions in thread units

`gridDim:`
grid dimensions in block units

```
i = blockIdx.x * blockDim.x + threadIdx.x;
j = blockIdx.y * blockDim.y + threadIdx.y;

index = j * gridDim.x * blockDim.x + i;
```

CUDA C Example

- You can think of this as restructuring the original loop

```
for (i=0;i<N;i++){
  c[i] = a[i] + b[i];
}
```

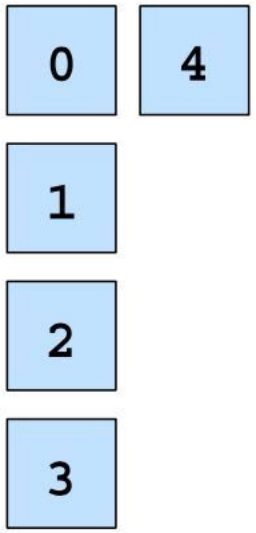
as a set of (N/32) blocks composed by 32 threads each

```
for (i0=0;i0<(N+31)/32);i0++){
  for (i1=0;i1<32 ;i1++){
    i = i0*32  + i1;
    c[i] = a[i] + b[i];
  }
}
```

If N%32 != 0 , e.g. 100%32 == 4, you need N/32 +1 blocks, with this technique you avoid to check %
- 100 / 32 == 3,125, so 4 blocks
- (100+31) / 32 == 4,09 => 4 blocks

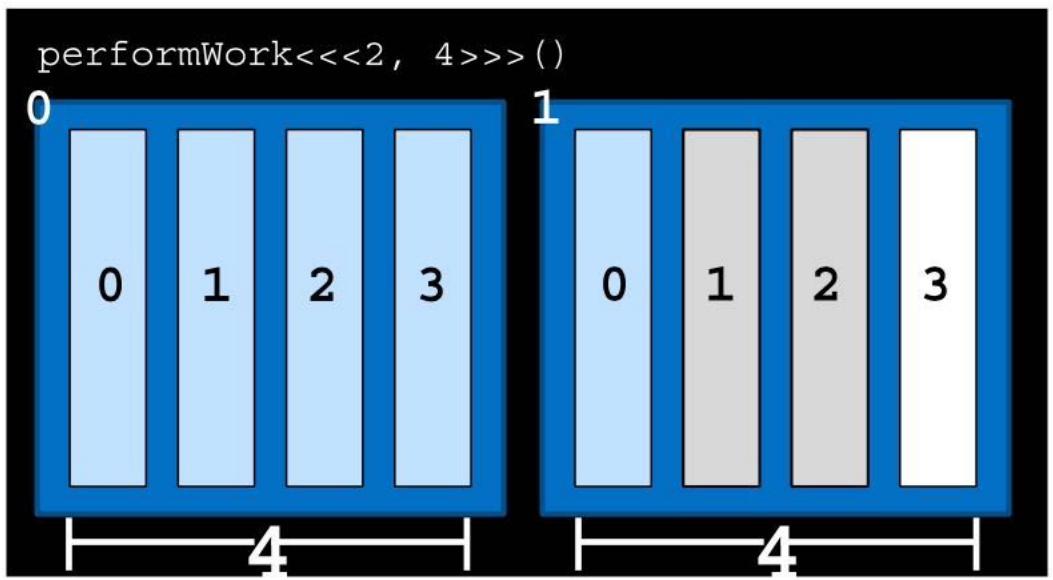
But we must check that i < N, because 4*32 == 128

and parallelising inner loop over threads in a block, outer loop over blocks.



threadIdx.x	+	blockIdx.x	*	blockDim.x
2		1		4

dataIndex	<	N	=	Can work
6		5		false



CUDA Kernel Launch Parameters Syntax

```
void vecAddGPU (int N, const float *A,
                const float *B, float *C)
{
   int i = blockIdx.x*blockDim.x + threadIdx.x;
    if ( i < N) c[i] = a[i] + b[i];
}

...
// call vecAddGPU on N elements
dim3 threads(32);
dim3 blocks ( (N+threads.x-1)/threads.x );
vecAddGPU<<< blocks, threads >>>( N, a, b, c );
```

Threads 99-127 will not compute anything as expected

Es. N = 100
100/32 = 3.125 , but 3*32 = 96 < 100
(100+31)/32 = 4.09 , and 4*32 = 128 > 100

for **blockIdx.x** = 0
    i = 0 * 32 + **threadIdx.x** = { 0, 1, 2, ... , 31 }

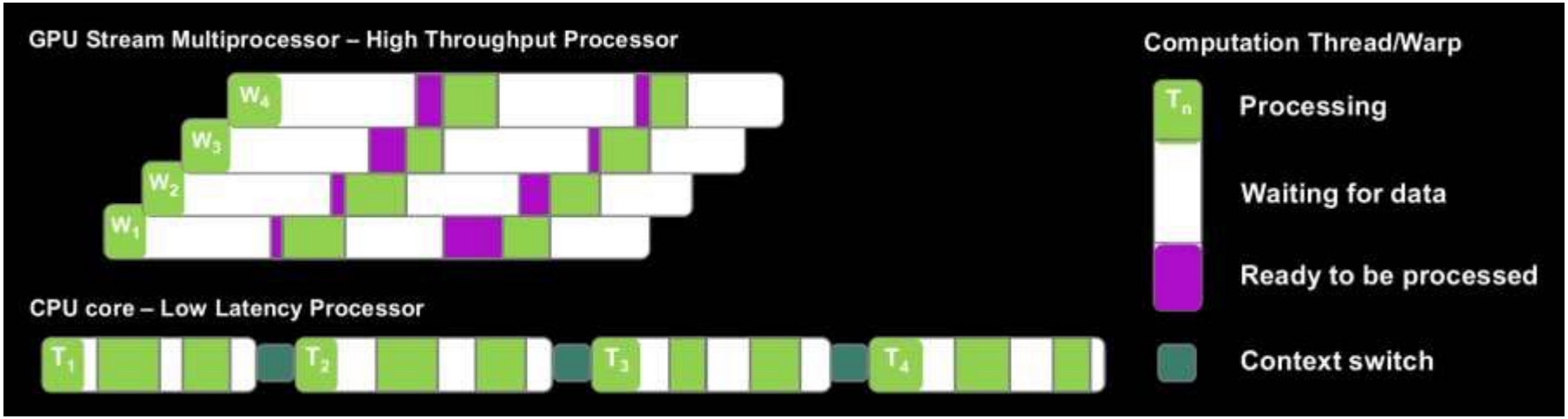
for **blockIdx.x** = 1
    i = 1 * 32 + **threadIdx.x** = { 32, 33, 34, ... , 63 }

for **blockIdx.x** = 2
    i = 2 * 32 + **threadIdx.x** = { 64, 65, 66, ... , 95 }

http://www.icl.utk.edu/~mgates3/docs/cuda.html

CUDA Programming Model

- GPU threads are extremely *light weight*
  - no penalty in case of a *context-switch*
  - each thread has its own registers
- the more are the threads *in flight*, the more the GPU hardware is able to hide memory or computational latencies



2D Example

- The previous examples were one dimensional.
- Each thread block can be 1D, 2D or 3D to best fit the algorithm, e.g. for matrix addition:

```
__global__ void matrixAdd(float a[N][N], float b[N][N], float c[N][N])
{
  int i = threadIdx.x;
  int j = threadIdx.y;

  c[i][j] = a[i][j] + b[i][j];
}

int main()
{
 dim3 blocksPerGrid(1); /* 1 block per grid (1D) */
 dim3 threadsPerBlock(N, N); /* NxN threads per block (2D) */
 matrixAdd<<<blocksPerGrid, threadsPerBlock>>>(a, b, c);
}
```

- **`dim3`** is a CUDA type, containing 3 integers (x,y and z components)

Multiple Block 2D Example

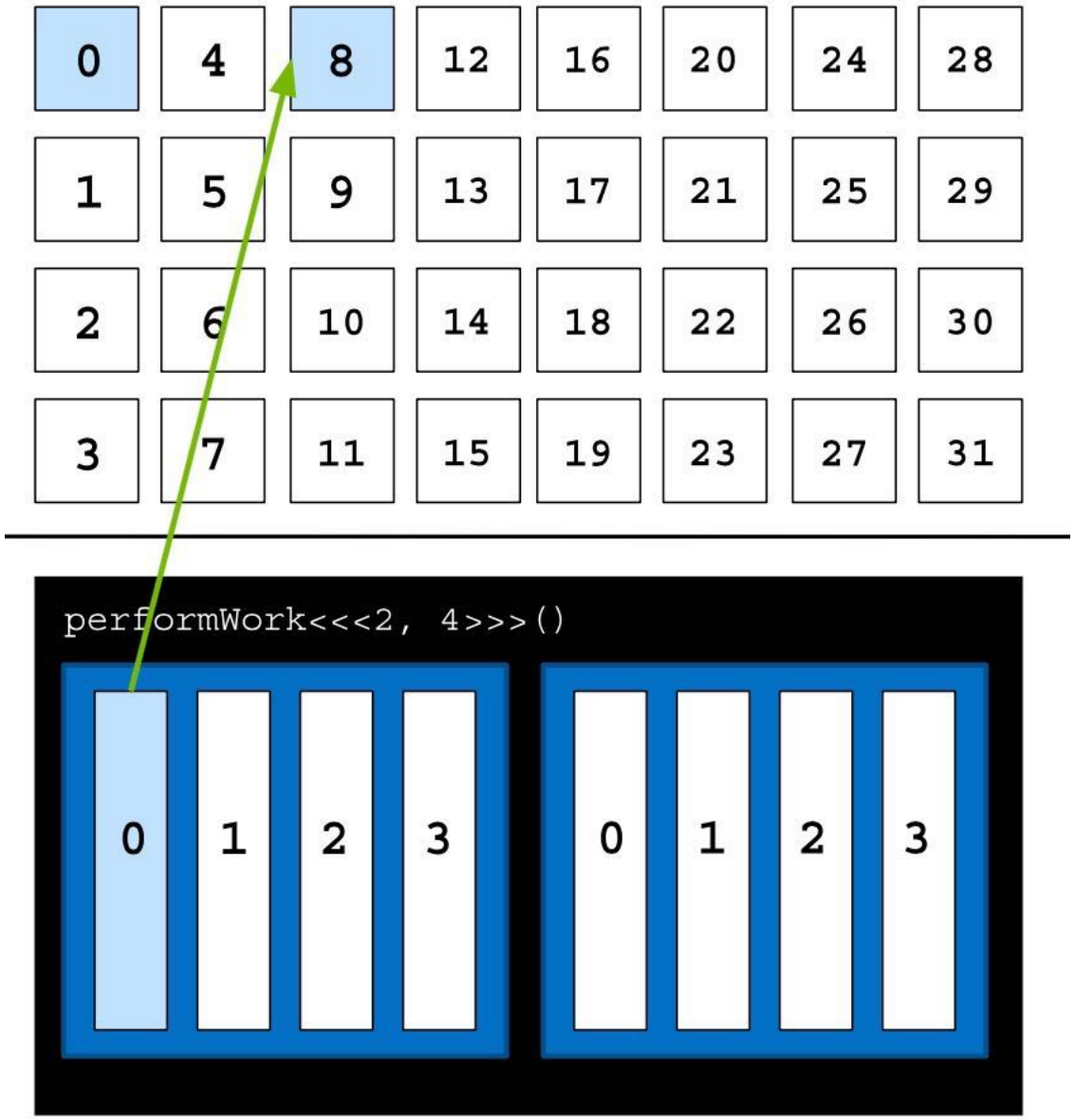
- Grid can also be be 1D, 2D or 3D

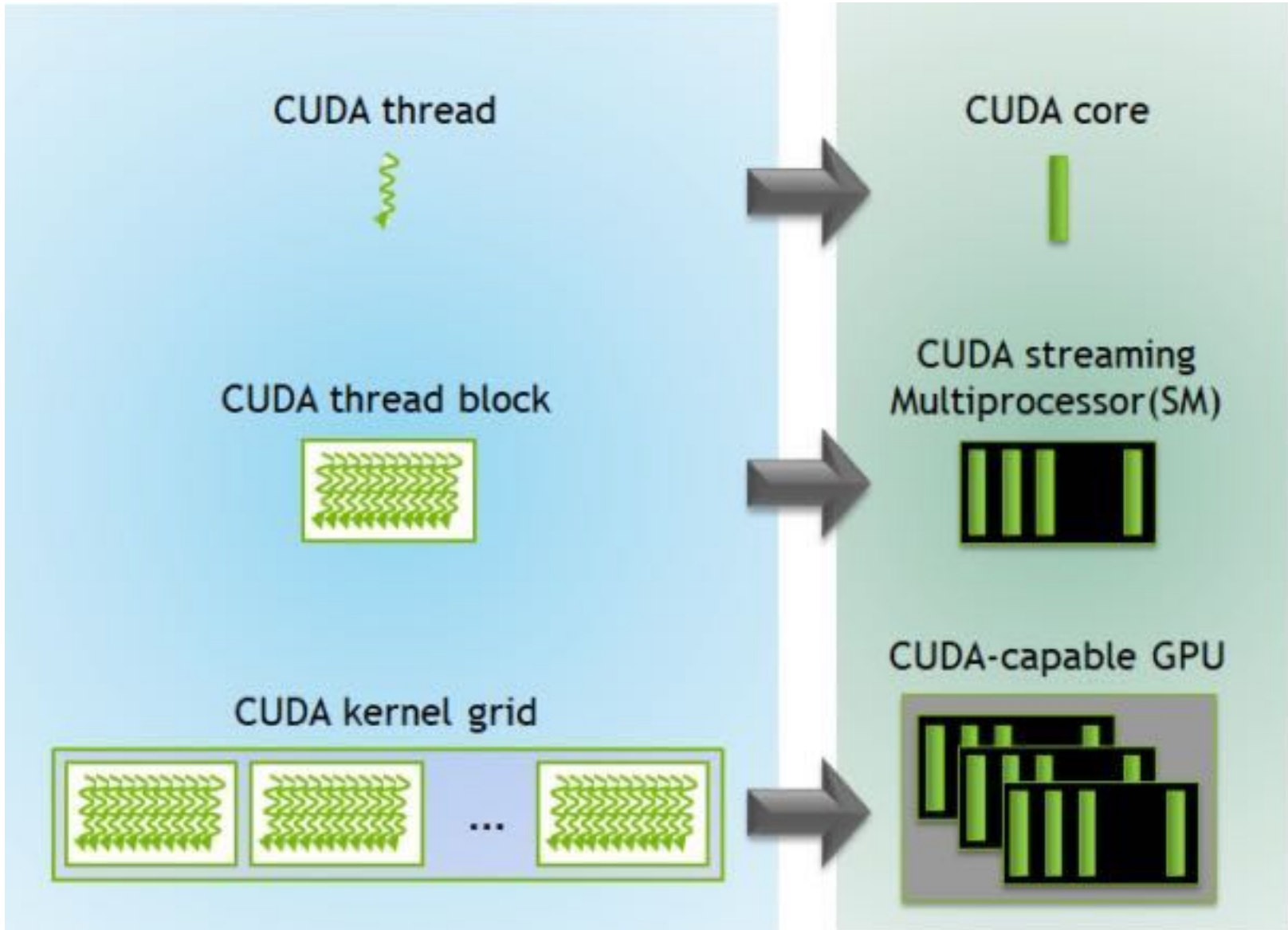
```
__global__ void matrixAdd(float a[N][N], float b[N][N], float c[N]
   [N])
{
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  int j = blockIdx.y * blockDim.y + threadIdx.y;

  c[i][j] = a[i][j] + b[i][j];
}

int main()
{
  dim3 blocksPerGrid(N/16,N/16); // (N/16)x(N/16) blocks/grid (2D)
  dim3 threadsPerBlock(16, 16); /. 16x16 threads/block (2D)
  matrixAdd<<<blocksPerGrid, threadsPerBlock>>>(a, b, c);
}
```

Grid-strided loops

- If more elements than threads

```
__global__ void kernel(int *a, int N) {

  int indexWithinTheGrid = threadIdx.x + blockIdx.x * blockDim.x;

  int gridStride = gridDim.x * blockDim.x;

  for (int i = indexWithinTheGrid; i < N; i += gridStride) {
    // do work on a[i];
  }
}
```




CUDA thread
CUDA core
CUDA thread block
CUDA streaming
Multiprocessor(SM)
CUDA kernel grid
...
CUDA-capable GPU

```
Device 0: "GeForce GTX 1650"
  CUDA Driver Version / Runtime Version          11.1 / 11.1
  CUDA Capability Major/Minor version number:    7.5
  Total amount of global memory:                 3904 MBytes (4093509632 bytes)
  (14) Multiprocessors, ( 64) CUDA Cores/MP:     896 CUDA Cores
  GPU Max Clock rate:                            1695 MHz (1.70 GHz)
  Memory Clock rate:                             4001 Mhz
  Memory Bus Width:                              128-bit
  L2 Cache Size:                                 1048576 bytes
  Maximum Texture Dimension Size (x,y,z)         1D=(131072), 2D=(131072, 65536), 3D=(16384, 16384, 16384)
  Maximum Layered 1D Texture Size, (num) layers  1D=(32768), 2048 layers
  Maximum Layered 2D Texture Size, (num) layers  2D=(32768, 32768), 2048 layers
  Total amount of constant memory:               65536 bytes
  Total amount of shared memory per block:       49152 bytes
  Total shared memory per multiprocessor:        65536 bytes
  Total number of registers available per block: 65536
  Warp size:                                     32
  Maximum number of threads per multiprocessor:  1024
  Maximum number of threads per block:           1024
  Max dimension size of a thread block (x,y,z): (1024, 1024, 64)
  Max dimension size of a grid size    (x,y,z): (2147483647, 65535, 65535)
  Maximum memory pitch:                          2147483647 bytes
  Texture alignment:                             512 bytes
  Concurrent copy and kernel execution:          Yes with 3 copy engine(s)
  Run time limit on kernels:                     No
  Integrated GPU sharing Host Memory:            No
  Support host page-locked memory mapping:       Yes
  Alignment requirement for Surfaces:            Yes
  Device has ECC support:                        Disabled
  Device supports Unified Addressing (UVA):      Yes
  Device supports Managed Memory:                Yes
  Device supports Compute Preemption:            Yes
  Supports Cooperative Kernel Launch:            Yes
  Supports MultiDevice Co-op Kernel Launch:      Yes
  Device PCI Domain ID / Bus ID / location ID:   0 / 1 / 0
  Compute Mode:
     < Default (multiple host threads can use ::cudaSetDevice() with device simultaneously) >
```

Nvidia TURING

- arch=sm_75
- compute_75

So, for example,

nvcc -arch=sm_75 -o first first.cu -run

Memory Management - allocation

- The GPU has a separate memory space from the host CPU
- We cannot simply pass normal C pointers to CUDA threads
- Need to manage GPU memory and copy data to and from it explicitly
- cudaMalloc is used to allocate GPU memory
- cudaFree releases it again

```
float *a;

cudaMalloc(&a, N*sizeof(float));

…

cudaFree(a);
```

Memory Management - cudaMemcpy

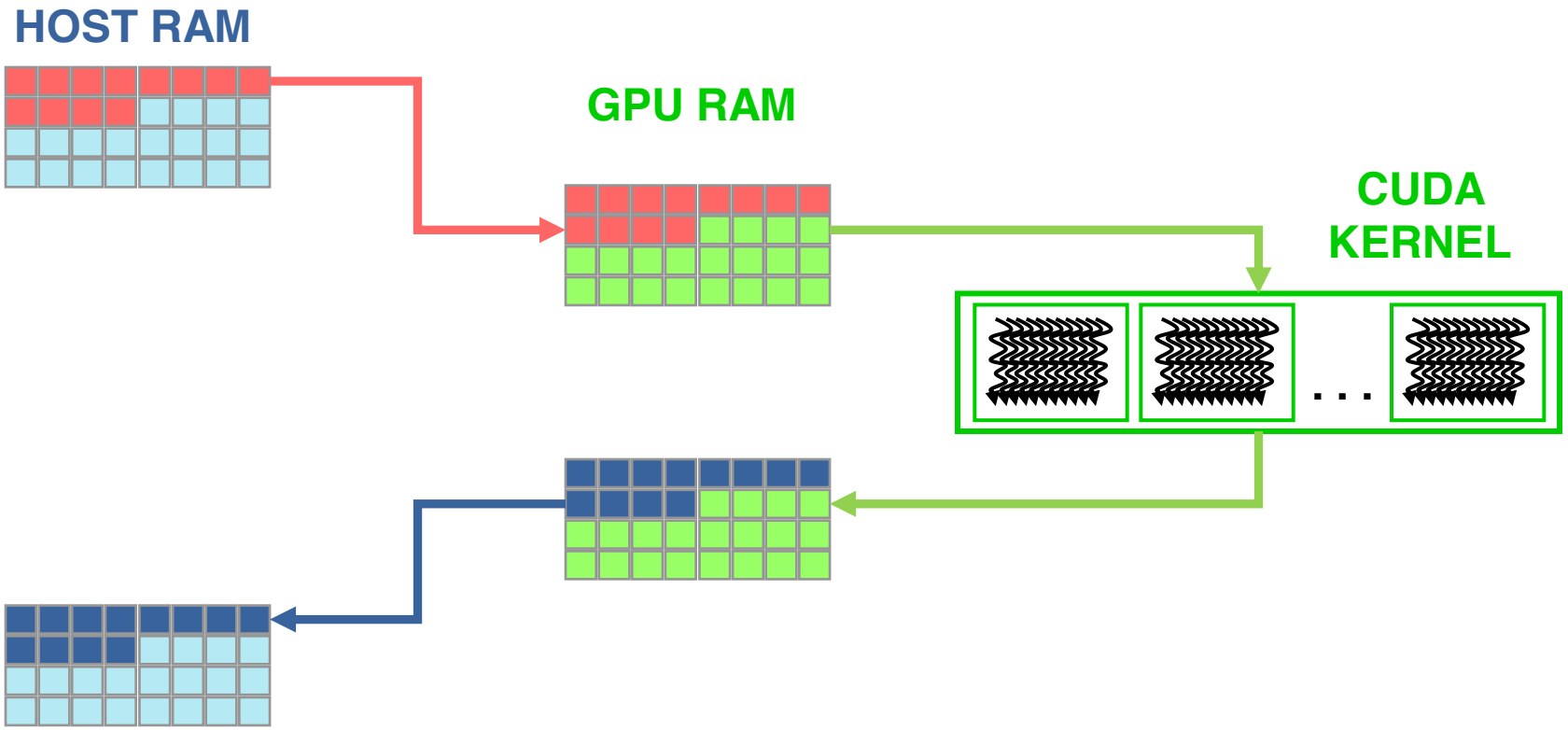
- Once we've allocated GPU memory, we need to be able to copy data to and from it
- cudaMemcpy does this:

```
cudaMemcpy(array_device, array_host, N*sizeof(float),
      cudaMemcpyHostToDevice);
cudaMemcpy(array_host, array_device, N*sizeof(float),
                  cudaMemcpyDeviceToHost);
```

- The first argument always corresponds to the *destination* of the transfer.
- Transfers between host and device memory are relatively slow and can become a bottleneck, so should be minimised when possible

Data movement

- data must be moved from HOST to DEVICE memory in order to be processed by a CUDA kernel
- when data is processed, and no more needed on the GPU, it is transferred back to HOST



The full example

```
#include <stdio.h>
#include <sys/time.h>
#define N   (32 * 1024)

__global__ void add( int *a, int *b, int *c ) {
   int tid = blockIdx.x*blockDim.x + threadIdx.x;
   if (tid < N) c[tid] = a[tid] + b[tid];
}

int main( void ) {
   int *a, *b, *c, *dev_a, *dev_b, *dev_c;
   struct timeval t1, t2;
   dim3 threads(32);
   dim3 blocks ( (N+threads.x-1)/threads.x );

   a = (int*)malloc( N * sizeof(int) );   //the same for b and c
   cudaMalloc( (void**)&dev_a, N * sizeof(int) ); );
   //the same for dev_b and dev_c

   for (int i=0; i<N; i++) { a[i] = i;   b[i] = 2 * i; }
   gettimeofday(&t1, 0);
```

```
// copy the arrays 'a' and 'b' to the GPU
   cudaMemcpy( dev_a, a, N * sizeof(int), cudaMemcpyHostToDevice
);
   cudaMemcpy( dev_b, b, N * sizeof(int),
cudaMemcpyHostToDevice );

    add<<<blocks,threads>>>( dev_a, dev_b, dev_c );

   // copy the array 'c' back from the GPU to the CPU
   cudaMemcpy( c, dev_c, N * sizeof(int), cudaMemcpyDeviceToHost
);
   cudaDeviceSynchronize();
   gettimeofday(&t2, 0);

   printf( "We did it!\n" );
  double time = (1000000.0*(t2.tv_sec-t1.tv_sec) + t2.tv_usec-
t1.tv_usec)/1000.0;
  printf("Time to generate:  %3.1f ms \n", time);

   // free the memory
   cudaFree( dev_a );
   free( a ); //the same for b and c
   return 0;
}
```

CUDA 6.x - Unified Memory

- Unified Memory creates a pool of memory with an address space that is shared between the CPU and GPU. In other word, a block of Unified Memory is accessible to both the CPU and GPU by using the same pointer;
- the system automatically *migrates* data allocated in Unified Memory mode between the host and device memory
  - no need to explicitly declare device memory regions
  - no need to explicitly copy back and forth data between CPU and GPU devices
  - greatly simplifies programming and speeds up CUDA ports
- REM: it can result in performances degradation with respect to an explicit, finely tuned data transfer.

SINGLE POINTER

Explicit vs Unified Memory

Explicit Memory Management

```
void *data, *d_data;
data = malloc(N);
cudaMalloc(&d_data, N);
cpu_func1(data, N);
cudaMemcpy(d_data, data, N, ...)
gpu_func2<<<...>>>(d_data, N);
cudaMemcpy(data, d_data, N, ...)
cudaFree(d_data);
cpu_func3(data, N);

free(data);
```

GPU code w/ Unified Memory

```
void *data;
data = malloc(N);

cpu_func1(data, N);

gpu_func2<<<...>>>(data, N);
cudaDeviceSynchronize();

cpu_func3(data, N);

free(data);
```

Sample code using CUDA Unified Memory

CPU code

```
void sortfile (FILE *fp, int N) {
  char *data;

  data = (char *) malloc (N);

  fread(data, 1, N, fp);

  qsort(data, N, 1, compare);



  use_data(data);

  free(data)
}
```

GPU code

```
void sortfile(FILE *fp, int N) {
  char *data;

  cudaMallocManaged(&data, N);

  fread(data, 1, N, compare);

  qsort<<< ... >>> (data, N, 1, compare);

  cudaDeviceSynchronize();

  use_data(data);

  cudaFree(data);
}
```

Checking CUDA Errors

- All CUDA API returns an error code of type **cudaError_t**
  - Special value **cudaSuccess** means that no error occurred
- CUDA runtime has a convenience function that translates a CUDA error into a readable string with a human understandable description of the type of error occured

```
char* cudaGetErrorString(cudaError_t code)
```

```
cudaError_t cerr = cudaMalloc(&d_a, size);

 if (cerr != cudaSuccess)
  fprintf(stderr, "%s\n", cudaGetErrorString(cerr));
```

- CUDA Asynchronous API returns an error which refers only on errors which may occur during the call on *host*
- CUDA kernels are asynchronous and void type so they don't return any error code

Checking Errors for CUDA kernels

- The error status is also held in an internal variable, which is modified by each CUDA API call or kernel launch.
- CUDA runtime has a function that returns the status of internal error variable.

`cudaError_t cudaGetLastError(void)`

1. Returns the status of internal error variable (**`cudaSuccess`** or other)
2. Resets the internal error status to **`cudaSuccess`**

- Error code from **`cudaGetLastError`** may refers to any other preceeding CUDA API runtime calls
- To check the error status of a CUDA kernel execution, we have to wait for kernel completition using the following synchronization API: **`cudaDeviceSynchronize()`**

```
// reset internal state
cudaError_t cerr = cudaGetLastError();
// launch kernel
kernelGPU<<<dimGrid,dimBlock>>>(...);
cudaDeviceSynchronize();
cerr = cudaGetLastError();
if (cerr != cudaSuccess)
  fprintf(stderr, "%s\n",
 cudaGetErrorString(cerr));
```

Checking CUDA Errors

- Error checking is strongly encouraged during developer phase
- Error checking may introduce overhead and unpleasant synchronizations during production run
- Error check code can become very verbose and tedious

  A common approach is to define a assert style preprocessor macro which can be turned on/off in a simple manner

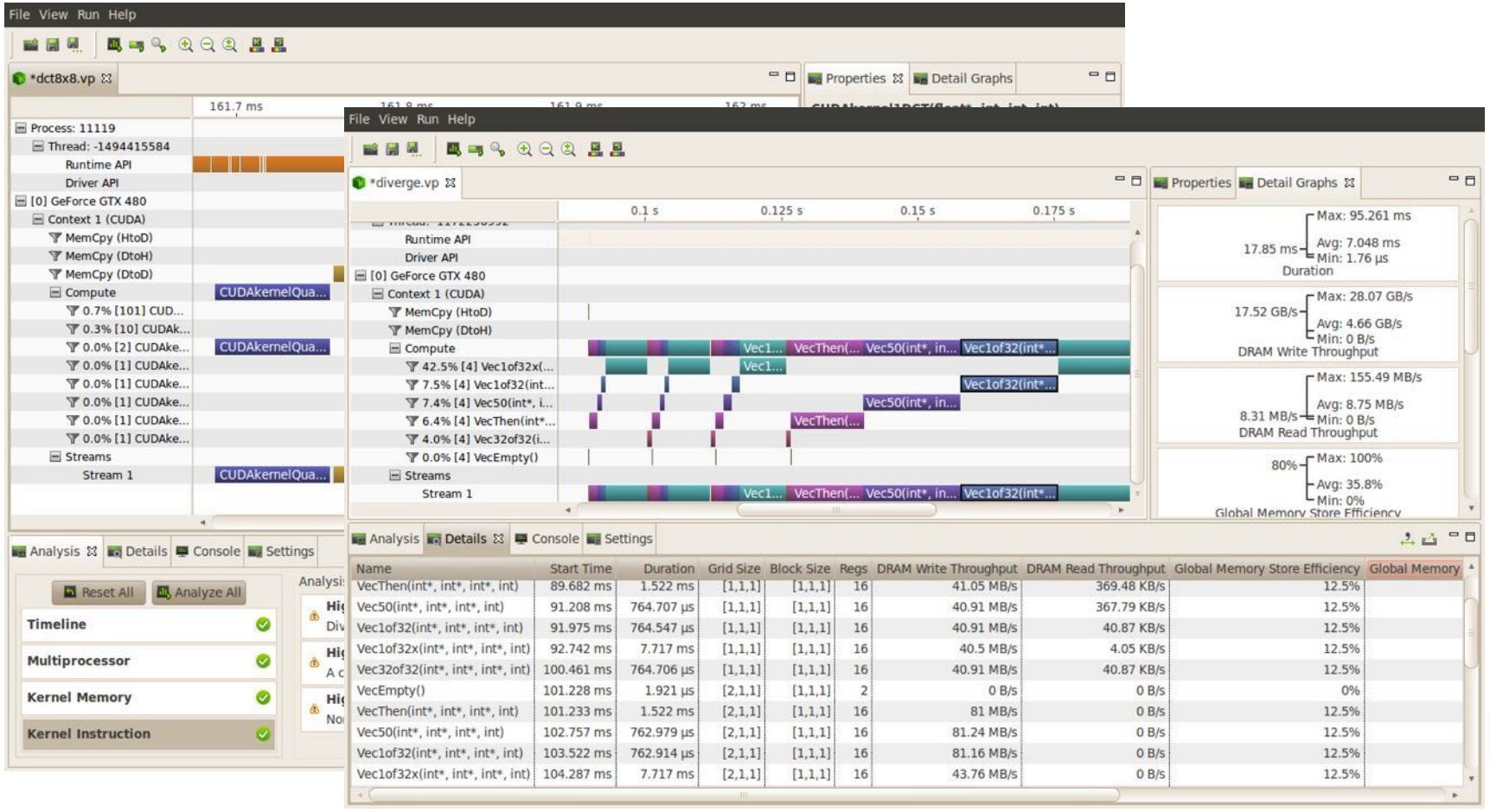
```
#define CUDA_CHECK(X) {\
 cudaError_t _m_cudaStat = X;\
 if(cudaSuccess != _m_cudaStat) {\
    fprintf(stderr,"\nCUDA_ERROR: %s in file %s line %d\n",\
    cudaGetErrorString(_m_cudaStat), __FILE__, __LINE__);\
    exit(1);\
 } }

...
CUDA_CHECK( cudaMemcpy(d_buf, h_buf, buffSize, cudaMemcpyHostToDevice) );
```

Development tools

- Common
  - Memory Checker
  - Built-in profiler
  - Visual Profiler
- Linux
  - CUDA GDB
  - Parallel Nsight for Eclipse
- Windows
  - Parallel Nsight for VisualStudio

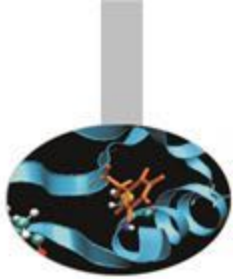
Profiling: Visual Profiler

- Traces execution at host, driver and kernel levels (unified timeline)
- Supports automated analysis (hardware counters)



Parallel NSight

https://developer.nvidia.com/tools-overview

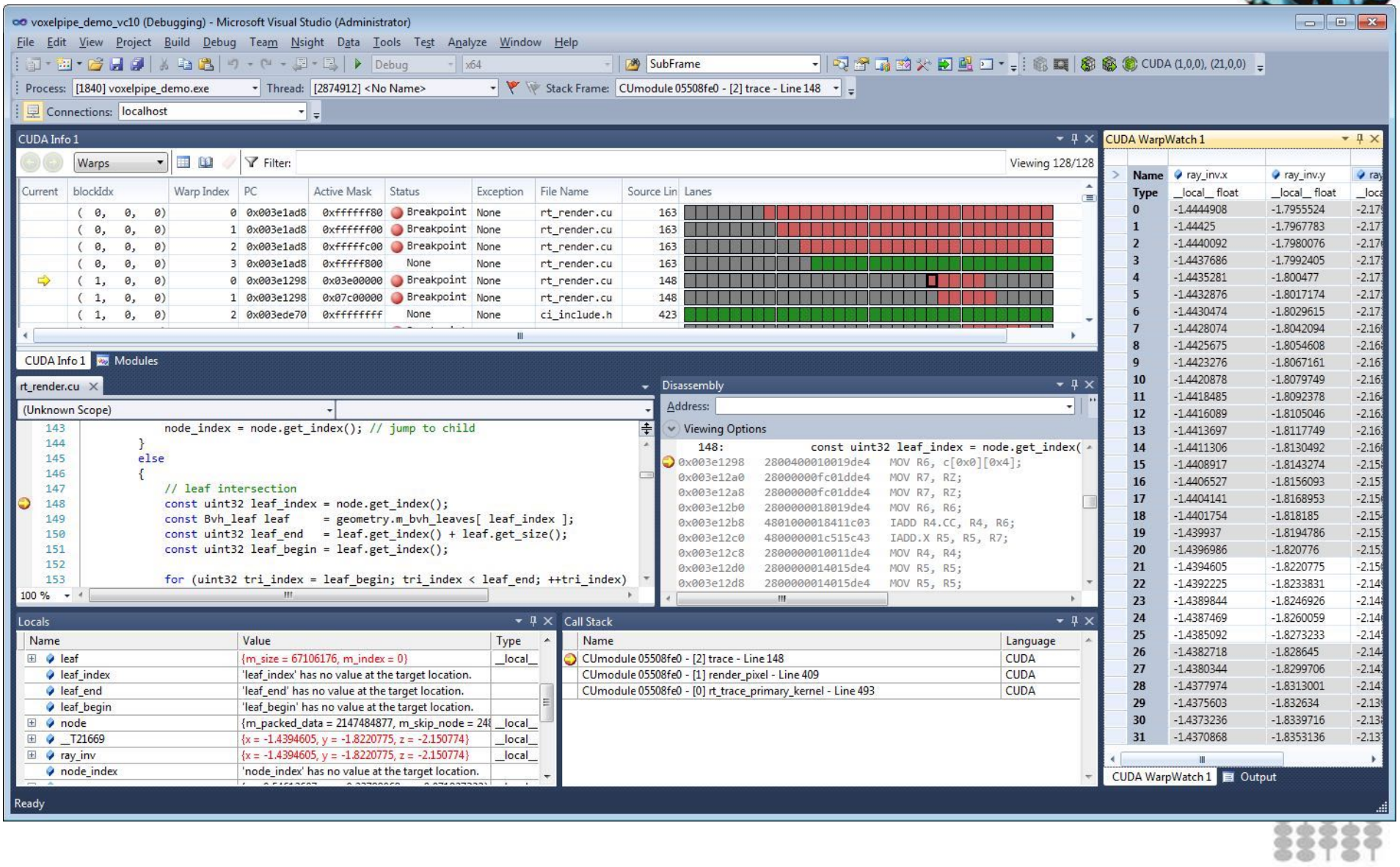


- Plug-in for major IDEs (Eclipse and VisualStudio)
- Aggregates all external functionalities:
  - Debugger (fully integrated)
  - Visual Profiler
  - Memory correctness checker
- As a plug-in, it extends all the convenience of IDEs to CUDA

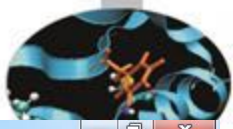
On Windows systems:

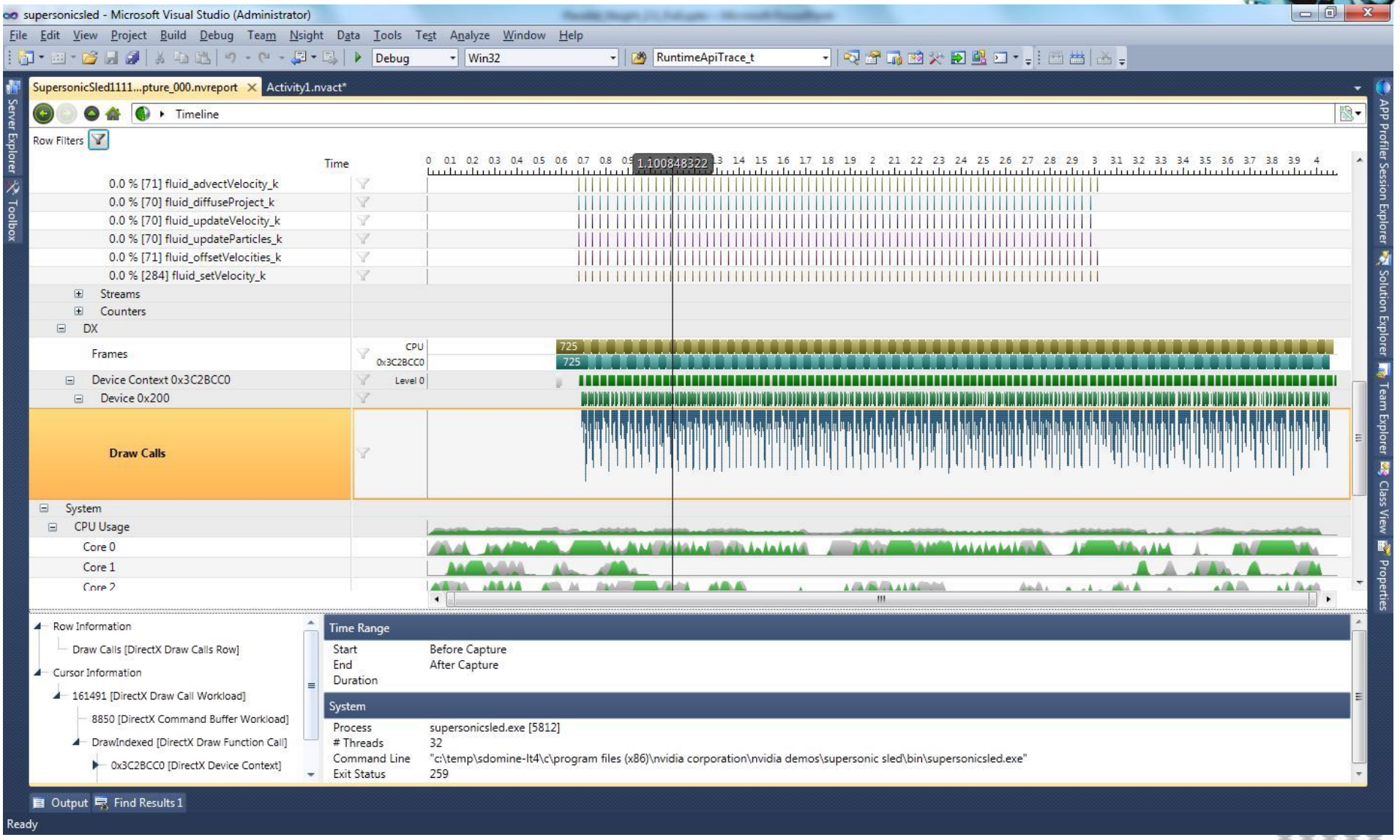
- Now works on a single GPU
- Supports remote debugging and profiling
- Latest version (2.2) introduced live PTX assembly view, warp inspector and expression lamination

Parallel NSight



Parallel NSight





Other CUDA command line programs

- nvidia-smi
  - Shows which GPUs are available and gives information about them
  - Can be used in scrolling mode when running CUDA programs
- nvprof
  - Quick profiler, useful for showing memory transfers between host and device.
  - More sophisticated profiling can be done with nvvp.
- cuda-memcheck
  - Ideal for spotting memory leaks in the CUDA program. Will considerably slow execution.
- cuda-gdb
  - CUDA debugger

Debugging: CUDA-MEMCHECK

- It's able to detect buffer overflows, misaligned global memory accesses and leaks
- Device-side allocations are supported
- Standalone or fully integrated in CUDA-GDB

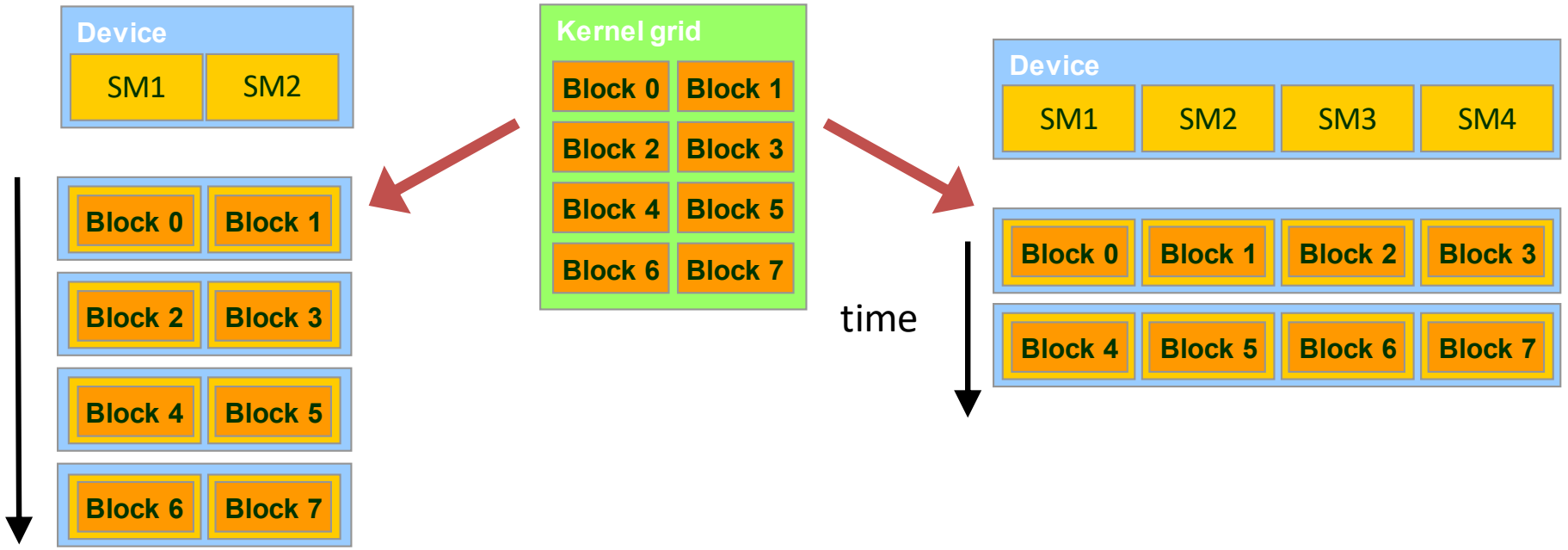
```
$ cuda-memcheck --continue ./memcheck_demo
========= CUDA-MEMCHECK
Mallocing memory
Running unaligned_kernel
Ran unaligned_kernel: no error
Sync: no error
Running out_of_bounds_kernel
Ran out_of_bounds_kernel: no error
Sync: no error
========= Invalid __global__ write of size 4
========= at 0x00000038 in memcheck_demo.cu:5:unaligned_kernel
========= by thread (0,0,0) in block (0,0,0)
========= Address 0x200200001 is misaligned
=========
========= Invalid __global__ write of size 4
========= at 0x00000030 in memcheck_demo.cu:10:out_of_bounds_kernel
========= by thread (0,0,0) in block (0,0,0)
========= Address 0x87654320 is out of bounds
=========
=========
========= ERROR SUMMARY: 2 errors
```

Some more details

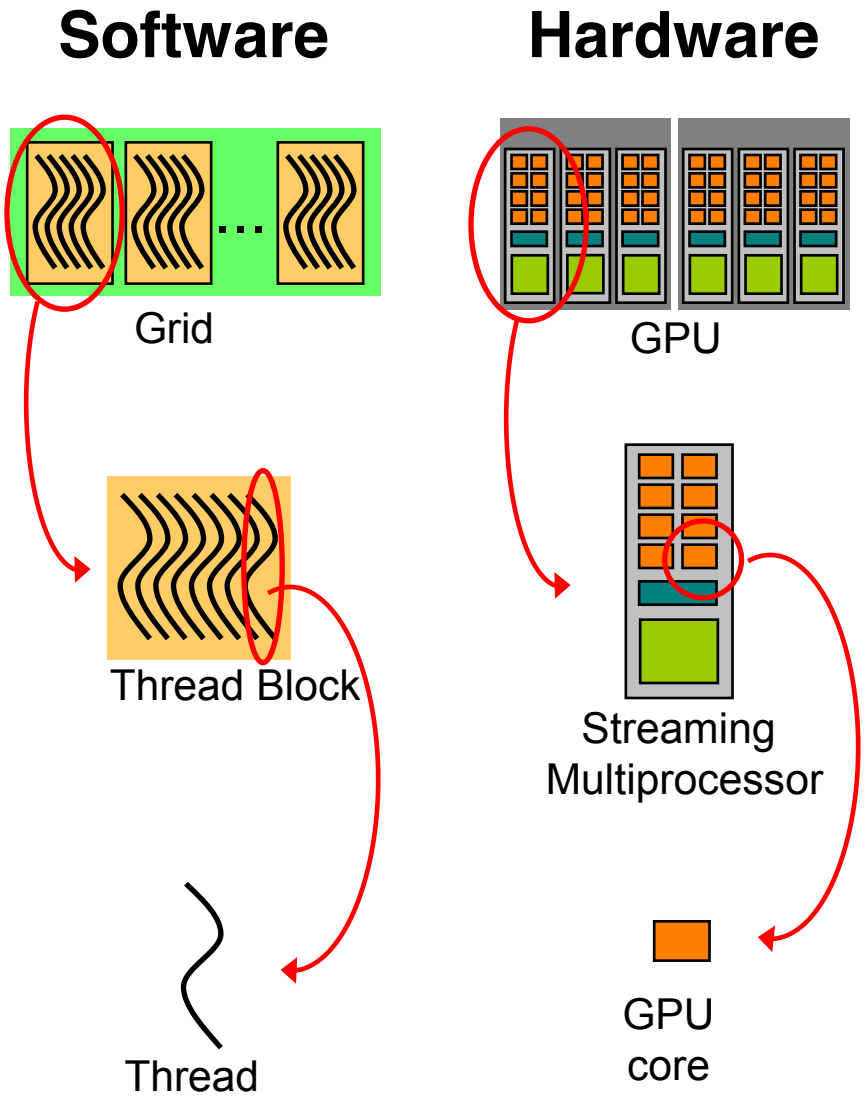
Trasparent Scalability

The GPU runtime system can execute blocks in any order relative to each other

This flexibility enables to execute the same application code on hardware with different numbers of SMs.



more on the GPU Execution Model



when a GPU kernel is invoked:

- each thread block is assigned to a SM in a round-robin mode
  - a maximum number of blocks can be assigned to each SM, depending on hardware generation and on how many resorces each requires (registers, shared memory, etc)
  - the runtime system maintains a list of active blocks and assigns new blocks to SMs as they complete
  - once a block is assigned to a SM, it remains on that SM until the work for all threads in the block is completed
  - each block execution is independent from the other (no synchronization is possible among them)
- threads of each block are partitioned into warps of consecutive *threads*
- the scheduler select for execution a warp from one of the residing blocks in each SM
- A warp execute one common set of instruction at a time
  - each GPU core take care of one thread in the warp
  - fully efficiency when all threads agree on their execution path

CUDA and NVIDIA GPUs

https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#compute-capabilities

How many threads and blocks can I use?

Depends on the *compute capability* of the device which describes the GPU features available.

Code name	Product name	Compute capability	SMM units	Max threads/ block	Max thread blocks/s m	#cores (FP32)
Kepler (GK210)	Tesla K40	3.7	15	1024	16	2496
Maxwell	Tesla M40	5.2	24	1024	32	3072
Pascal	Tesla P100	6.0	56	1024	32	3584
Volta	Tesla V100	7.0	80	1024	32	5120

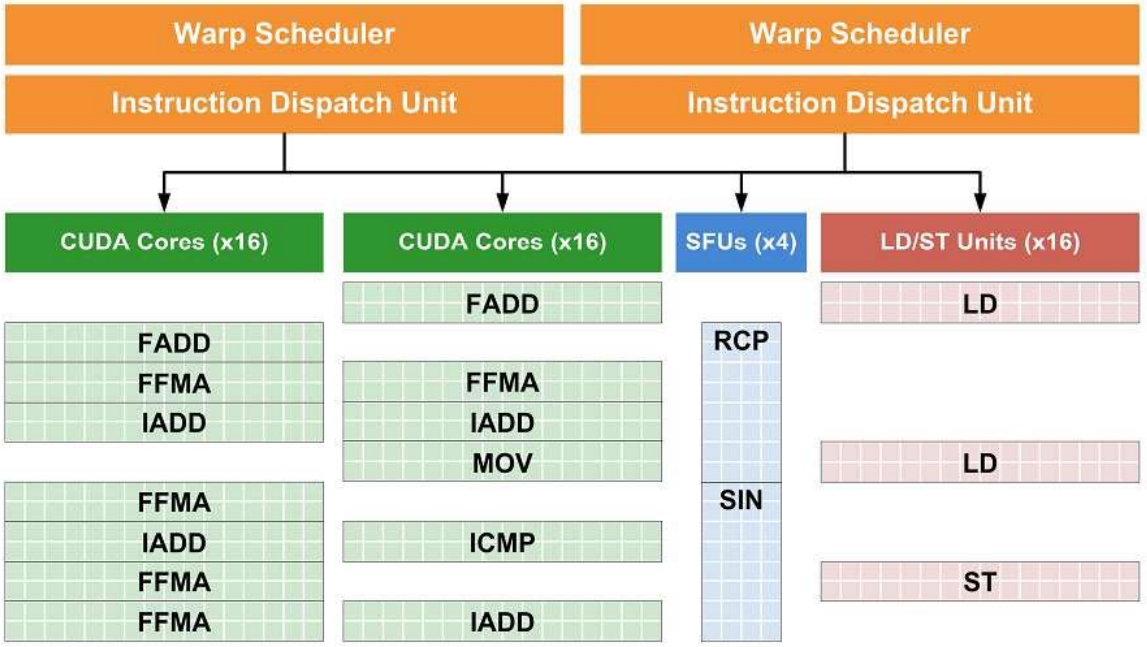
But often makes sense to set threads/block=1024 and make the number of blocks = problem_size/1024

	Compute Capability											
Technical Specifications	**3.0**	**3.2**	**3.5**	**3.7**	**5.0**	**5.2**	**5.3**	**6.0**	**6.1**	**6.2**	**7.0**	**7.5**
Maximum number of resident grids per device (Concurrent Kernel Execution)	16	4	32				16	128	32	16	128	
Maximum dimensionality of grid of thread blocks	3											
Maximum x-dimension of a grid of thread blocks	$2^{31}-1$											
Maximum y- or z-dimension of a grid of thread blocks	65535											
Maximum dimensionality of thread block	3											
Maximum x- or y-dimension of a block	1024											
Maximum z-dimension of a block	64											
Maximum number of threads per block	1024											
Warp size	32											
Maximum number of resident blocks per multiprocessor	16				32							16
Maximum number of resident warps per multiprocessor	64											32
Maximum number of resident threads per multiprocessor	2048											1024
Number of 32-bit registers per multiprocessor	64 K			128 K	64 K							
Maximum number of 32-bit registers per thread block	64 K	32 K	64 K				32 K	64 K		32 K	64 K	
Maximum number of 32-bit registers per thread	63	255										
Maximum amount of shared memory per multiprocessor	48 KB			112 KB	64 KB	96 KB	64 KB		96 KB	64 KB	96 KB	64 KB
Maximum amount of shared memory per thread block [27]	48 KB										96 KB	64 KB
Number of shared memory banks	32											
Amount of local memory per thread	512 KB											
Constant memory size	64 KB											
Cache working set per multiprocessor for constant memory	8 KB							4 KB	8 KB			
Cache working set per multiprocessor for texture memory	Between 12 KB and 48 KB							Between 24 KB and 48 KB			32 ~ 128 KB	32 or 64 KB

Warps

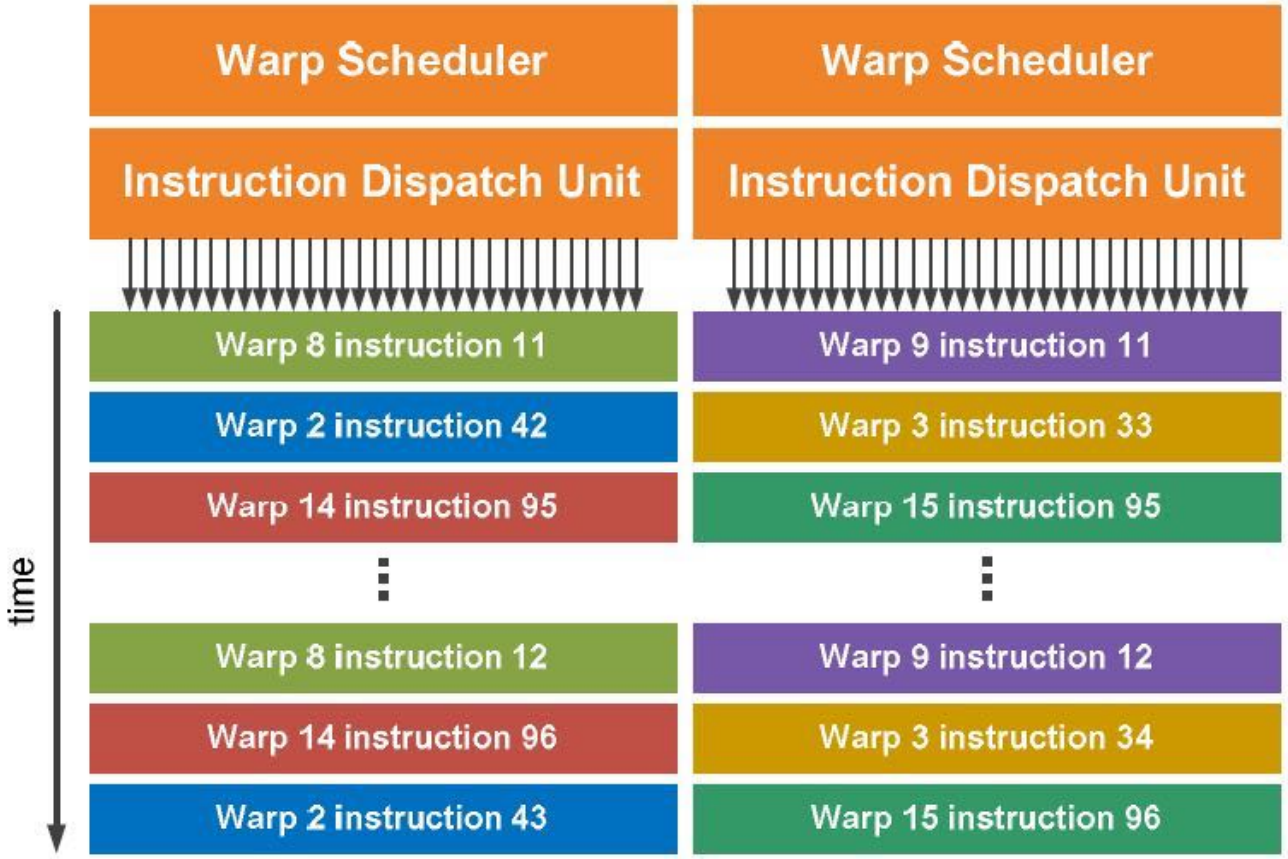
- The GPU multiprocessor creates, manages, schedules, and executes threads in groups of 32 parallel threads called *warps*.
- Individual threads composing a warp start together at the same program address, but they have their own instruction address counter and register state and are therefore free to branch and execute independently

- each *warp* can execute instructions on
  - SM cores
  - load/store units
  - SFUs units



The SM warp scheduler

- The NVIDIA SM schedules threads in groups of 32 threads, called *warps*
- Using 2 warp schedulers per SM allows two warps to be issued and executed concurrently if hardware resources are available



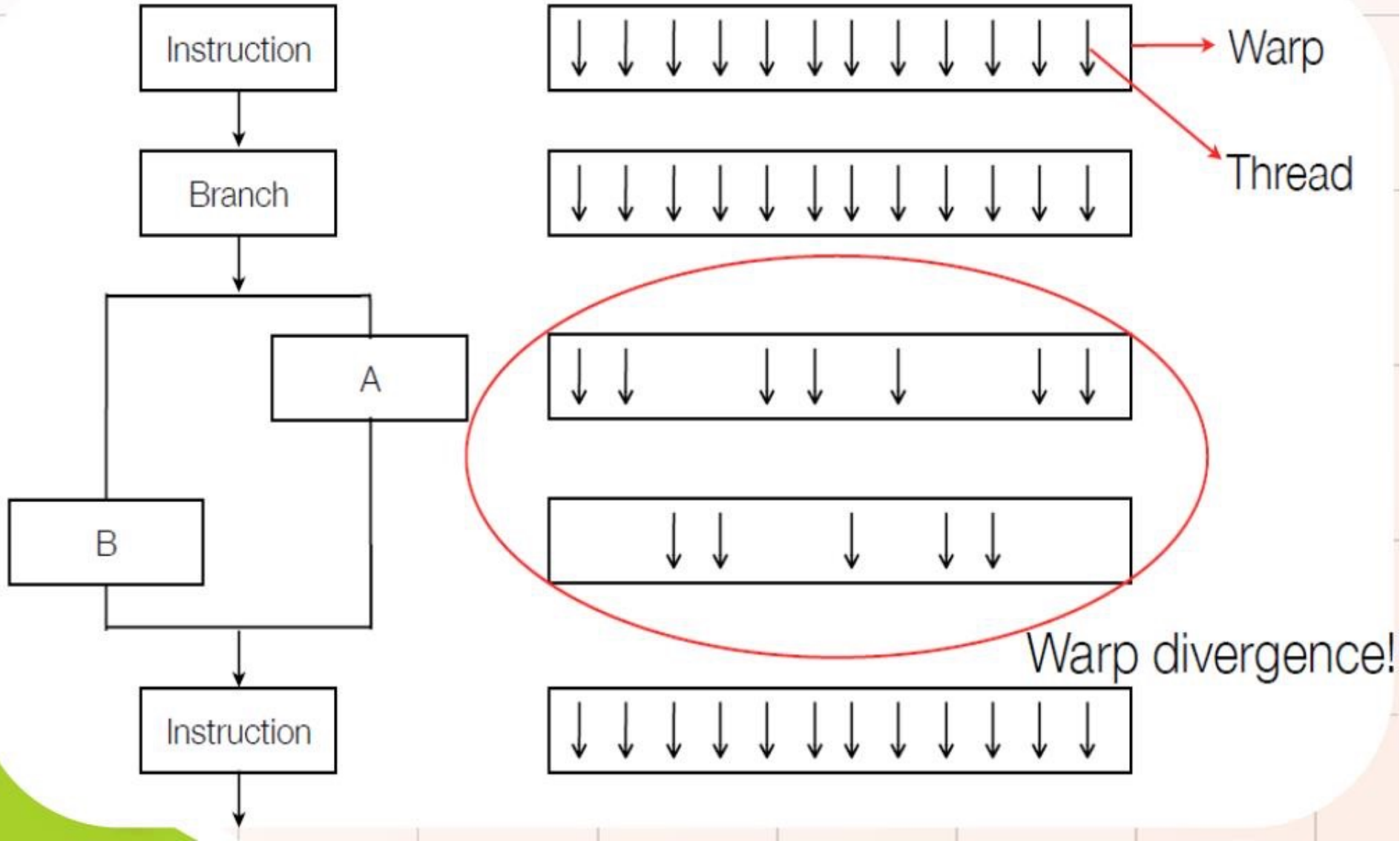
Warps

- A warp executes one common instruction at a time, so full efficiency is realized when all threads of a warp agree on their execution path.
- If threads of a warp diverge via a data-dependent conditional branch**, the warp serially executes each branch path taken**, disabling threads that are not on that path, and when all paths complete, the threads converge back to the same execution path.
- **Branch divergence occurs only within a warp**; different warps execute independently regardless of whether they are executing common or disjointed code paths.
- Each single instruction **in a warp** is performed in a lockstep. The next instruction can be fetched only when the previous one has completed.
- An SM statically distributes its warps among its schedulers. Then, at every instruction issue time, each scheduler issues one instruction for one of its assigned warps (half and quarter-warp) that is ready to execute, if any.
- Volta is equipped with 4 warp-scheduler units. Instructions are performed over two cycles, and the schedulers can issue independent instructions every cycle. Dependent instruction issue latency for core FMA math operations are reduced to four clock cycles, so **execution latencies of core math operations can be hidden** by as few as 4 warps per SM, assuming 4-way instruction-level parallelism *ILP* per warp. Many more warps are, of course, recommended to cover the much greater latency of memory transactions and control-flow operations.

https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#simt-architecture

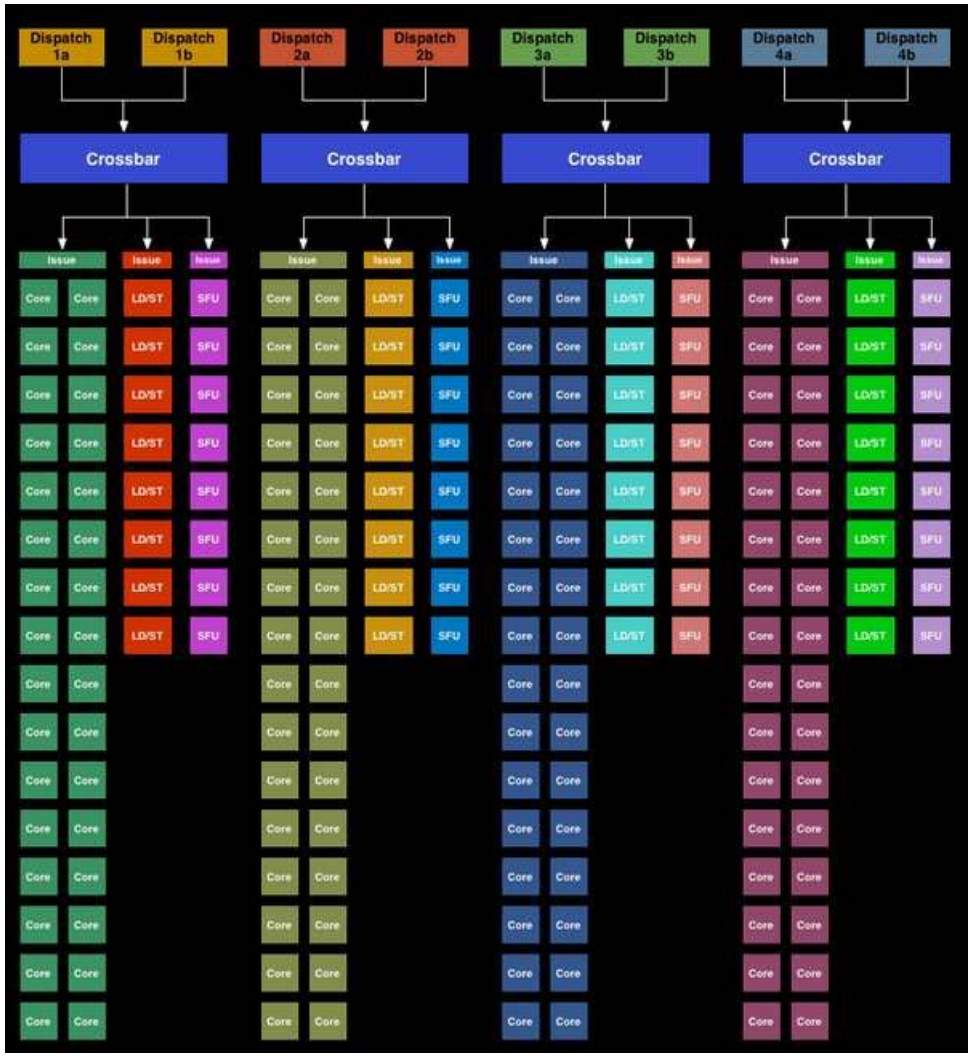
MANY DETAILS HERE http://taylorlloyd.ca/gpu,/pascal,/cuda/2017/01/07/gpu-pipelines.html

Flow Divergence



Volta SM Warp Scheduler

- Volta SM has 4 warp scheduler
- Each scheduler is responsible for
  - feeding 32 CUDA cores
  - 8 load/store units
  - 8 Special Function Units
- There are two dispatch ports per warp schedule
  - a warp scheduler can use little instruction level parallelism (ILP) by issuing a second instruction to an unused resource



Instruction Execution

Example

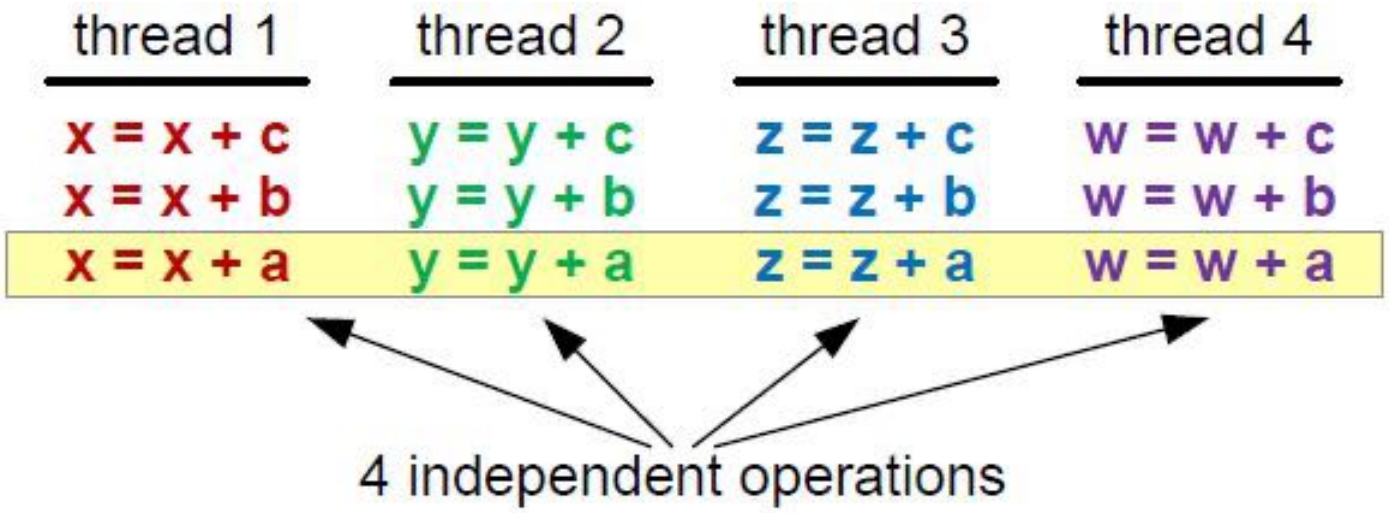
- a single Volta processing block has 16 FP32/INT32 and 8 FP64 ALU units
- a CUDA warps is 32 threads wide
- a FP32 operation on a warp will execute in 32 threads / 16 FP32 ALU = 2 cycles
- a FP64 operation on a warp will execute in 32 threads / 8 FP64 ALU = 4 cycles
- Each arithmetic operation has a pipe line stage so that, as soon as one warp has entered the first stage, a second independent warp can push its operand into the pipeline.
- FMA operations have four clock cycles on Volta: execution latencies of FMA core math operations can be hidden by as few as 4 warps per SM, assuming 4-way instruction-level parallelism ILP per warp

Hiding Latencies

- What is latency?
  - the number of clock cycles needed to complete an istruction
  - ... that is, the number of cycles I need to wait for before another **dependent operation** can start
    - arithmetic latency (~ 18-24 cycles)
    - memory access latency (~ 400-800 cycles)
- We cannot discard latencies (it's an hardware design effect), but we can lesser their effect and hide them.
  - saturating computational pipelines in computational bound problems
  - saturating bandwidth in memory bound problems
- We can organize our code so to provide the scheduler a sufficient number of **independent operations,** so that the more the warp are available, the more content-switch can hide latencies and proceed with other useful operations
- There are two possible ways and paradigms to use (can be combined too!)
  - Thread-Level Parallelism (TLP)
  - Instruction-Level Parallelism (ILP)

Thread-Level Parallelism (TLP)

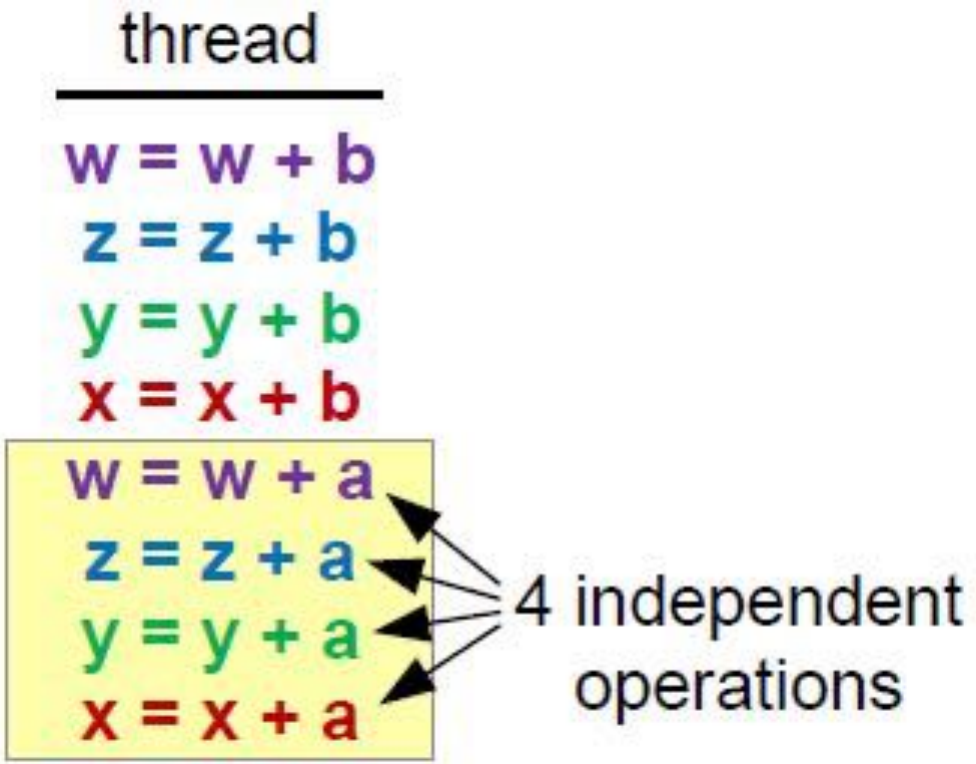
- Strive for high SM **occupancy**: that is try to provide as much threads per SM as possible, so to easy the scheduler find a warp ready to execute, while the others are still busy
- This kind of approach is effective when there is a low level of independet operations per CUDA kernels



Instruction-Level Parallelism (ILP)

- Strive for multiple independent operations inside you CUDA kernel: that is, let your kernel act on more than one data
- this will grant the scheduler to stay on the same warp and fully load each hardware pipeline

- note: the scheduler will not select a new warp untill there are eligible instructions ready to execute on the current warp



Branching example

- E.g you want to split your threads into 2 groups:

```
i = blockIdx.x*blockDim.x + threadIdx.x;
if (i%2 == 0)
   …
else
   …
```

Threads within warp diverge

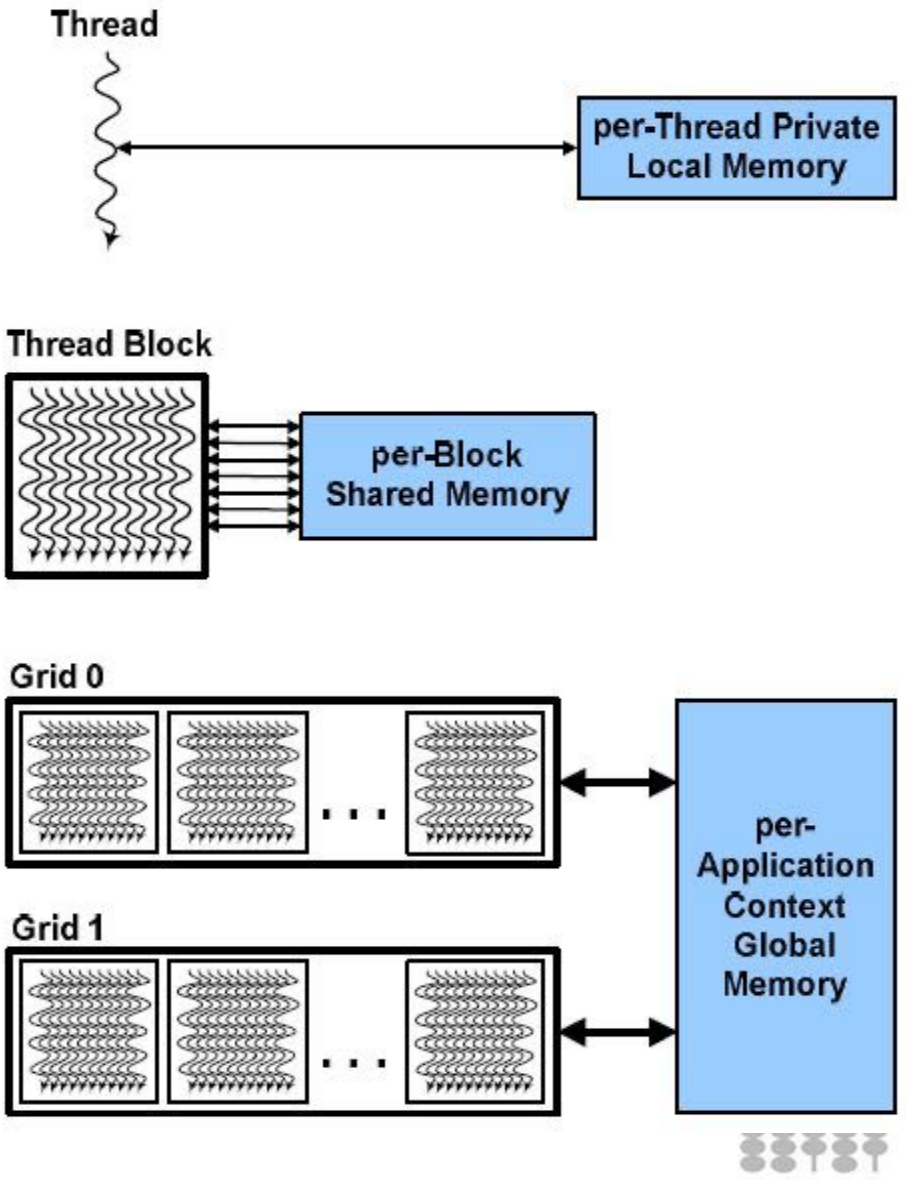
```
i = blockIdx.x*blockDim.x + threadIdx.x;
if ((i/32)%2 == 0)
   …
else
   …
```

Threads within warp follow same path

Hierarchy of device memories

CUDA's hierarchy of threads maps to a hierarchy of memories on the GPU:

- Each thread has some **registers**, used to hold automatic scalar variables declared in kernel and device functions, and a **per-thread private memory space** used for register spills, function calls, and C automatic array variables
- Each thread block has a **per-block shared memory space** used for inter-thread communication, data sharing, and result sharing in parallel algorithms
- Grids of thread blocks share results in **global memory space**



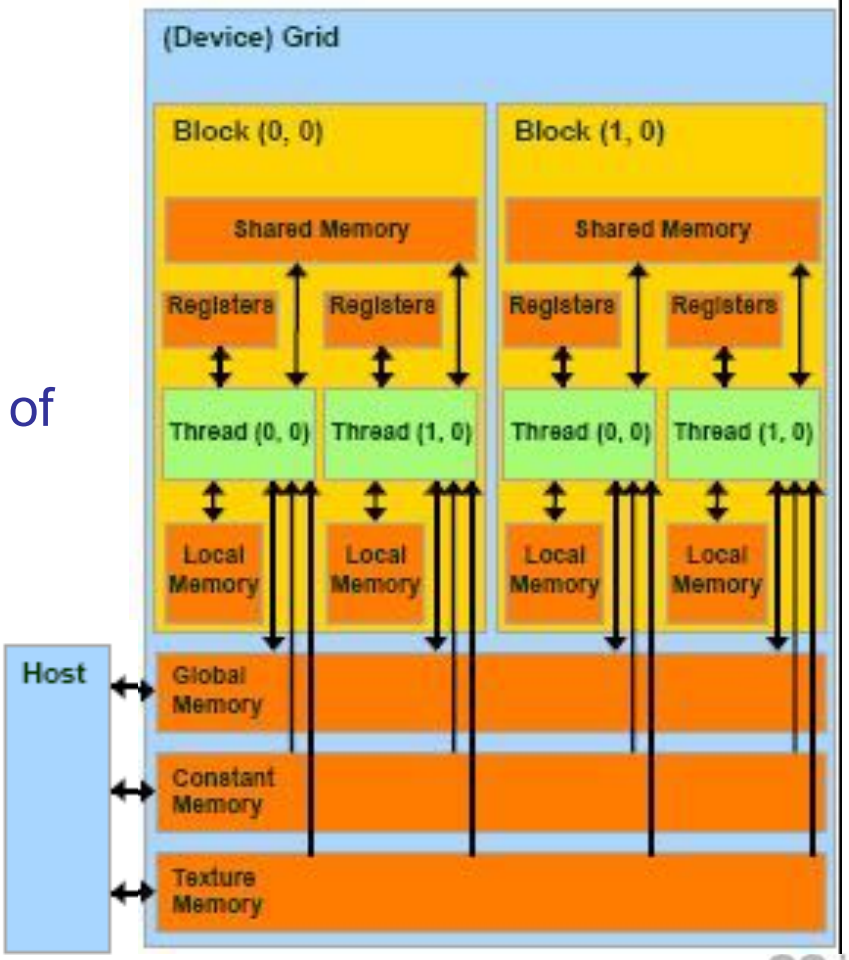
CUDA device memory model

on-chip memories:

- *registers* (~8KB) → SP
- *shared memory* (~16KB) → SM
- they can be accessed at very high speed in a highly parallel manner.

per-grid memories:

- *global memory* (~4GB)
  - long access latencies (hundreds of clock cycles)
  - finite access bandwith
- *constant memory* (~64KB)
  - read only
  - short-latency (cached) and high bandwith when all threads simultaneously access the same location
- *texture memory* (read only)
- CPU can transfer data to/from all per-grid memories.



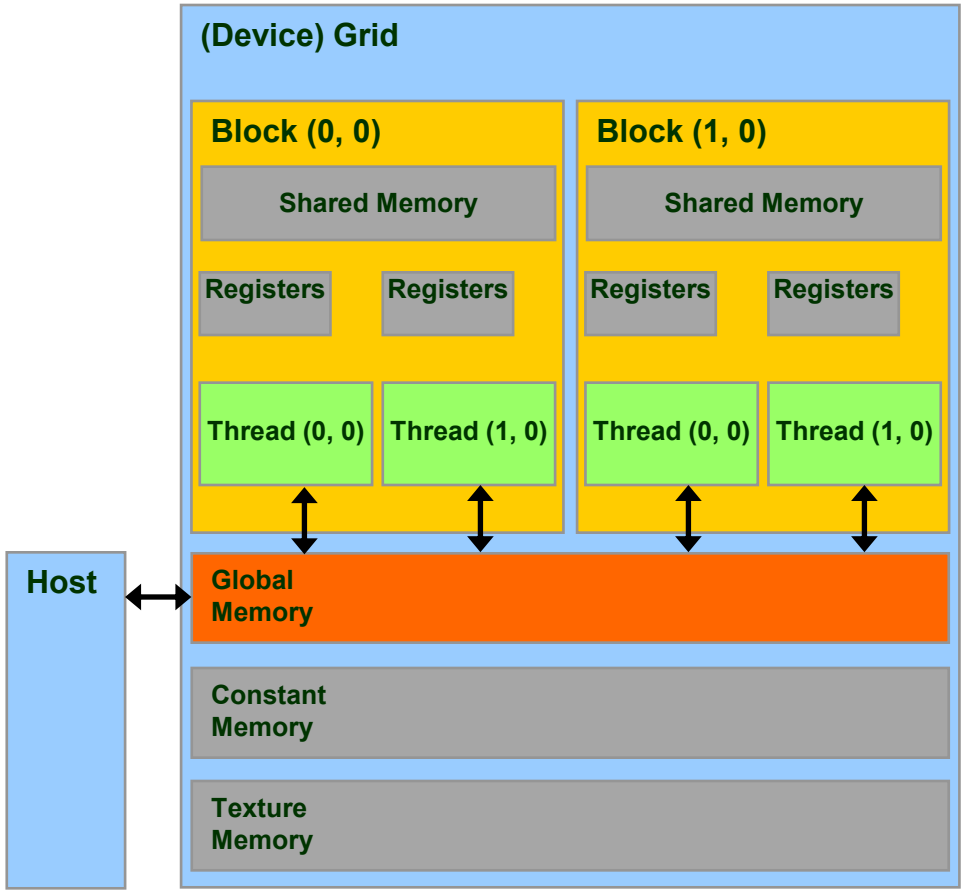
Local memory is implemented as part of the global memory, therefore has a long access latencies too.

Global Memory

- ***Global Memory*** is the larger memory available on a *device*
  - Comparable to a RAM for CPU
  - Its status is maintained among different kernel launches
  - Can be access both read/write from all threads of the kernel grid
  - Unique memory that can be used in read/write access from the CPU
  - **Very high bandwidth**

    Throughput > 900 GB/s
  - **Very high latency**

    about 400-800 clock cycles



Memory coalescing

- Global memory bandwidth for graphics memory on GPU is high compared to CPU
  - But there are many data-hungry cores
  - Memory bandwidth is a botteneck
- Maximum bandwidth achieved when data is loaded for multiple threads in a single transaction: coalescing
- This will happen when data access patterns meet certain conditions: 16 consecutive threads (half-warp) must access data from within the same memory segment
- E.g. condition met when consecutive threads read consecutive memory addresses within a warp.
- Otherwise, memory accesses are serialised, significantly degrading performance
- Adapting code to allow coalescing can dramatically improve performance

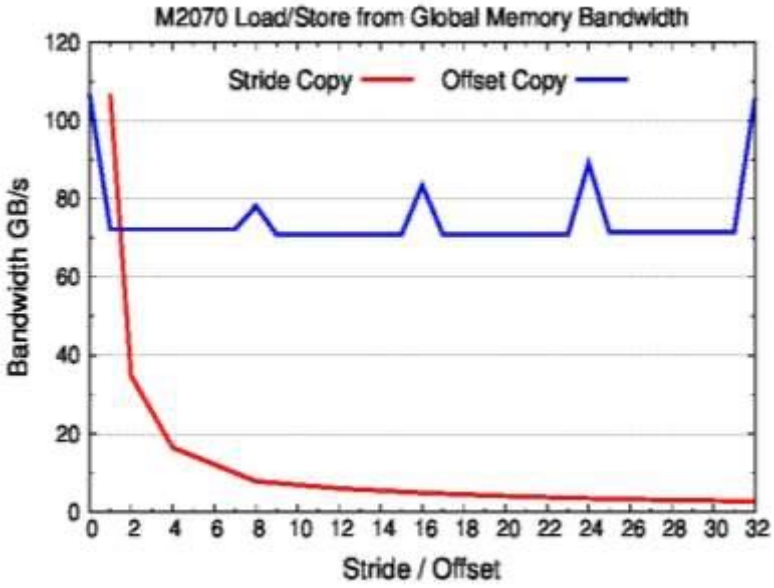
Global Memory Load/Store

```
// strided data copy
__global__ void strideCopy (int N, float *odata, float* idata, int stride) {
   int xid = (blockIdx.x*blockDim.x + threadIdx.x) * stride;
   if (xid < N) odata[xid] = idata[xid];
}
```

```
// offset data copy
__global__ void offsetCopy(int N, float *odata, float* idata, int offset) {
   int xid = blockIdx.x * blockDim.x + threadIdx.x + offset;
   if (idx < N) odata[xid] = idata[xid];
}
```

Stided based copy	
Stride	Bandwidth GB/s
1	106.6
2	34.8
8	7.9
16	4.9
32	2.7

Offset based copy	
Offset	Bandwidth GB/s
0	106.6
1	72.2
8	78.2
16	83.4
32	105.7



Measured on a M2070; Total elements = 16776960; Used Blocks = 65535; Block lenght = 256

Data alignment in *Global Memory*

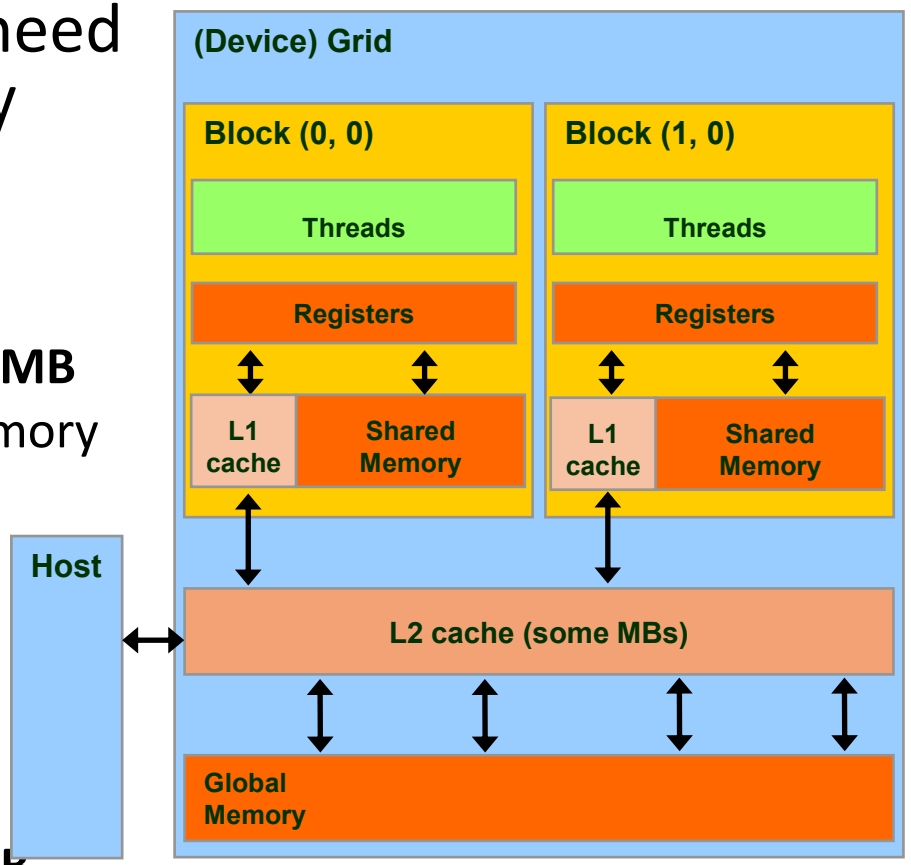
- It is very important to align data in memory so to have aligned accesses (*coalesced*) during load/store operation in global memory, reducing the number of segments moved across the bus
  - **`cudaMalloc()`** grants the alignment of first element in global memory, useful for one dimensional arrays
  - **`cudaMallocPitch()`** must be used to allocate 2d buffers
    - elements are padded so each row is aligned for coalescing accesses
    - returns an integer (`pitch`) which can be used as a stride to access row elements

```
// host code
int width = 64, heigth = 64; int pitch; float *devPtr;
cudaMallocPitch(&devPtr, &pitch, width * sizeof(float), height);

// device code
__global__  myKernel(float *devPtr, int pitch, int width, int height)
{
  for (int r = 0; r < height; r++) {
    float *row = devPtr + r * pitch;
    for (int c = 0; c < width; c++)
      float element = row[c];
  }
  ...
}
```

Cache Hierarchy for Global Memory Accesses

- GPU designs include cache hierarchy in order to easy the need for space and time data locality
- 2 Levels of cache:
  - **L2** : shared among all SM
    - **Kepler 1MB**, **Pascal 4MB, Volta 6MB**
    - 25% less latency than Global Memory

  NB : all accesses to the global memory pass through the L2 cache, also for H2D and D2H memory transfers
  - **L1** : private to each SM
    - [**16/48 KB**] configurable
    - L1 + Shared Memory = 64 KB
    - **Kepler** : configurable also as **32 KB**




```
cudaFuncSetCacheConfig(kernel1, cudaFuncCachePreferL1);     // 48KB L1 / 16KB ShMem
cudaFuncSetCacheConfig(kernel2, cudaFuncCachePreferShared); // 16KB L1 / 48KB ShMem
```

Set cache configuration

`cudaDeviceSetCacheConfig ( cudaFuncCache cacheConfig )`

Description:

- On devices where the L1 cache and shared memory use the same hardware resources, this sets through cacheConfig the preferred cache configuration for the current device. This is only a preference. **The runtime will use the requested configuration if possible, but it is free to choose a different configuration if required to execute the function**.
- Any function preference set via cudaFuncSetCacheConfig () will be preferred over this device-wide setting. Launching a kernel with a different preference than the most recent preference setting, may insert a device-side synchronization point.
- The supported cache configurations are:
  - cudaFuncCachePreferNone: no preference for shared memory or L1 (default)
  - cudaFuncCachePreferShared: prefer larger shared memory and smaller L1 cache
  - cudaFuncCachePreferL1: prefer larger L1 cache and smaller shared memory
  - cudaFuncCachePreferEqual: prefer equal size L1 cache and shared memory

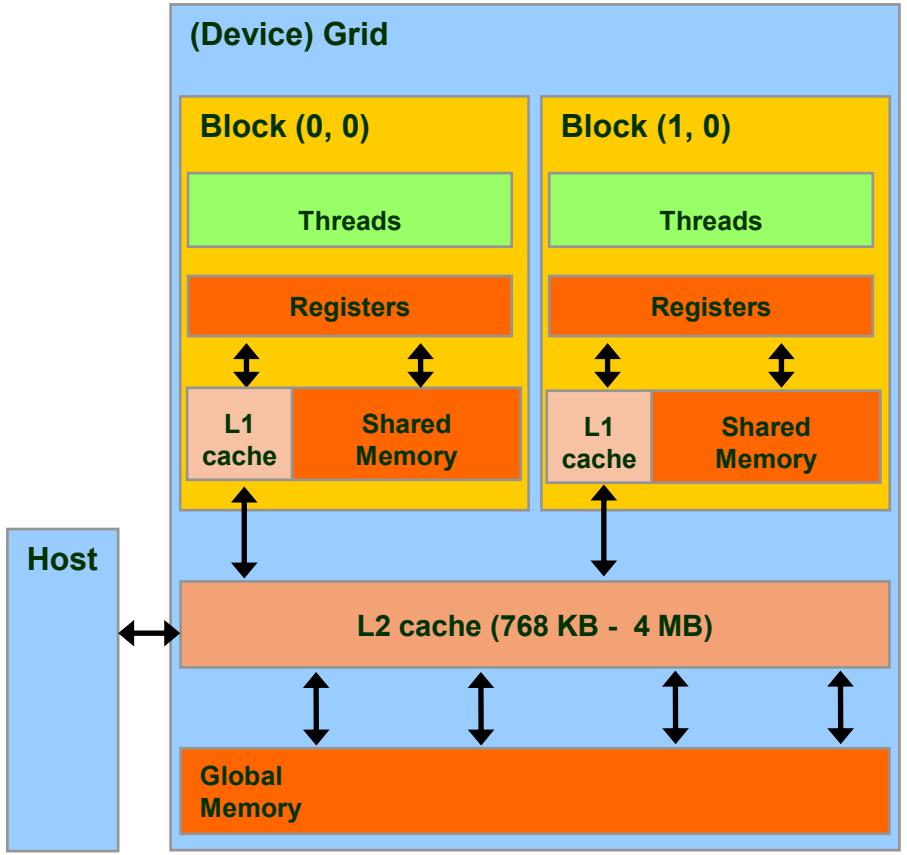
Cache Hierarchy for Global Memory Accesses

Just one type of ***store*** operation:

- when data should be updated in global memory, its L1 copy is invalidated and updated the L2 cache value

Two different type of ***load*** operations:

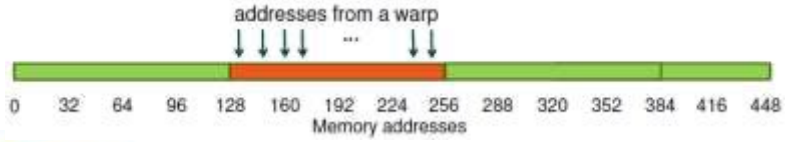
- **Caching (default mode)**
  - when data is requested by some thread, data is first searched in L1 cache, then in L2 cache, last in global memory
  - cache line lenght is **128-byte**
- **Non-caching (compile time selected)**
  - the L1 cache is disabled
  - when data is requested by some thread, data is first searched in L2 cache, then in global memory
  - cache line lenght is **32-bytes**
  - this mode is activated at *compile time* using the compiler option:
    `-Xptxas -dlcm=cg`

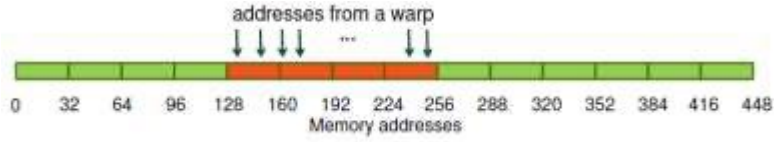


Load Operations from Global Memory

- All load/store requests in global memory are issued per *warp* (as all other instructions)
  1. each *thread* in a *warp* compute the address to access
  2. *load/store* units select segments where data resides
  3. *load/store* start transfer of needed segments

Warp requires 32 consecutive 4-byte word aligned to segment (total 128 bytes)	
Caching Load	**Non-caching Load**
all addresses belong to 1 line cache segment	all addresses belong to 4 line cache segments
128 bytes are moved over the bus	128 bytes are moved over the bus
bus utilization: **100%**	bus utilization: **100%**

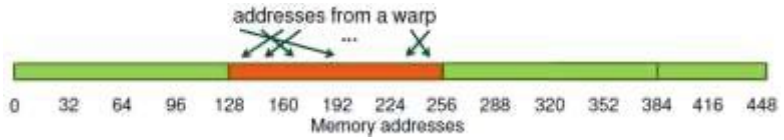


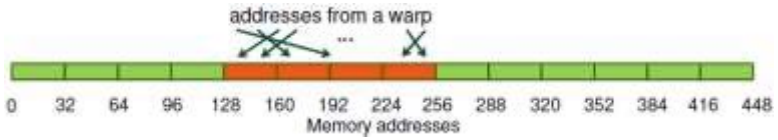


Load Operations from Global Memory

Warp requests 32 permuted 4-byte words alined to segment (total 128 bytes)

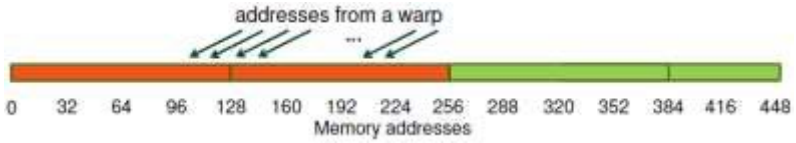
Caching Load	Non-caching Load
addresses belong to 1 line cache segments	addresses belong to 4 line cache segments
128 bytes are moved over the bus	128 bytes are moved over the bus
bus utilization: **100%**	bus utilization: **100%**

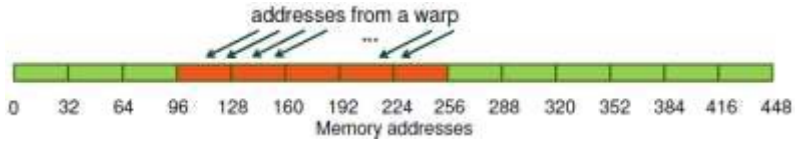




Warp requires 32 consecutive 4-bytes words not alined to segment (total 128 bytes)

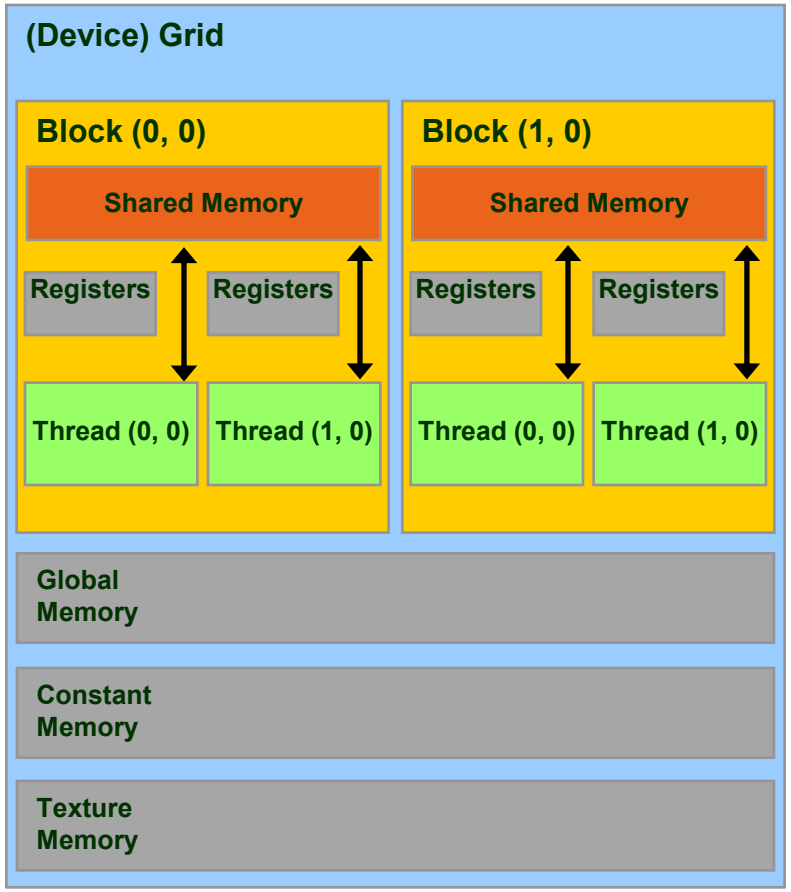
Caching Load	Non-caching Load
addresses belong to 2 line cache segments	addresses belong to 5 line cache segments
256 bytes are moved over the bus	160 are moved over the bus
bus utilization: **50%**	bus utilization: **80%**





Shared Memory

- The ***Shared Memory*** is a small, but quite fast memory mounted on each SM
  - read/write access for threads of blocks residing on the same SM
  - a cache memory under the direct control of the programmer
  - its status is not mantained among different kernel calls
- Specifications:
  - **Very low latency**: 2 clock cycles
  - Throughput: 32 bit every 2 cycles
  - Dimension : **48 KB [default]**
    (Configurable : 16/32/48 KB)



Shared Memory Allocation

```
// statically inside the kernel
__global__  myKernelOnGPU (...) {
  ...
  __shared__  type  shmem[MEMSZ];
  ...
}

or using dynamic allocation

// dynamically sized
extern  __shared__  type  *dynshmem;

__global__  myKernelOnGPU (...) {
  ...
  dynshmem[i] = ... ;
  ...
}

void myHostFunction() {
  ...
  myKernelOnGPU<<<gs,bs,MEMSZ>>>();
}
```

```
! statically inside the kernel
attribute(global)
  subroutine myKernel(...)
  ...
  type, shared:: variable_name
  ...
end subroutine

oppure

! dynamically sized
type, shared:: dynshmem(*)

attribute(global)
  subroutine myKernel(...)
  ...
  dynshmem(i) = ...
  ...
end subroutine
```

- variables allocated in shared memory has storage duration of the kernel launch (not persistent!)
- only accessible by threads of the same block

Thread Block Synchronization

- All threads in the same block can be synchronized using the CUDA runtime API call:

```
__syncthreads()  |  call syncthreads()
```

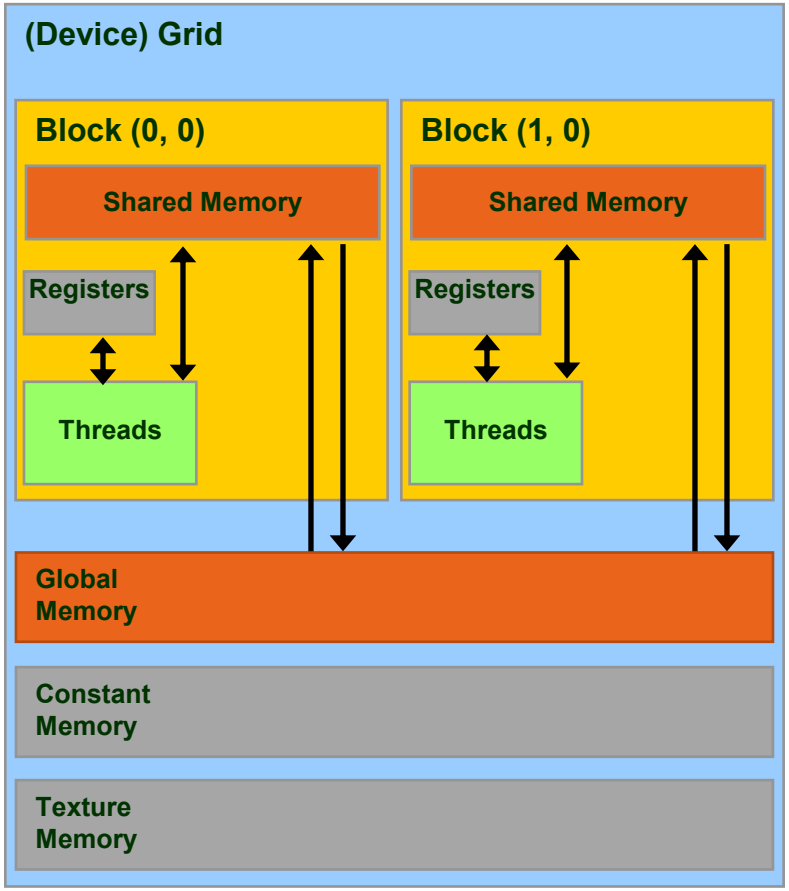
which blocks execution until all other threads reach the same call location

- can be used in conditional too, but only if all thread in the block reach the same synchronization call

"... otherwise the code execution is likely to hang or produce unintended side effects"

Using Shared Memory for Thread Cooperation

- Threads belonging to the same block can cooperate togheter using the shared memory to share data
  - if a thread is in need of some data which has been already retrived by another thread in the same block, this data can be shared using the shared memory
- typical Shared Memory usage pattern:
  - declare a buffer residing on shared memory (this buffer is per block)
  - load data into shared memory buffer
  - synchronize threads so to make sure all needed data is present in the buffer
  - performe operation on data
  - synchronize threads so all operations have been performed
  - write back results to global memory

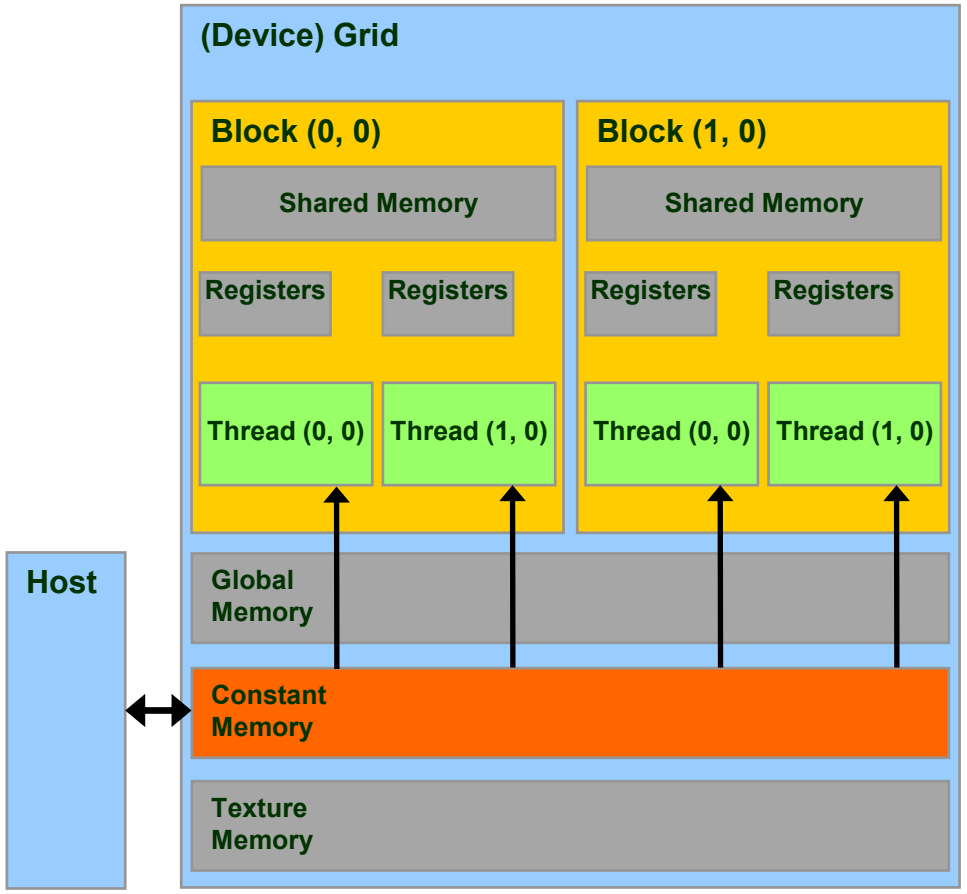


Kernel Synchronization

```
__global__ void vector_sum(int *in, int *out) {
    __shared__ int temp[BLOCK_SIZE+2*RADIUS];
    int gindex=threadIdx.x+blockIdx.x*blockDim.x;   // global index
    int lindex=threadIdx.x+RADIUS;                  // local index

    // Read input elements into shared memory
    temp[lindex]=in[gindex];
    if (threadIdx.x<RADIUS) {                       // some extra work
        temp[lindex-RADIUS]=in[gindex-RADIUS];
        temp[lindex+BLOCK_SIZE]=in[gindex+BLOCK_SIZE];
    }
    __syncthreads();
    int offset, result=0;
    for (offset=-RADIUS; offset<=RADIUS; offset++)
        result+=temp[lindex+offset];
    out[gindex]=result;
}
```

Constant Memory

- ***Constant Memory*** is the ideal place to store constant data in **read-only** access from all threads
  - constant memory data actually reside in the global memory, but fetched data is moved into a dedicated *constant-cache*
  - very effective when all *thread* of a *warp* request the same memory address
  - it's values are initialized from host code using a special CUDA API
- Specifications:
  - Dimension : **64 KB**
  - Throughput: 32 bits per warp every 2 clock cycles



Accessing Constant Memory

Suppose a kernel is launched using 320 warps per SM and all threads requests the same data

- if data is on global memory:
  - all *warp* will request the same segment from global memory
  - the first time segment is copied into L2 cache
  - if other data pass through L2, there are good chances it will be lost
  - there are good chances that data should be requested 320 times
- if data is in constant memory:
  - during first *warp* request, data is copied in *constant-cache*
  - since there is less traffic in *constant-cache , there are good chances all other warp will find the data already in cache*, so no more traffic on the BUS

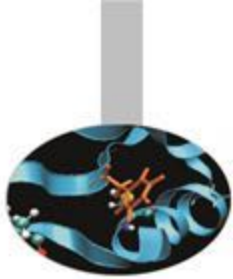
Constant Memory Allocation

```
__constant__  type  variable_name; // static

cudaMemcpyToSymbol(const_mem, &host_src, sizeof(type), cudaMemcpyHostToDevice);

// warning
// cannot be dynamically allocated
```

```
type, constant :: variable_name

! warning
! cannot be dynamically allocated
```

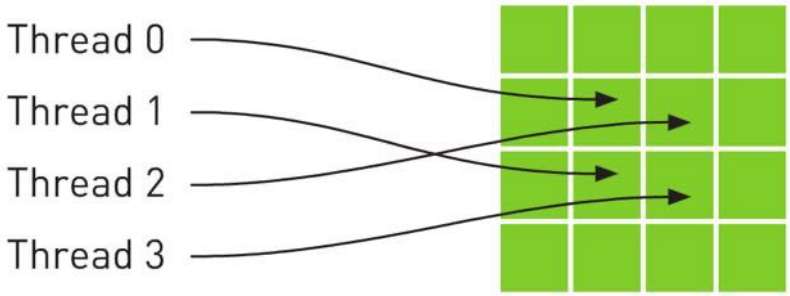
- data will reside in the constant memory address space
- has static storage duration (persists until the application ends)
- readable from all threads of a running kernel

Texture Memory



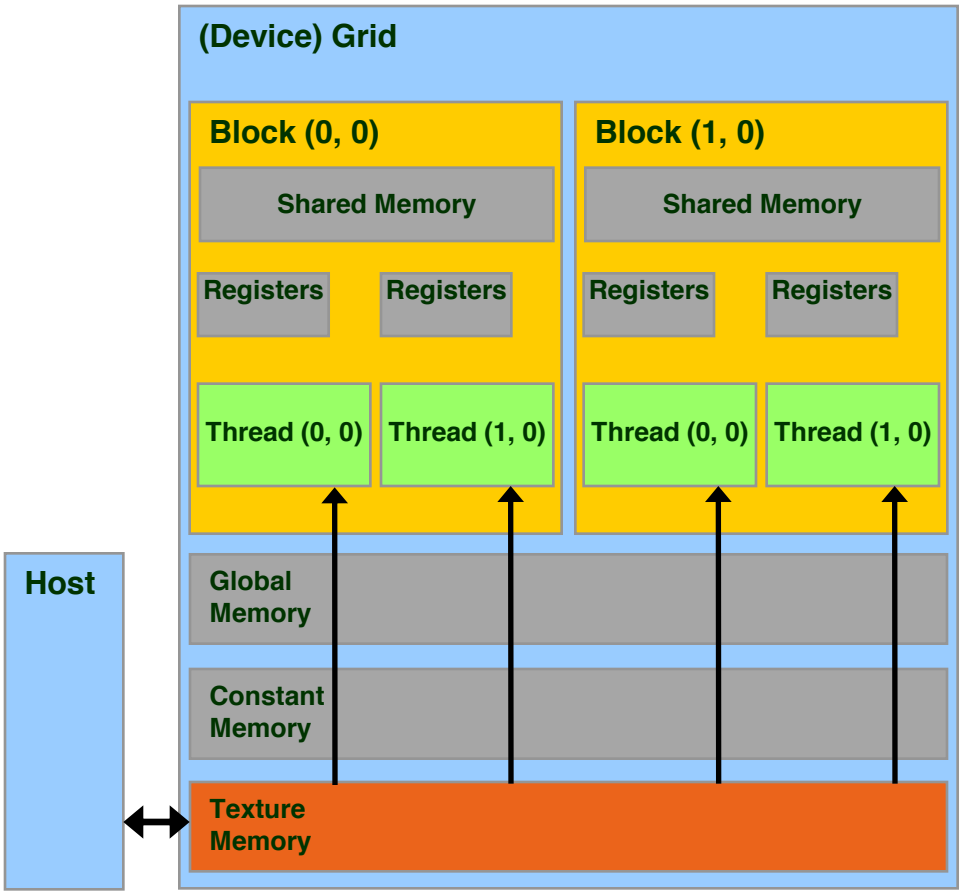
- **Read only**, must be set by the host;
- Load requests are cached (dedicated cache);
- specifically, texture memories and caches are designed for graphics applications where memory access patterns exhibit a great deal of spatial locality;
- Dedicated texture cache hardware provides:
  - Out-of-bounds index handling (clamp or wrap-around)
  - Optional interpolation (on-the-fly interpolation)
  - Optional format conversion
- could bring benefits if the threads within the same block access memory using regular 2D patterns, but you need appropriate binding;

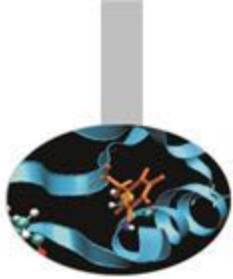
For typical linear patterns, global memory (if coalesced) is faster.



Texture Memory

- **Texture Memory** is afterall a remain of basic graphic rendering functionality needs
- as for constant memory, data actually reside in the global memory, fetched across dedicated texture-cache
- data is accessed in **read-only** using special CUDA API function, called **texture fetch**
- Specifications:
  - address resolution is more efficient since it is performed on dedicated hardware
- specialized hardware for:
  - out-of-bound address resolution
  - floating-point interpolation
  - type conversion or bit operations





Registers

- Just like CPU registers, access has no latency;
- used for scalar data local to a thread;
- taken by the compiler from the Streaming Multiprocessor (SM) pool and statically allocated to each thread;
  - each SM of a Fermi GPU has a 32KB register file, 64KB for a Kepler GPU
- *register pressure one of the most dangerous occupancy limiting factors.*

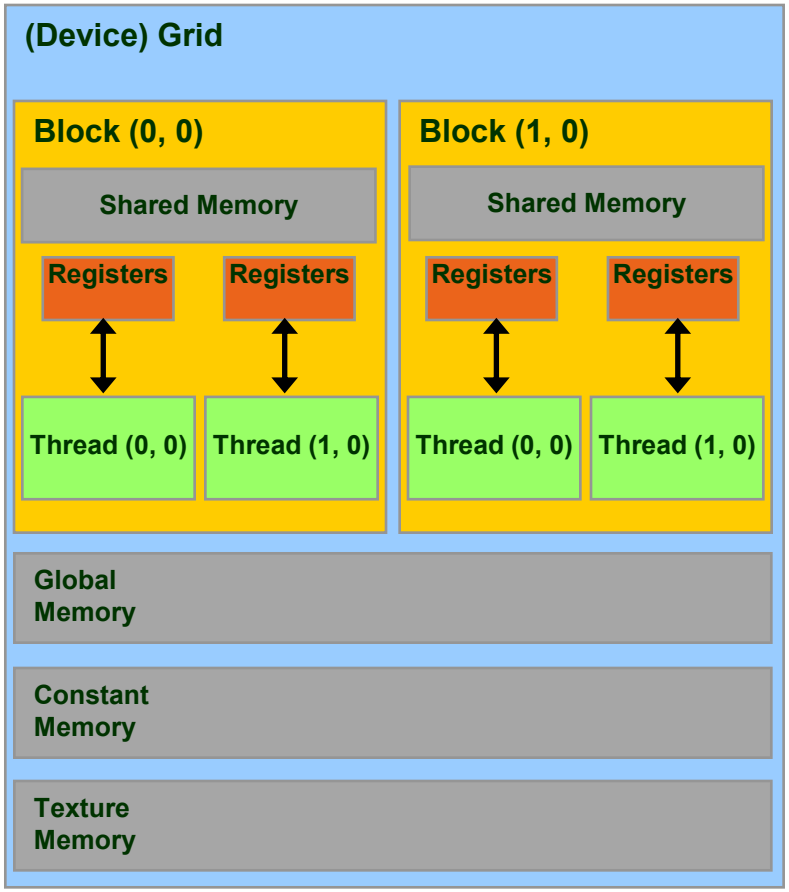
Registers

- **registers** are used to store scalar or small array variables with frequent access by each thread
  - **Kepler, Pascal, Volta** : 255 registers per thread
- WARNING:
  - the less registers a kernel needs, the more blocks can be assigned to a SM
  - pay attention to *Register Pressure*: can be a limiting factor for performances
  - the number of register per kernel can be limited during *compile time:*

    --maxregcount max_registers
  - the number of active blocks per kernel can be forced using the CUDA special qualifier

    __launch_bounds__

```
__global__ void __launch_bounds__
(maxThreadsPerBlock,  minBlocksPerMultiprocessor)
my_kernel( … ) { … }
```



Local Memory

- ***Local Memory*** does not correspond to a real physical memory place
- Automatic variables are often placed in local memory by the compiler:
  - large structures or arrays that would consume too much register space
- If a kernel uses more registers than available (register spilling), the compiler shall move variables into local memory
- Local memory is often mapped to global memory
  - using the same *Caching* hierachies (L1 for read-only variables)
  - facing the same latency and bandwidth limitation of global memory
- In order to obtain information on how much local, constant, shared memory and registers are required for each kernel, you can provide the following compiler options

`--ptxas-options=-v`

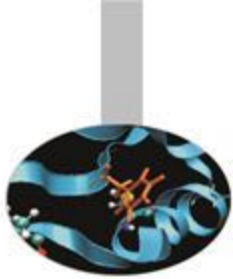
```
$ nvcc -arch=sm_60 -ptxas-options=-v my_kernel.cu
...
ptxas info : Used 34 registers, 60+56 bytes lmem, 44+40 bytes
smem, 20 bytes cmem[1], 12 bytes cmem[14]
...
```

Local Memory

- ***Local Memory*** does not correspond to a real physical memory place
- Automatic variables are often placed in local memory by the compiler:
  - large structures or arrays that would consume too much register space
- If a kernel uses more registers than available (register spilling), the compiler shall move variables into local memory
- Local memory is often mapped to global memory
  - using the same *Caching* hierachies (L1 for read-only variables)
  - facing the same latency and bandwidth limitation of global memory
- In order to obtain information on how much local, constant, shared memory and registers are required for each kernel, you can provide the following compiler options

`--ptxas-options=-v`

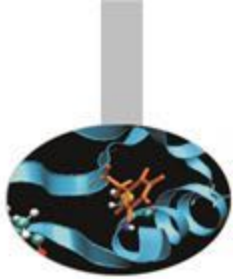
```
$ nvcc -arch=sm_60 -ptxas-options=-v my_kernel.cu
...
ptxas info : Used 34 registers, 60+56 bytes lmem, 44+40 bytes
smem, 20 bytes cmem[1], 12 bytes cmem[14]
...
```



Occupancy

The board's occupancy is the ratio of active warps to the maximum number of warps supported on a multiprocessor.

Keeping the hardware busy helps the warp scheduler to hide latencies.

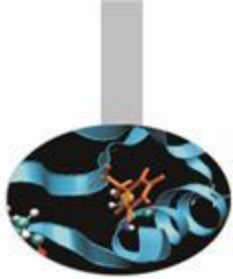


Occupancy: constraints

Every board's resource can become an occupancy limiting factor:

- shared memory allocated per block,
- registers allocated per thread,
- block size
  - (max threads (warp) per SM/max blocks per SM)

Given an actual kernel configuration, is possible to predict the maximum *theoretical occupancy* allowed.



Occupancy: block sizing tips

Some experimentation is required.

However there are some heuristic rules:

- threads per block should be a **multiple of warp size**;
- a minimum of **64 threads per block** should be used;
- **128-256 threads per block** is universally known to be a good starting point for further experimentation;
- prefer to split **very large** blocks into **smaller blocks**.

Three steps for a CUDA porting

1. identify data-parallel, computational intensive portions
   1. isolate them into functions (CUDA kernels candidates)
   2. identify involved data to be moved between CPU and GPU

2. translate identified CUDA kernel candidates into real CUDA kernels functions
   1. choose the appropriate thread index map to access data
   2. change code so that each thead acts on its own data

3. modify code in order to manage memory and kernel calls
   1. allocate memory on the device
   2. transfer needed data from host to device memory
   3. insert calls to CUDA kernel with execution configuration syntax
   4. transfer resulting data from device to host memory

Vector Sum

1. identify data-parallel computational intensive portions

```
int main(int argc, char *argv[]) {
  int i;
  const int N = 1000000;
  double u[N], v[N], z[N];

  initVector (u, N, 1.0);
  initVector (v, N, 2.0);
  initVector (z, N, 0.0);

  printVector (u, N);
  printVector (v, N);

  // z = u + v
  for (i=0; i<N; i++)
      z[i] = u[i] + v[i];

  printVector (z, N);

  return 0;
}
```

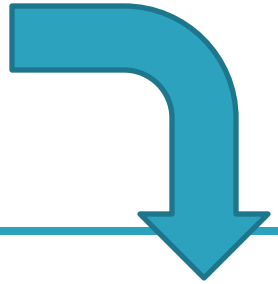
```
program vectoradd
integer :: i
integer, parameter :: N=1000000
real(kind(0.0d0)),dimension(N):: u, v, z

call initVector (u, N, 1.0)
call initVector (v, N, 2.0)
call initVector (z, N, 0.0)

call  printVector (u, N)
call  printVector (v, N)

! z = u + v
do i = 1,N
    z(i) = u(i) + v(i)
end do

call printVector (z, N)

end program
```

Vector Sum

2. translate the identified data-parallel portions into CUDA kernels

- each *thread* execute the same kernel, but acts on different data:
  - turn the loop into a CUDA kernel function
  - map each CUDA *thread* onto a unique index to access data
  - let each *thread* retrieve, compute and store its own data using the unique address
  - prevent out of border access to data if data is not a multiple of thread block size

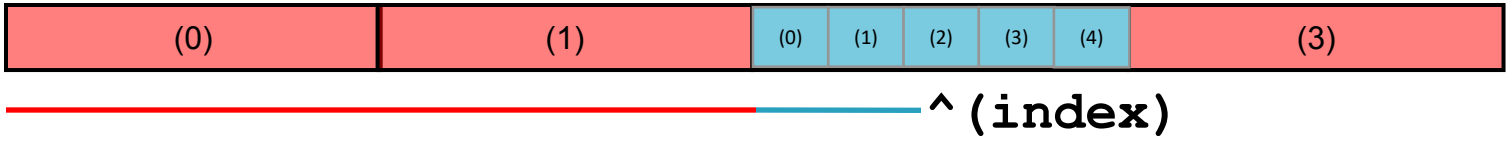
```
// z = u + v
for (i=0; i<N; i++)
    z[i] = u[i] + v[i];
```



```
__global__  void gpuVectAdd (int N, const double *u, const double *v, double *z)
{
    // index is a unique identifier of each GPU thread
    int index =  blockIdx.x * blockDim.x + threadIdx.x  ;
    if (index < N)
      z[index] = u[index] + v[index];
}
```

Vector Sum

2. translate the identified data-parallel portions into CUDA kernels




```
__global__ void gpuVectAdd (int N, const double *u, const double *v, double *z)
{
    // index is a unique identifier of each GPU thread
    int index =  blockIdx.x * blockDim.x + threadIdx.x  ;
    if (index < N)
      z[index] = u[index] + v[index];
}
```

The **__global__** qualifier declares this function to be a CUDA kernel

CUDA kernels are special C functions:

- can be called from host only
- must be called using the *execution configuration* syntax
- the return type must be *void*
- they are **asynchronous**: control is returned immediately to the host code
- an explicit synchronization is needed in order to be sure that a CUDA kernel has completed the execution

CUDA kernels

```
__global__ void add( int *a, int *b, int *c) {

 int index = threadIdx.x + blockIdx.x *
blockDim.x; // global thread id

 if (index <N)
   c[index] = a[index] + b[index];

}
```

- Kernel functions indicated in the code by __global__ (called by the host) or __device__ (called by another function on the device).
- Must be void - cannot return values
- Remember that every CUDA thread executes the code in the function. May need to use if statements to make sure unallocated memory is not accessed.

CUDA Function modifiers

CUDA extends C function declarations with three qualifier keywords.

Function declaration	Executed on the	Only callable from the
`__device__` (*device functions*)	device	device
`__global__` (*kernel function*)	device	host
`__host__` (*host functions*)	host	host

Vector Sum

3. manage memory transfers and kernel calls

- CUDA C API: `cudaMalloc(void **p, size_t size)`
  - allocates size bytes of GPU global memory
  - p is a valid device memory address (i.e. SEGV if you dereference p on the host)

```
double *u_dev, *v_dev, *z_dev;

cudaMalloc((void **)&u_dev, N * sizeof(double));
cudaMalloc((void **)&v_dev, N * sizeof(double));
cudaMalloc((void **)&z_dev, N * sizeof(double));
```

- in CUDA Fortran the attribute **device** needs to be used while declaring a GPU array. The array can be allocated by using the Fortran statement **allocate**:

```
real(kind(0.0d0)), device, allocatable, dimension(:,:) :: u_dev, v_dev, z_dev

allocate( u_dev(N), v_dev(N), z_dev(N) )
```

Vector Sum

- CUDA C API:

```
cudaMemcpy(void *dst, void *src, size_t size, direction)
```

  - copy size bytes from the src to dst buffer

```
cudaMemcpy(u_dev, u, sizeof(u), cudaMemcpyHostToDevice);
cudaMemcpy(v_dev, v, sizeof(v), cudaMemcpyHostToDevice);
```

- in CUDA Fortran you can rely on operator overload or use the array syntax to slice subelements of the array

```
u_dev = u ; v_dev = v
```

Vector Sum

3. manage memory transfers and kernel calls

Insert calls to CUDA kernels using the execution configuration syntax:

`kernelCUDA<<<numBlocks,numThreads>>>(...)`

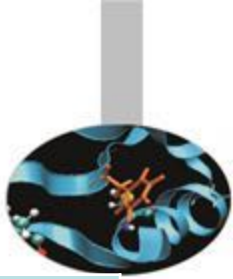
specifing the thread/block hierarchy you want to apply:

- **`numBlocks:`** specify grid size in terms of thread blocks along each dimension
- **`numThreads:`** specify the block size in terms of threads along each dimension

```
dim3 numThreads(32);
dim3 numBlocks( ( N + numThreads - 1 ) / numThreads.x );
gpuVectAdd<<<numBlocks, numThreads>>>( N, u_dev, v_dev, z_dev );
```

```
type(dim3) :: numBlocks, numThreads
numThreads = dim3( 32, 1, 1 )
numBlocks = dim3( (N + numThreads%x - 1) / numThreads%x, 1, 1 )
call gpuVectAdd<<<numBlocks,numThreads>>>( N, u_dev, v_dev, z_dev )
```

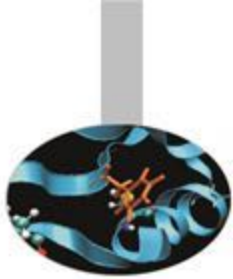
CUDA variable qualifiers



Variable declaration	memory	lifetime	scope
Automatic scalar variables	register	kernel	thread
Automatic array variables `__device__ __local__`	local	kernel	thread
`__device__ __shared__`	shared	kernel	block
`__device__`	global	application	grid
`__device__ __constant__`	constant	application	grid

- Global variables are often used to pass information from one kernel to another.
- Constant variables are often used for providing input values to kernel functions.

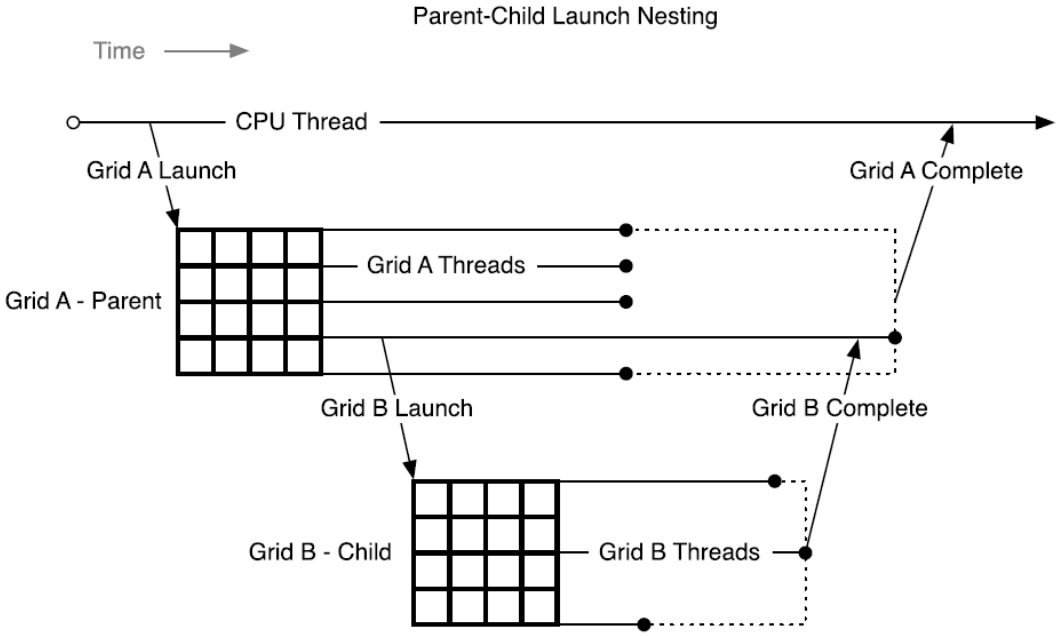
Kepler: dynamic parallelism



- One of the biggest CUDA limitations is the need to fit a single grid configuration for the whole kernel.

If you need to reshape the grid, you have to resync back to host and split your code.

- Kepler K20 (in addition to CUDA 5.x) introduced *Dynamic Parallelism*
- It enables a global kernel to be called from within another kernel
- The child grid can be *dynamically sized* and *optionally synchronized*




```
__global__ ChildKernel(void* data){
  //Operate on data
}

__global__ ParentKernel(void *data){
  ChildKernel<<<16, 1>>>(data);
}

// In Host Code:
ParentKernel<<<256, 64>>>(data);
```

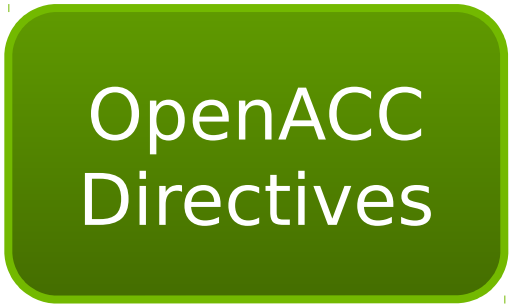
OpenCL

- Open Compute Language (OpenCL): “The Open Standard for Heterogeneous Parallel Programming”
  - Open cross-platform framework for programming modern multicore and heterogeneous systems
- Supports wide range of applications and architectures, including GPUs
  - Supported on NVIDIA Tesla + AMD FireStream
- See http://www.khronos.org/opencl/

OpenCL vs CUDA on NVIDIA

- NVIDIA support both CUDA and OpenCL as APIs to the hardware.
  - But put much more effort into CUDA
  - CUDA more mature, well documented and performs better
- OpenCL and C for CUDA conceptually very similar
  - Very similar abstractions, basic functionality etc
  - Different names e.g. “Thread” CUDA -> “Work Item” (OpenCL)
  - Porting between the two should in principle be straightforward
- OpenCL is a lower level API than C for CUDA
  - More work for programmer
- OpenCL obviously portable to other systems
  - But in reality work will still need to be done for efficiency on different architecture
- OpenCL may well catch up with CUDA given time

OpenACC Friendly Disclaimer



Easily Accelerate Applications

OpenACC does not make GPU programming easy. (...)

GPU programming and parallel programming is not easy. It cannot be made easy. However, GPU programming need not be difficult, and certainly can be made straightforward, once you know how to program and know enough about the GPU architecture to optimize your algorithms and data structures to make effective use of the GPU for computing. OpenACC is designed to fill that role.

(Michael Wolfe, The Portland Group)

OpenACC History

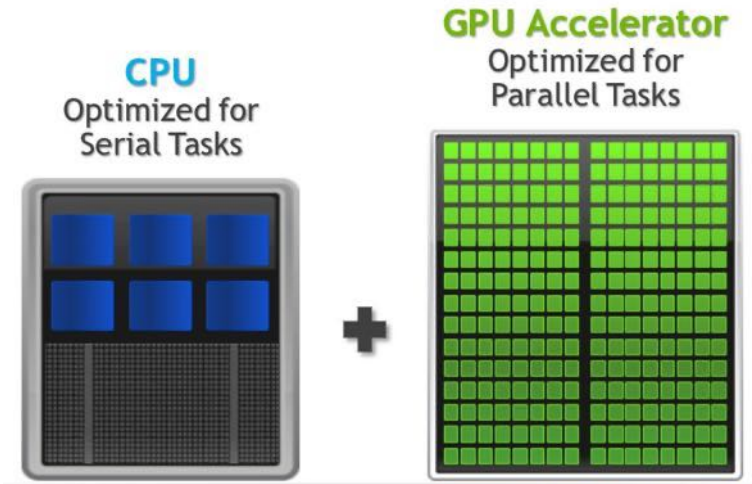
- OpenACC is a high-level specification with compiler directives for expressing parallelism for accelerators.
  - Portable to a wide range of accelerators.
  - One specification for Multiple Vendors and Multiple Devices
- OpenACC specification was released in November 2011.
  - Original members: CAPS, Cray, NVIDIA, Portland Group
- OpenACC 2.0 was released in June 2013
  - More functionality
  - Improve portability
- OpenACC 2.5 in November 2015
- OpenACC 2.6 in November 2017
- OpenACC had more than 10 member organizations

OpenACC Info & Vendors

- **http://www.openacc.org**
- Novelty in OpenACC 2.0 are significant
  - OpenACC 1.0 maybe not very mature...
- Some changes are inspired by the development of CUDA programming model
  - but the standard is not limited to NVIDIA GPUs: one of its pros is the interoperability between platforms
- Standard implementation
  - CRAY provides full OpenACC 2.0 support in CCE 8.2
  - PGI support to OpenACC 2.5 is almost complete (starting from version 15.1)
    - Suppurt for OpanACC 2.0 starting from 14.1
  - **GNU implementation effort ongoing** (there is a partial implementation in the 5.1 release and a dedicated branch for 7.1 realease)
- We will focus on PGI compiler
  - 30 days trial license useful for testing
- PGI:
  - all-in-one compiler, easy usage
  - sometimes the compiler tries to help you...
  - but also a constraint on the compiler to use

OpenACC – Simple, Powerful, Portable

```
main()
{

    <serial code>

    #pragma acc kernels
    //automatically runs on GPU
    {
        <parallel code>
    }
}
```

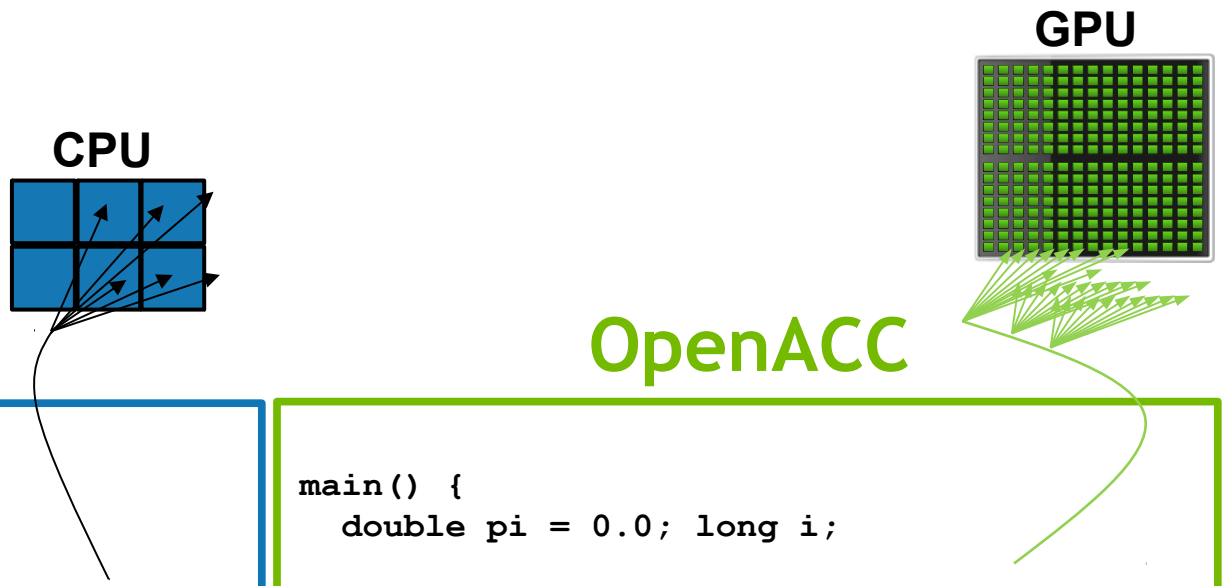


1. **Simple**:
   - Simple compiler directives
   - Directives are the easy path to accelerate compute intensive applications
   - Compiler parallelizes code
2. **Open**:
   - OpenACC is an open GPU directives standard, making GPU programming straightforward and portable across parallel and multi-core processors
3. **Portable**:
   - Works on many-core GPUs and multi-core CPUs
4. **Powerful**:
   - GPU Directives allow complete access to the massive parallel power of a GPU

OpenMP 4.0/4.5 alternative

- OpenMP 4.0/4.5 supports heterogeneous systems (accelerators/devices)
- What's new in OpenMP 4.x for support accelerator model
  - **Target regions**
    - Structured and unstructured target data regions
      - `omp target [clause[[,] clause],…]`
      - `omp declare target`
    - **Asynchronous** execution (`nowait`) and **data dependency** (`depend`)
  - Manage device data environment
    - **Data mapping** APIs
      - `map ([map-type:] list)`
    - **Data regions**
      - `omp target data [clause[[,] clause], …]`
      - `omp target enter/exit data [clause[[,] clause], …]`
  - **Parallelism & Workshare** for devices
    - `omp teams [clause[[,] clause],…]`
    - `omp distribute [clause[[,] clause],…]`
  - **SIMD** parallelism

Familiar to OpenMP Programmers



OpenMP

```
main() {
  double pi = 0.0; long i;

  #pragma omp parallel for reduction(+:pi)
  for (i=0; i<N; i++)
  {
    double t = (double)((i+0.05)/N);
    pi += 4.0/(1.0+t*t);
  }

  printf("pi = %f\n", pi/N);
}
```

OpenACC

```
main() {
  double pi = 0.0; long i;

  #pragma acc parallel loop reduction(+:pi)
  for (i=0; i<N; i++)
  {
    double t = (double)((i+0.05)/N);
    pi += 4.0/(1.0+t*t);
  }

printf("pi = %f\n", pi/N);
}
```